



UNIVERSITÀ DEGLI STUDI DI TRENTO

Dipartimento di Ingegneria e Scienza dell'Informazione

Appunti di Introduction to Computer Network Secutity

Lezioni, e Simulazioni d'esame

A cura di:

Mirco Negri

Versione Light Mode. Disponibile anche la [versione Dark Mode](#).

settembre 2026

Indice

1	Basic Notions	6
1.1	What is “*-Security”?	6
1.1.1	Modelling the Adversary	7
1.2	Trust and Dependability	7
1.3	Security Controls and Residual Risk	8
1.4	The CIA Triad	8
1.5	Vulnerability, Threat, and Risk	9
1.5.1	The Risk Matrix	10
1.6	Security Policy, Mechanism, and Service	10
1.7	Summary	11
2	Security in Perspective	11
2.1	Attacks, the CIA Triad, and Impact	11
2.1.1	Confidentiality Violations: Data Breaches	12
2.1.2	Integrity Violations: The FCC Bot Comments	12
2.1.3	Availability Violations: the Mirai Botnet	12
2.1.4	Growing Scale: WannaCry (2017)	13
2.1.5	The Attribution Problem	13
2.1.6	Further Difficulties	14
2.2	Examples of Security Services	14
2.3	Security vs. Privacy	14
2.3.1	Two Flavors of Privacy	15
2.3.2	When Security and Privacy Conflict	15
2.4	Privacy-Preserving Techniques	16
2.5	Risk-Based Privacy and Legal Constraints	16
2.5.1	GDPR Overview	16
2.5.2	GDPR Focus	16
2.6	Security and Human Factors	16
2.6.1	Usable Security	16
2.6.2	Awareness	17
2.6.3	Security Training Across Roles	17
2.7	Summary	17
3	Authentication I: Passwords and Digital Identity	17
3.1	A Brief History of Passwords	17
3.1.1	Timeline	18
3.2	User Authentication and the Digital Identity Lifecycle	18
3.2.1	Enrollment in Detail	19
3.2.2	Identity Assurance Levels (IAL)	19
3.3	Password Attacks and Mitigations	19
3.3.1	Password Space and Entropy	19
3.3.2	Common Attack Patterns	20
3.3.3	Mitigations	20
3.4	Protecting the Password File: Hashing and Salting	20
3.4.1	Requirements of a Cryptographic Hash Function h	20
3.4.2	Salting	21
3.5	Extensions of Password-Based Authentication	23

3.5.1	Multi-Factor Authentication (MFA)	23
3.5.2	Time-Based One-Time Password (TOTP)	24
3.5.3	Risk-Based (Adaptive) Authentication	24
3.5.4	NIST Authenticator Assurance Levels	24
3.5.5	MFA Fatigue Attacks	24
3.5.6	FIDO (Fast IDentity Online)	25
3.6	Outsourcing Identity Management	25
3.6.1	Summary	25
4	Cryptography I: Symmetric and Public-Key Cryptography	26
4.1	A Brief History of Cryptography	26
4.2	Basic Notions	26
4.2.1	Cryptography, Cryptanalysis, Cryptology	26
4.2.2	Security Services Provided by Cryptographic Mechanisms	26
4.2.3	The Cryptosystem	27
4.2.4	Kerckhoffs' Principle	27
4.2.5	Keys: Keyspace, Entropy, and Storage	28
4.2.6	Key Management and Key Distribution	28
4.2.7	Computational Security and Brute Force	28
4.2.8	Cryptography Is No Silver Bullet	29
4.3	Classical Ciphers: Substitution and Transposition	29
4.3.1	Substitution Ciphers: the Caesar Cipher and ROTk	29
4.3.2	The Vigenère Cipher	30
4.3.3	Transposition Ciphers: Columnar Transposition	31
4.4	Symmetric Key Cryptography	32
4.4.1	From Classical to Modern Cryptography	32
4.4.2	Definition and Notation	32
4.4.3	Stream Ciphers vs. Block Ciphers	32
4.4.4	The Data Encryption Standard (DES)	33
4.4.5	The Advanced Encryption Standard (AES)	34
4.5	Asymmetric (Public) Key Cryptography	36
4.5.1	History and Basic Idea	36
4.5.2	One-Way and Trapdoor One-Way Functions	36
4.5.3	Digital Signatures	37
4.6	RSA	39
4.6.1	Mathematical Background	39
4.6.2	The RSA Algorithm	40
4.6.3	Worked Example	41
4.6.4	History: Rivest, Shamir, Adleman	41
4.6.5	A Real Vulnerability: the Infineon RSA Key-Generation Flaw (2017)	41
4.6.6	Integrity with RSA: Signing and Verifying	42
4.7	Diffie–Hellman Key Exchange	42
4.7.1	Mathematical Background	42
4.7.2	The Protocol	42
4.7.3	Security Considerations and the Man-in-the-Middle Attack	43
4.7.4	History: Diffie and Hellman	43
4.8	Takeaways	44
4.9	Relative Security by Key Size	44

4.10	Summary	45
5	Cryptography II: Public Key Infrastructure and TLS	46
5.1	Public Key Cryptography, Revisited	46
5.1.1	Pros and Cons of PKC	46
5.1.2	Mitigating the Cons: Digital Signatures	46
5.1.3	Hash Functions: Recap and Their Limits	46
5.2	Public Key Infrastructure (PKI)	47
5.2.1	The Identity-Binding Problem	47
5.2.2	From Bulletin Boards to Digital Certificates	47
5.2.3	Digital Certificates	48
5.2.4	PKI Roles: CA, RA, VA, and Revocation	48
5.2.5	Obtaining a Certificate: the Certificate Signing Request (CSR)	49
5.2.6	Requirements for a Trustworthy PKI	49
5.3	SSL and TLS: Introduction	51
5.3.1	HTTPS: SSL/TLS in the Browser	51
5.4	TLS in Client-Server Architectures	52
5.4.1	Client-Server Architecture and TCP	52
5.4.2	The Transport Layer, in General	52
5.5	How TLS Works: the Handshake and Record Protocols	52
5.5.1	The Handshake Protocol, in a Nutshell	52
5.5.2	The Record Protocol, in a Nutshell	53
5.5.3	Two Companion Protocols	53
5.6	The TLS 1.2 Handshake, Step by Step	53
5.6.1	Certificate Verification, Revisited	55
5.7	How TLS Delivers the CIA Triad	56
5.7.1	Authentication	56
5.7.2	Confidentiality	56
5.7.3	Integrity	56
5.7.4	MAC, Revisited	56
6	Halfway Wrap-Up	56
6.1	Clarifying the Security Terminology Map	57
6.1.1	Mapping Attacker Actions onto the CIA Triad	57
6.1.2	Asset, Threat, Vulnerability, Risk: the Venn Version	57
6.2	Filling Two Small Gaps	58
6.2.1	Credential Stuffing	58
6.2.2	Two Refinements on Hashing and Salting	58
6.3	Case Study: How Passwords and TLS Combine in a Real Login	59
6.4	Case Study: Let's Encrypt and the Slow Rise of HTTPS	59
6.5	Case Study: the 2021 EU Digital COVID Certificate Key Compromise	60
6.6	Cross-Reference Index for the Recap Questions	60
7	Authentication II: Single Sign-On, SAML, SPID, and CIE 3.0	62
7.1	Single Sign-On (SSO)	62
7.1.1	Motivation: Password Fatigue	62
7.1.2	SSO: Basic Idea	62
7.1.3	SSO Properties	62
7.2	SAML: Overview	63

7.2.1	What SAML Is	63
7.2.2	SAML's Building Blocks	63
7.2.3	Entities: Identity Provider, Service Provider, Federation	64
7.2.4	Metadata and Trust Establishment	64
7.2.5	Worked Example: Alice, the Airline, and the Car Rental	64
7.2.6	Choosing an IdP: the Discovery Service	65
7.3	SAML: Assertions in Detail	65
7.3.1	Worked Example: an Authentication Assertion	65
7.3.2	Worked Example: an Attribute Assertion	66
7.4	SAML: Protocols and Bindings	67
7.4.1	Protocols	67
7.4.2	Bindings	67
7.5	SSO Within a Domain vs. Federated SSO Across Domains	68
7.6	The Web Browser SSO Profile, Step by Step	68
7.6.1	IdP-Initiated SSO	68
7.6.2	SP-Initiated SSO	69
7.7	SAML: Security and Privacy Considerations	69
7.7.1	Authentication Context and Assurance Levels	69
7.7.2	Metadata, Trust Bootstrapping, and Signing/Encryption in Practice	70
7.7.3	Trust, PKI, and TLS Recommendations	70
7.7.4	A Small Catalogue of SAML-Specific Attacks	70
7.7.5	Privacy Mechanisms	71
7.8	National Identity Infrastructures: SPID	71
7.9	National Identity Infrastructures: CIE 3.0	72
7.10	European Identity Infrastructure: eIDAS	73
7.10.1	Purpose and Architecture	73
7.10.2	Case Study: the 2019 eIDAS-Node Signature-Verification Bypass	73
7.10.3	Looking Ahead: Digital Identity Wallets	74
7.11	Summary	74
8	Access Control I: Basic Notions, the Access Matrix, and DAC/MAC/RBAC	76
8.1	Access Control: Basic Notions	76
8.1.1	Definition and Architecture	76
8.1.2	A Motivating Example: the Operating System	78
8.1.3	Policy, Model, Enforcement	78
8.2	The Access Control Matrix, ACLs, and Capabilities	79
8.2.1	The Access Control Matrix (ACM)	79
8.2.2	Domains of Protection	80
8.2.3	The Confused Deputy Attack	80
8.3	Discretionary Access Control (DAC)	81
8.3.1	Groups, for Manageable DAC	81
8.3.2	DAC: Pros and Cons	81
8.3.3	Worked Example: DAC Defeated by a Trojan	81
8.4	Mandatory Access Control (MAC) and Multi-Level Security	82
8.4.1	The Motivating Problem	82
8.4.2	Sensitivity Labels	82
8.4.3	Clearances and the Dominates Relation	82
8.4.4	No Read Up, No Write Down	83

8.4.5	The Tranquility Principle	83
8.4.6	Bell–La Padula and Its Shortcoming	83
8.5	Worked Example: a Military MLS Scenario	84
8.6	MAC: Pros, Cons, and Covert Channels	85
8.7	Closing DAC vs. MAC	86
8.8	Role-Based Access Control (RBAC)	86
8.8.1	Motivation	86
8.8.2	A University Example: the Naive Approach	87
8.8.3	A Better Answer: Roles, UA, and PA	87
8.8.4	An Even Better Answer: a Role Hierarchy	87
8.8.5	Core RBAC: the Formal Model	87
8.8.6	Worked Example: Core RBAC (No Hierarchy)	88
8.8.7	Hierarchical RBAC: the Formal Model	88
8.8.8	Worked Example: Hierarchical RBAC	89
8.8.9	Mathematical Digression: Binary Relations, Revisited	89
8.9	Constrained RBAC: Separation of Duty	90
8.10	RBAC: Pros and Cons	90
8.11	Summary	90

Course Overview

Lecturer: Silvio Ranise (Dept. of Mathematics, University of Trento & Center for Cybersecurity, FBK).

Aims of the Course

By the end of the course you should be able to:

- understand the **theoretical** and **practical** problems of information security;
- recognize **threats** to the fundamental security properties: **confidentiality**, **integrity**, and **availability**;
- understand how **authentication**, **authorization**, **access control**, and **cryptography** can be combined to mitigate vulnerabilities.

Course philosophy: the focus is on how to **defend** (i.e., mitigate risk), not on how to attack — although some basic attack techniques are shown to motivate the defenses.

Syllabus

1. Basic notions
2. Authentication
3. Introduction to cryptography
4. Applications of cryptography
5. Access Control
6. Application security
7. Privacy & GDPR

Exam

Written exam (questions and/or exercises); past papers are made available online.

1 Basic Notions

1.1 What is “*-Security”?

There is no single agreed-upon definition of **cybersecurity**, and the term overlaps with several related ones:

- **Information security:** preventing unauthorized access, use, disclosure, disruption, modification, or destruction of information, *regardless of its form* (electronic or physical).
- **Computer security:** protecting computer systems and information from harm, theft, and unauthorized use.

- **Network security:** physical and software measures protecting the underlying networking infrastructure.
- **Cybersecurity (NIST):** the ability to protect or defend the use of **cyberspace** from **cyber attacks**, where cyberspace is the interdependent network of information systems (Internet, telecom networks, computer systems, embedded controllers). A cyber attack targets an organization's use of cyberspace to disrupt, disable, destroy, or maliciously control a computing environment, or to steal/corrupt information.
- Information security substantially contributes to cybersecurity; computer and network security can be seen as *parts* of it — cybersecurity is spread over many domains (identity & access management, application security, risk management, governance & compliance, security operations, etc.).

1.1.1 Modelling the Adversary

Security is protection **against an adversary** (or, occasionally, against a random/physical process), and it typically focuses on **malicious** adversaries. Core to any security argument is modelling the attacker along three axes:

- **motivations** (historically glory, nowadays overwhelmingly money);
- **capabilities** (intercepting messages, modifying messages, reading keystrokes, ...);
- **threats**, i.e., the negative impact on systems, operations, organization, or business.

Claiming a system is “secure” without stating an attacker model is meaningless. Two illustrations:

- A **0-day attack** exploits a vulnerability that is *not yet known* to defenders — no model anticipated it, so no control was in place.
- **Absolute security does not exist**, not even for **air-gapped** systems (physically disconnected from any network). **Stuxnet** is the textbook example: a highly sophisticated worm, widely believed to have targeted Iranian uranium-enrichment centrifuges, that reportedly reached its air-gapped target via infected USB drives — showing that physical isolation alone is not a guarantee.

1.2 Trust and Dependability

- **Failure:** an event where the delivered service deviates from the correct service.
- **Dependability:** the ability to avoid failures that are more frequent or severe than acceptable.
- **Trust:** *accepted dependence* — i.e., choosing to rely on a component despite not being able to fully verify it.

Case study — SolarWinds (2020): attackers compromised the build infrastructure of SolarWinds' *Orion* network-monitoring platform and distributed **trojanized updates** to its users — a textbook **supply-chain attack** (a trojan being malware that misrepresents its true intent). The impact was enormous: 425 of the US Fortune 500, the top ten US

telecom companies, the top five US accounting firms, all branches of the US military, the Pentagon, the State Department, and hundreds of universities worldwide were affected.

On trusting trust: Ken Thompson, in his 1984 Turing Award lecture “*Reflections on Trusting Trust*,” argued that no amount of source-code review can fully protect you from code you did not write entirely yourself — ultimately, trusting software means trusting the *people* who built it. This applies directly to today’s software supply chains (3rd-party libraries, APIs, cloud services, mobile, edge).

1.3 Security Controls and Residual Risk

To mitigate threats, security relies on **controls** (or mitigation strategies) affecting three dimensions:

- **people** (e.g., rules for setting passwords);
- **process** (e.g., regularly updating software);
- **technology** (e.g., authentication and access control).

Controls are further classified by **when** they act:

- **Preventive:** before the bad event (e.g., locking out unauthorized intruders).
- **Detective:** during the bad event (e.g., an intruder alert on a dashboard).
- **Corrective:** after the bad event (e.g., restoring deleted data from backup).

Selecting controls is driven by **risk management** — identifying vulnerabilities/threats to an organization’s information resources and deciding what countermeasures (if any) reduce risk to an acceptable level — while always considering the **human factor**: controls must be easy to use and must not demand stress or memorizing long rule sets (a classic satirical illustration of *ignoring* this principle is Dilbert’s “Mordac, the Preventer of Information Services,” who insists security matters more than usability).

Residual risk: security controls are human artifacts — they mitigate an attack’s impact but rarely eliminate it completely, and they can themselves **contain vulnerabilities**. What is left over is the **residual risk**: the danger remaining after controls are applied. *Analogy:* seatbelts reduce the risk of injury but do not cancel it — accidents can still be severe, and a defective installation reduces the mitigation further.

1.4 The CIA Triad

Violating security can mean different things — sharing patient data without authorization, changing a bank balance without authorization, or making a network service unavailable. These correspond to three properties, collectively the **CIA triad**:

- **Confidentiality:** preserving authorized restrictions on information access and disclosure (covering data *at rest*, *in processing*, and *in transit*), while still permitting authorized sharing.
- **Integrity:** guarding against improper, unauthorized, or undetected **modification or destruction** of information, while permitting authorized modification. It also underpins two related properties:

- **Authenticity**: confidence that information/a message is genuine and its origin can be verified and trusted.
- **Non-repudiation**: protection against an individual falsely denying having performed an action (e.g., sending a message, approving a document).
- **Availability**: ensuring timely, reliable access to data and services for **authorized** users, and protection against unauthorized deletion or denial of service.

Esercizio 1

(Tema d'esame, 19 febbraio 2026 — 20 punti) Describe the CIA triad properties along with the authenticity and non-repudiation properties; give for each one an example.

Soluzione:

- **Confidentiality** — prevents unauthorized disclosure. *Example*: a healthcare record must only be readable by the treating physician and the patient, not by unrelated hospital staff.
- **Integrity** — prevents unauthorized/undetected modification. *Example*: the balance of a bank account must not be alterable by anyone other than authorized transactions.
- **Availability** — ensures authorized users can access a resource when needed. *Example*: a hospital's patient-monitoring system must remain reachable even under a denial-of-service attempt.
- **Authenticity** — confidence that a message/document genuinely originates from the claimed source. *Example*: verifying that a signed contract PDF was really produced by the counterparty and not forged.
- **Non-repudiation** — the sender of a message cannot later deny having sent it. *Example*: in e-commerce, a digitally signed purchase order should be provable as having come from the buyer, so they cannot deny placing the order.

1.5 Vulnerability, Threat, and Risk

- **Vulnerability**: a weakness in an information system, its security procedures, internal controls, or implementation that can be exploited or triggered by a threat source. *Examples*: hidden backdoors (e.g., allegations around Huawei networking equipment), unknown software bugs such as **buffer overflows** (writing past a buffer boundary can overwrite a function's return address, redirecting execution to attacker-supplied shellcode), and weak/common passwords.
- **Threat**: any circumstance or event — deliberate or accidental — with the potential to adversely impact operations, assets, or individuals via unauthorized access, disclosure, modification, or denial of service; equivalently, the potential for a threat-source to exploit a specific vulnerability. *Examples*: **hackers** (breaking/sniffing passwords, social engineering, denial-of-service by flooding a service with bogus requests); **viruses** (self-replicating, require user action to activate, e.g., opening an email attachment); **worms** (self-replicating, *no* user action needed — they propagate over the network on their own, find vulnerable machines, and spread further).

- **Risk:** the probability that a particular threat will exploit a system vulnerability — a function of (i) the **likelihood** of occurrence (driven by threats' *intent* + *capability*, and vulnerabilities' ease of *identification* + *exploitation*) and (ii) the **impact**, which must be evaluated *per stakeholder*: e.g., an unauthorized disclosure of personal health data can be catastrophic for the patients involved, yet negligible for the hospital if few records were exposed.

In short: risk = likelihood × impact.

1.5.1 The Risk Matrix

Likelihood and impact severity are each divided into 5 discrete levels; multiplying them gives a risk score used to prioritize response:

Likelihood \ Impact	Negligible (1)	Marginal (2)	Serious (3)	Critical (4)	Catastrophic (5)
Highly Probable (5)	5	10	15	20	25
Probable (4)	4	8	12	16	20
Occasional (3)	3	6	9	12	15
Remote (2)	2	4	6	8	10
Improbable (1)	1	2	3	4	5

Case study — the 2015 Jeep hack: researchers remotely compromised a Fiat Chrysler Jeep over its cellular connection, ultimately controlling steering/brakes — a vivid illustration that **message-integrity** on the car's internal **CAN bus** was the root problem (any ECU on the bus could be made to accept forged messages, and the industry has no easy path to replace CAN bus for better security). Impact was severe for every stakeholder: the manufacturer had to **recall 1.4 million vehicles**, and drivers'/passengers' safety was directly put at risk.

Esercizio 2

(Tema d'esame, 9 giugno 2026 — 10 punti) Define the notions of vulnerability and threat and give (at least) an example for each one.

Soluzione: see the definitions above. *Vulnerability example:* a buffer-overflow bug in a network service. *Threat example:* a worm that automatically scans the network for machines with that unpatched bug and infects them without any user interaction — the worm is the threat, the unpatched buffer overflow is the vulnerability it exploits.

1.6 Security Policy, Mechanism, and Service

- **Security policy:** the rules and requirements established by an organization governing the acceptable use of its information/services, and how confidentiality, integrity, and availability are to be protected.
- **Security mechanism:** a device or function implementing a security policy, providing one or more security services (rated by strength and design assurance).
- **Security service:** a capability supporting one or more CIA requirements — e.g., **key management, access control, authentication**.

Example policy skeleton (adapted from a real-world template):

- *Purpose*: protect restricted/confidential/sensitive data from loss, avoiding reputational damage and customer impact; primary goal is user awareness and preventing accidental data leakage.
- *Scope*: any employee, contractor, or individual with system/data access; defines what counts as protected data (personal, financial, intellectual property).
- *Example rules*: complete security-awareness training; escort visitors at all times; enforce a clean-desk policy; use secure, unique passwords per the password policy; terminated employees must return all records containing personal information.

Concrete examples of security **mechanisms**:

- **Authentication**: verifying the identity of a user, process, or device, typically a prerequisite for granting access.
- **Authorization**: granting or denying access rights to a user, program, or process.
- **Access control**: the combination of authentication + authorization; the process of granting/denying specific requests to obtain/use information (or to enter a physical facility).

1.7 Summary

- For every scenario, the CIA triad must be carefully instantiated; **policies** specify CIA requirements, and **mechanisms/services** enforce those policies.
- The causal chain is: a **security control** affects a **threat**, which **exploits** a **vulnerability**, which **leads to risk** — risk is in turn **mitigated by** the control (impact always evaluated per stakeholder).
- Common attack categories previewed here (details in later lessons): malware, phishing, cross-site scripting, HTTP floods, zero-day exploits, man-in-the-middle, DoS/DDoS, SQL injection.
- Security interacts with **usability** and **performance** (often a three-way trade-off), should be embedded across the whole **software development lifecycle** (planning, analysis, design, implementation, maintenance), and must account for human/organizational factors (stakeholders, customers, users are overlapping but distinct groups).

2 Security in Perspective

2.1 Attacks, the CIA Triad, and Impact

Recall the **CIA triad** (Section 1.4). Real-world incidents show how each property is violated in practice, and how the resulting impact can be severe.

2.1.1 Confidentiality Violations: Data Breaches

Large-scale data breaches (e.g., Facebook 533M records, Equifax, Experian Brazil 220M, Microsoft 250M, Marriott International, Aadhaar 1.1 billion) illustrate **violation of confidentiality** at massive scale. Impact is twofold:

- **Privacy:** disclosure of personal information, including sensitive healthcare data;
- **Economy:** attackers can perform financial transactions on the victim's behalf.

2.1.2 Integrity Violations: The FCC Bot Comments

In **December 2017**, the US Federal Communication Commission's *net neutrality* public comment form received roughly **2 million identical comments**, submitted under the names and addresses of real people — generated by bots. The fraud was discovered only because some of the impersonated people reported that the names used belonged to **deceased** family members and friends. This is a **violation of integrity**: the impact is on the **trustworthiness** of resources and services, since bots can manipulate social/democratic processes by mass-posting content under false attribution.

2.1.3 Availability Violations: the Mirai Botnet

Mirai is an IoT botnet whose timeline illustrates the scale attacks can reach:

- **Aug 2016:** Mirai surfaces.
- **Sep 18, 2016:** OVH (hosting provider) attacked at **600 Gbps**.
- **Sep 21, 2016:** peak attack on KrebsOnSecurity, the site of security journalist Brian Krebs — who afterwards described the episode as “*The Democratization of Censorship*”: anyone can now rent enough compromised IoT devices to silence almost any target online.
- **Sep 30, 2016:** Mirai source code released publicly.
- **Oct 21, 2016:** attack on **Dyn** (a major DNS provider) makes popular sites such as **Amazon, Twitter, and PayPal** unreachable.
- **Oct 31, 2016:** attack on **Lonestar**, a Liberian telco — shutting down Internet access for an **entire country**.
- **Jan/Feb 2017:** Mirai's author identified and arrested.

Geographic distribution of infected devices (share of total):

Country	Share
Brazil	15.0%
Colombia	14.0%
Vietnam	12.5%
China	6.5%
South Korea	6.0%
Russia	4.7%
Turkey	4.2%
India	4.1%
Taiwan	3.5%
Argentina	2.2%

This is a **violation of availability**: impact is manifold — **economic** (the longer the downtime, the larger the loss, up to entire national ecosystems) and on **fundamental rights** (censorship-like effects are inherently damaging to democratic life).

2.1.4 Growing Scale: WannaCry (2017)

The **WannaCry** ransomware hit roughly **200,000 computers across 150 countries**, demanding payment to unfreeze encrypted machines:

- **UK**: National Health Service disrupted, hospitals/clinics lost access to computers.
- **France**: carmaker Renault forced to halt production at several sites.
- **Germany**: Deutsche Bahn (rail) suffered disruption of electronic displays at train stations.
- **Spain**: mobile operator Telefónica and utility/power firm Gas Natural affected.
- **Russia**: around 1,000 Interior Ministry computers infected; Megafon (largest mobile operator) and Sberbank (biggest bank) also hit.
- **US**: delivery company FedEx struck.
- **Japan**: about 2,000 computers at 600 companies.
- **China**: more than 29,000 institutions, including universities, railway stations and government services; payment systems at state-oil-company petrol stations cut off.

Russia, Ukraine, India, and Taiwan were the most seriously affected countries.

Key takeaway: cyberattacks are **borderless**, and to be effective they try to reach as many victims as possible.

2.1.5 The Attribution Problem

A CNBC (2016) headline captures a core difficulty: *“Most hacks take minutes to do — and weeks to discover.”* As Hunker, Hutchinson & Margulies (2008, Dartmouth College) put it: without the fear of being caught, convicted, and punished, individuals and organizations will continue to conduct malicious activity online. This creates a cycle: **anonymity** → **low deterrence** → (reinforces) **anonymity**, since weak attribution removes the disincentive to attack.

At EU level, this motivated the creation of **ENISA** (European Union Agency for Network and Information Security). As European Commission President Jean-Claude Juncker stated in his 2017 State of the Union address: cyber-attacks can be more dangerous to the stability of democracies and economies than guns and tanks, since they know no borders and no one is immune.

2.1.6 Further Difficulties

- **Growing attack surface:** how do you evaluate risk from 3rd-party software, from infrastructure interdependencies, or from dependencies on the physical world? *Illustrative case:* an internet-connected fish-tank thermometer was reportedly used as the entry point to breach a casino's network — a vivid example of how an innocuous IoT device can widen the attack surface of an entire organization.
- **Cyber-intelligence sharing:** it is difficult to exploit threat information from other stakeholders, and equally difficult to contribute one's own information back to others in a trusted way.

2.2 Examples of Security Services

Recall: a **security service** is a capability supporting one or more CIA requirements. Two examples:

- **Access control** (the combination of **authentication** + **authorization**) is typically used to guarantee **confidentiality** and **integrity** of data *at rest*.
- **TLS** (Transport Layer Security) guarantees **confidentiality** and **integrity** of data *in transit*, and is the cornerstone of web security.

HTTP vs. HTTPS example: if Helen sends her password `abc123` over plain **HTTP**, an eavesdropper sees it in clear (`abc123`). If Carol sends the same password over **HTTPS**, an eavesdropper only sees the encrypted form (e.g., `xyaerXzabc`) — illustrating concretely why TLS matters.

2.3 Security vs. Privacy

Security and privacy are **overlapping but different**:

- **Privacy** concerns: collection of personal information, using/disclosing it only in an authorized manner, data quality, and access to one's own personal information.
- **Security** (CIA) concerns, more broadly: **confidentiality** (data safe from unauthorized access/use), **integrity** (data reliable and accurate), **availability** (data available when needed) — for *any* data, not only personal data.
- Their overlap is exactly the **protection of personal information**.

2.3.1 Two Flavors of Privacy

1. **Privacy as controlled sharing:** who gets to see, use, use-for-a-purpose, or receive-by-transfer which information.
2. **Privacy as confidentiality:** making the task of **correlating** data/actions back to an individual very difficult.

Esercizio 1

(Tema d'esame, 5 settembre 2025 — punti (a) e (c) di una domanda da 40 punti)
(a) Give two possible definitions of privacy and briefly comment on them — are they overlapping? If not, how do they differ? (c) Explain whether using TLS protects client and server privacy by preventing a third party (who is not the client or server) from reading the transmitted messages, and indicate which definition from (a) applies here. (Il punto (b), sugli attacchi di linkage, lo tratteremo nella lezione sulla Privacy e il GDPR.)

Soluzione:

- (a) The two definitions are *controlled sharing* and *confidentiality* (above). They are **related but not equivalent**: controlled sharing is about **consent/authorization** — who is allowed to receive which data; confidentiality is about **unlinkability** — how hard it is to correlate data back to an identity, *regardless* of whether the original sharing was authorized. The two can come apart: data can be shared with full consent (controlled sharing satisfied) yet later be trivially re-identified if leaked (confidentiality violated); conversely, aggregated data might be hard to re-identify (confidentiality satisfied) even without a specific, granular consent choice having been made (controlled sharing not really exercised).
- (c) **Yes.** TLS encrypts the channel, so a third party who is neither the client nor the server cannot read the plaintext content — exactly the HTTP-vs-HTTPS example above. This matches definition **(2), privacy as confidentiality**: TLS makes it computationally infeasible for an outside eavesdropper to correlate the observed traffic with its actual content. It does *not* address definition (1) (controlled sharing), since that concerns the client–server relationship itself (what the server is authorized to receive), which TLS does not regulate. *Caveat: TLS hides content, not metadata — an observer may still infer which server is contacted, connection timing, and traffic volume.*

2.3.2 When Security and Privacy Conflict

Non-repudiation (part of integrity) protects against an individual falsely denying having performed an action (creating information, sending/approving/receiving a message) — desirable when signing a contract or in e-commerce. But in other contexts it can **harm privacy**: e.g., in **e-voting** or in *off-the-record* conversations, one may specifically *want* the opposite guarantee. The opposite of repudiation is **plausible deniability**: ensuring that an instance of communication leaves behind *no unequivocal evidence* that it took place.

Key point: a security requirement can, in some contexts, be seen as a *violation* of a privacy requirement, and vice versa.

2.4 Privacy-Preserving Techniques

Privacy-preserving techniques roughly correspond to security services — some security services (notably from cryptography) double as privacy-preserving techniques.

- **Privacy as confidentiality — end-to-end encryption:** Bob obtains Alice’s public key from a Certificate Authority and encrypts his message with it; an intermediary server only ever handles the encrypted, unreadable form; Alice decrypts it with her private key. No one in between (including the server) can read the content.
- **Privacy as control — access control:** the same authentication/authorization/guard/policy/audit log mechanism used for general security also supports privacy, by ensuring that only entitled employees of a company can access customers’ personal information.

2.5 Risk-Based Privacy and Legal Constraints

Key distinction from general risk (Section 1.4): in risk-based privacy, impact is always evaluated specifically **on the end-user** whose personal data is being processed — this is exactly the logic behind the **Data Protection Impact Assessment (DPIA)**.

2.5.1 GDPR Overview

- **Tough penalties:** fines of up to **4% of annual global revenue or €20 million**, whichever is **greater**.
- **Extraterritorial scope:** the regulation also applies to **non-EU companies** that process personal data of individuals in the EU; international transfers of data continue to be governed by EU GDPR rules.
- **Broader definition of personal data:** now includes identifiers such as **genetic, mental, cultural, economic, and social-identity** data — not just names or ID numbers.

2.5.2 GDPR Focus

- **Privacy risk impact assessments (DPIAs)** are required for projects where privacy risks are high.
- Products, systems, and processes must consider **privacy-by-design** during development.
- Certifications such as **ISO 27001** help demonstrate “adequate technical and organizational measures” to protect people’s data and systems.

Guiding philosophy: “*It’s not a matter of **if** a cybersecurity breach will happen, but **when**.*” This is precisely why both cybersecurity and the GDPR adopt a **risk-based approach**.

2.6 Security and Human Factors

2.6.1 Usable Security

NIST’s usable-cybersecurity principle: “*Make it Easy to Do the Right Thing, Hard to Do the Wrong Thing, And Easy to Recover When the Wrong Thing Happens.*”

2.6.2 Awareness

Even in 2020, the **worst (most common) passwords** were still dominated by trivial choices: 123456, 123456789, picture1, password, 12345678, 111111, 123123, 12345, 1234567890, *senha* — direct evidence that awareness campaigns are still needed. One such campaign frames it memorably: “*Treat your passwords like your underwear*” — change them regularly, keep them off your desk, never share them with anyone.

2.6.3 Security Training Across Roles

Security knowledge needs differ by role, forming a spectrum:

- **Require deeper security knowledge:** Security Architect (design/implement/improve policies for users, hardware, software, networks), Security Engineer (implement/enforce hardware, software, and user security policies), Security Analyst (identify security issues via testing/simulation).
- **Middle of the spectrum:** Network Admin (secure the physical/digital network infrastructure), System Admin (secure processes on critical systems such as servers/workstations), Developers (build software according to security policies/practices).
- **Require basic security knowledge:** Management & C-Suite (must factor security into business decisions), Everyone Else (must understand basic secure practices).

Problem: IT professionals are increasingly making decisions with real security (and privacy) consequences, but on average are **not trained** to do so — resulting in a weak security posture that offers attackers an “easy lunch.”

2.7 Summary

- Attacks violate one or more CIA properties; identifying *which* helps understand the impact and choose the right security services/mechanisms for mitigation.
- Ensuring security is hard because of: the **attribution** problem, the **scale** and **borderless** nature of attacks, the **growing attack surface**, and **lack of collaboration** among victims.
- Security and privacy **overlap** but are different, and can sometimes **conflict**.
- Privacy-preserving techniques matter from a **legal** standpoint too — the GDPR encapsulates a risk-based approach to privacy.
- Security is **not purely technical**: human factors play a crucial role, hence the importance of security **awareness** and **training**.

3 Authentication I: Passwords and Digital Identity

3.1 A Brief History of Passwords

Authentication is the process of verifying a user’s identity, typically as a prerequisite to granting access to a system’s resources. A secure system generally needs to track user identities for two reasons:

- the user’s identity is recorded when logging security-relevant events in an **audit trail**;
- the user’s identity is a parameter used in **access control** decisions.

Passwords have dominated human-computer authentication for over half a century, despite a rising consensus that more **secure** and **user-friendly** alternatives should be found. This creates a duality: security **researchers** tend to push for strict password composition policies, while **practitioners** focus on the practical problem of protecting accounts and data — and in practice there is little evidence that strict composition rules actually reduce security incidents.

3.1.1 Timeline

- **1960s**: passwords were introduced in time-sharing operating systems mainly to protect against pranks or abuse of shared resources. Security issues appeared almost immediately: passwords could be guessed, and the master password file was often stored *in clear*, allowing leaks.
- **1970s**: passwords started protecting both resources *and* sensitive data. This is when **hashing** began to be used to protect stored passwords, later complemented by **salting** (both discussed in Section 3.4).
- **Morris Worm (1988)**: a landmark event exploiting weak/guessable passwords, which pushed the industry toward hashed and access-controlled password files.
- **Today**: attempts to build a business around consumer authentication have had limited success. Hardware tokens for second-factor authentication never saw wide adoption due to cost; smartphones may change this, since they can generate or receive one-time passwords without extra dedicated hardware. National digital identity infrastructures (e.g., Italy’s **SPID**, *Sistema Pubblico di Identità Digitale*) have seen uneven adoption due to unclear business models for identity providers — SPID itself only became widely adopted after the COVID-19 pandemic.

3.2 User Authentication and the Digital Identity Lifecycle

An **identity** is a set of attributes related to an entity (e.g., a person or an organization); a **digital identity** is an identity whose attributes are stored and transmitted digitally.

The digital identity lifecycle has four phases:

1. **Enrollment / onboarding**: an applicant applies to become a subscriber of an identity system, which must validate the applicant’s identity. At the end, the user receives a credential/authenticator from a **Credential Service Provider (CSP)**.
2. **Authentication**: verifying the identity of a user, process, or device.
3. **Authorization / Access Control**: checking the user’s permissions, typically automated by evaluating the subject’s attributes.
4. **Deregistration**: ends the relationship between the entity and the provider.

3.2.1 Enrollment in Detail

Enrollment starts with collecting **Personally Identifiable Information (PII)** — at least full name, date of birth, and home address — plus at least two forms of identity evidence (ID card, passport, driving licence). It proceeds through three steps:

- **Resolution:** uniquely distinguishing the applicant within a population or context;
- **Validation:** checking the **authenticity, validity, and accuracy** of the supplied information against an authoritative source;
- **Verification:** confirming the **linkage between the claimed identity and the real-life existence** of the subject (e.g., checking in person that the applicant matches the ID photo, or confirming possession of a claimed phone number).

3.2.2 Identity Assurance Levels (IAL)

- **IAL 1:** weakest; attributes are self-asserted (or treated as such).
- **IAL 2:** requires either **remote or in-person** identity proofing.
- **IAL 3:** strongest; requires **in-person** proofing, with attributes verified by an authorized representative examining physical documents.

Note: a weak enrollment (e.g., IAL 1) undermines every later authentication event, no matter how strong the login mechanism itself is — identity proofing and authentication are two distinct security problems.

3.3 Password Attacks and Mitigations

3.3.1 Password Space and Entropy

A password’s resistance to brute force depends on its **password space**, which grows exponentially with length and character-set size (figures below for an 8-character password):

Character set	Combos/char	Space (8 chars)
Lowercase only	26	$26^8 = 208,827,064,576$
Lower + upper case	52	$52^8 = 53,459,728,531,456$
Lower + upper + digits	62	$62^8 = 218,340,105,584,896$
Lower + upper + digits + symbols	≈ 92	$92^8 = 5,132,188,731,375,616$

Despite these large numbers, modern GPU cloud computing makes short passwords cheap to crack: a widely cited 2019 estimate put the cost of cracking an **average 8-character password at about \$25** using rented AWS P3 GPU instances, in **12 minutes or less**. This is why guidance has shifted from “short but complex” toward simply **longer**.

Outdated advice: the classic 2003 NIST recommendation (uppercase + numbers + symbols + frequent forced changes) is now considered a mistake: it produces passwords that are *hard for humans to remember* but *easy for computers to guess* (predictable substitutions like Tr0ub4dor&3), whereas a long passphrase of unrelated common words (e.g., **correct horse battery staple**) can reach comparable or higher entropy while being far easier to memorize. Bill Burr, author of the original 2003 NIST guidance, publicly stated in 2017 that he regretted it.

3.3.2 Common Attack Patterns

- **Default credentials:** many devices ship with defaults (`admin/admin`); attackers keep large default-credential lists, which is how IoT botnets like Mirai compromise devices at scale.
- **Dictionary attacks:** trying passwords from lists of commonly used passwords, freely available online for almost every language.
- **Spoofing:** an attacker leaves a program showing a **fake login screen** and walks away. Any victim who types credentials into it has them captured; the program then shows a bogus error and returns control to the real OS login, so the victim assumes it was a glitch.
- **Phishing:** password authentication is **unilateral** — the system verifies the user, but the user has no cryptographic guarantee about who is receiving the password — so a user can be misled into sending it to a fake endpoint.

3.3.3 Mitigations

- **Always set a password** — with none, there is nothing to guess.
- **Change default passwords** immediately after deployment.
- **Avoid guessable passwords:** minimum length + mixed character classes (mindful of the entropy/memorability trade-off above).
- **Forbid commonly used / breached passwords.**
- **Avoid password hints** and knowledge-based recovery questions (often guessable).
- **Limit login attempts** to slow down online brute-force/dictionary attacks.

Password managers/generators trade one problem for another: a randomly generated 10–16 character password removes guessability, but is now unmemorable — the manager itself becomes a high-value target, especially if it syncs online.

3.4 Protecting the Password File: Hashing and Salting

Instead of storing passwords in clear, systems store the output of a **one-way function**. A function f is **one-way** if $f(x)$ is easy to compute given x , but it is computationally hard to recover x given only $y = f(x)$. At login, the system applies f to the entered password x' and compares $f(x')$ to the stored $f(x)$.

3.4.1 Requirements of a Cryptographic Hash Function h

- **Ease of computation:** $h(x)$ is easy to compute from x .
- **Compression:** h maps inputs of *arbitrary* length to outputs of *fixed* length n .
- **One-way:** given y , it is infeasible to find x with $h(x) = y$.
- **Weak collision resistance:** given x , infeasible to find $x' \neq x$ with $h(x') = h(x)$.

- **Strong collision resistance:** infeasible to find *any* pair $x \neq x'$ with $h(x) = h(x')$.

A **collision** is a pair $x \neq x'$ with $h(x) = h(x')$. Collisions always exist in theory (inputs are unbounded, outputs are only 2^n), but a good hash makes them **negligibly unlikely**: for $n = 128$, there are $2^{128} \approx 3.4 \times 10^{38}$ possible hashes, so — assuming uniformly distributed outputs — finding a collision by chance is less likely than one person winning the lottery repeatedly. That uniformity is exactly what is hard to guarantee by design, and has been the weak point exploited historically:

- **MD4:** weak; meaningful collisions are feasible.
- **MD5:** former Internet standard, now **broken**.
- **SHA-1:** 160-bit output; **collision attacks reported**, no longer recommended.
- **RIPEND-160:** still used by some European providers.
- **SHA-256:** recommended today for longer hash values.

3.4.2 Salting

Hashing alone is still vulnerable to **dictionary/rainbow-table attacks**: an attacker precomputes hashes of every dictionary password once, then looks up any stolen hash instantly. A **salt** is a fixed-length **random** value concatenated with the password *before* hashing:

- two users with the *same* password get *different* stored hashes;
- an attacker must redo the precomputation *per salt* (e.g., classic Unix used a 12-bit salt), multiplying the attack cost by the number of distinct salts in the file.

A typical password-file entry (e.g., Unix `/etc/shadow`) has the structure:

```
username : salt+hash : last_change : min_days : max_days : warn_days
          : inactive_days : expire_date
```

Common misconception: the salt does *not* need to be secret — it is normally stored *in clear*, right alongside the hash. Its security value comes from being **random and unique per user**, not from being hidden; this is different from a *pepper* (a separate, genuinely secret value).

Esercizio 3.1

(Tema d'esame, 9 giugno 2026 — 50 punti) Explain how to protect a password file with hashing and salting. Discuss: (a) what hashing means; (b) whether hashing alone is secure enough, and why; (c) what salting means; (d) whether salts can be stored in clear; (e) now imagine that, at each login, the system generates a random private key, encrypts the password with a block cipher, stores the resulting ciphertext, and deletes the key. Does this make the login process secure? Why?

Soluzione:

- Hashing means applying a one-way cryptographic hash function h to the password and storing only the digest $h(x)$, never the plaintext.

- (b) No. Hashing alone stops brute force from being trivially fast, but it does *nothing* against **dictionary attacks**: identical passwords always produce identical hashes, so an attacker with precomputed hash tables (or who cracks one user's hash) instantly knows every other account sharing that password.
- (c) Salting means concatenating a random, per-user value to the password before hashing, so identical passwords yield different stored digests.
- (d) Yes — the salt can be stored in clear next to the hash. Its purpose is uniqueness/randomness, not secrecy; storing it does not help the attacker recover the password, it only forces them to redo the attack per salt.
- (e) **No, this is not secure — in fact it does not even work as a login mechanism.** Encryption is a *two-way* function: it is designed to be reversible by whoever holds the key. Deleting the key after encrypting means that on the *next* login attempt there is no way to check whether the newly-typed password matches: you cannot decrypt the stored ciphertext (the key is gone), and if you instead encrypt the new attempt with a *fresh* random key, the resulting ciphertext will differ from the stored one even for the *correct* password (different key $\Rightarrow$ different ciphertext). The scheme confuses **encryption** (reversible, needs key management) with **hashing** (one-way, no key to manage), which is exactly the property password storage needs.

Esercizio 3.2

(Scenario applicato) **LastHope** offers a password manager: the user's master password is concatenated with the username (as salt) and hashed to derive an encryption key, which encrypts all credentials before upload; the encryption key is hashed again into an authentication digest sent to the LastHope server. As a security expert, answer: (a) which hash algorithm would you recommend for the local device, and why? (b) is using the *username* as salt secure? (c) which security service should protect the local device-server channel? (d) which primitive should encrypt the credential database on the server?

Soluzione:

- (a) Since this hash is used to **derive a key from a password** (not just to verify a login), a generic fast hash like SHA-256 is a *weak* choice — it is cheap to brute-force on GPUs. The correct choice is a dedicated **password-hashing / key-derivation function** such as **Argon2**, **scrypt**, **bcrypt**, or **PBKDF2**, which are deliberately slow and memory-hard to make brute-forcing the master password expensive.
- (b) No. A salt must be **random and unique**; a username is neither guaranteed to be unique across systems nor random, and it is public information, giving an attacker a head start. A dedicated random salt (32–64 bytes) generated at account creation should be used instead.
- (c) **TLS**, to guarantee confidentiality and integrity of the credentials in transit. A configuration risk to flag: outdated TLS versions/cipher suites or unpatched library implementations (e.g., historical OpenSSL bugs) can undermine the guarantee even when “TLS” is nominally in use.

- (d) A strong **symmetric block cipher such as AES** (128-bit keys or higher), since the server must be able to decrypt on the user’s behalf when credentials are retrieved — this is a legitimate use case for (reversible) encryption, unlike password verification.

3.5 Extensions of Password-Based Authentication

3.5.1 Multi-Factor Authentication (MFA)

MFA requires **two or more independent factors** from different categories before granting access. The three canonical categories are:

Something you know	Something you have	Something you are
Password / PIN	Smartphone, smart card, token	Fingerprint, face, iris
Passphrase	Wearable device	Voice pattern
Secret fact	Badge	DNA signature

Smartphones as a possession factor — caveats: they are not always available or connected; **SIM cloning** lets attackers intercept SMS-based codes; an unprotected phone means anyone with it can also receive the second factor for every account tied to that number.

Esercizio 3.3

(Tema d’esame, 3 luglio 2026 — 35 punti) You ask an LLM to describe MFA and its factor categories. It answers: “*MFA is a security mechanism that requires users to provide one or more independent factors [...]. The goal of MFA is [...] to reduce the risk of authorized access [...] MFA uses a combination of different authentication factors, which fall into five main categories: Something you know [...]; Something you have [...]; Something you do — e.g., solve a CAPTCHA, record a video showing your ID; Something you ask — e.g., a one-time code sent via SMS; Something you are [...]*”. Describe the errors and provide a correct, more comprehensive version.

Soluzione: the answer contains three distinct errors:

- “**one or more**” is wrong: MFA by definition requires **two or more** factors — a single factor is just single-factor authentication.
- “**reduce the risk of authorized access**” inverts the meaning: the goal is to reduce the risk of **unauthorized** access.
- “**five main categories**” is fabricated: there are **three** canonical categories (know/have/are). “Something you ask” (an SMS code) is not a new category — an SMS OTP is simply an instance of *something you have* (possession of the SIM/phone). “Something you do” (CAPTCHA, video ID) is not a login-time authentication factor either — a CAPTCHA verifies you’re not a bot (not *who* you are), and a video-ID check is an **identity-proofing/liveness** technique used at enrollment, not a recognized MFA factor category.

3.5.2 Time-Based One-Time Password (TOTP)

After the first authentication step, a one-time password valid for a short window is generated on a device only the user should hold, from a shared **seed** plus the last generated OTP; the server computes the same value and compares. TOTP is a form of **challenge-response protocol**: the server sends a **challenge**, the device answers using a function of the (hashed) password, so the password itself never crosses the network; this also defeats **replay attacks**, since the challenge is valid only briefly and must include transaction-specific and random data.

Real-world attack: in 2011, **Lockheed Martin** was breached after attackers compromised the seed database of an RSA SecurID (TOTP-style) deployment, allowing them to clone valid one-time codes.

The **revised Payment Services Directive (PSD2)**, an EU regulation, mandates **Strong Customer Authentication (SCA)**: two or more factors, and if a possession factor uses TOTP, the challenge must mix transaction-specific data with random information to remain unpredictable. One practical effect of PSD2 has been to phase out dedicated hardware TOTP tokens in favour of smartphone apps.

3.5.3 Risk-Based (Adaptive) Authentication

The context of a login (location, IP, device, time) is assessed to decide how much to trust a claimed identity; anomalies trigger extra verification.

Example: Rich logs in every morning from San Francisco on his usual desktop around 8am PST. He then travels to Beijing and logs in from an unfamiliar laptop, over hotel wifi, at the local-time equivalent of 5pm PST. Every contextual signal (location, IP, device, time) falls outside his established profile, so the system flags a **high risk score** and triggers an identity-verification alert to Rich and/or IT — and once confirmed, this new behaviour is folded into his profile.

3.5.4 NIST Authenticator Assurance Levels

- **Level 1:** some assurance the claimant controls the authenticator; at least single-factor authentication.
- **Level 2:** high confidence; two different factors required, using approved cryptographic techniques.
- **Level 3:** very high confidence; based on proof of possession of a key via a cryptographic protocol, requiring a “hard” cryptographic authenticator.

3.5.5 MFA Fatigue Attacks

An attacker who already holds valid credentials repeatedly triggers push notifications, hoping a distracted or overwhelmed user accepts one by mistake, mistaking it for a bug or a legitimate request. Technical difficulty is low — it only needs stolen credentials. The main mitigation is removing free-form “just tap accept” push approval in favour of number-matching or requiring an explicit code from the challenge.

3.5.6 FIDO (Fast IDentity Online)

Nota: FIDO non compare nelle slide ufficiali di questa lezione — non è nel materiale originale ma è richiesto in un tema d'esame (Settembre 2025); contenuto integrato dagli appunti di Bordignon.

FIDO aims to reduce reliance on passwords via public-key cryptography. At **registration**, the user's device generates a fresh **public/private key pair**: the public key is sent to and stored by the online service, while the **private key never leaves the local device**. At **authentication**, the user first unlocks the local authenticator (PIN, biometric, etc.), then the device uses the account identifier to select the right private key and **cryptographically signs a server-issued challenge**; the server verifies the signature with the stored public key. Because the private key never travels over the network and is tied to the specific service origin, FIDO is inherently resistant to phishing and credential-stuffing.

Esercizio 3.4

(Tema d'esame, 5 settembre 2025 — 10 punti) Briefly describe what FIDO is and how it works.

Soluzione: see Section 4.3.6 above — cover: (1) the goal (reduce/replace password reliance), (2) the registration step (device-generated key pair, public key sent to the service, private key stays local), (3) the authentication step (unlock authenticator locally → sign server challenge with the private key → server verifies with the public key), and (4) why this resists phishing (no shared secret ever crosses the network).

3.6 Outsourcing Identity Management

Securing every phase of the identity lifecycle is far from trivial, and many organizations (especially SMEs) lack the resources to do it well while focusing on their core business. This motivates outsourcing authentication to a trusted **Identity Provider (IdP)** — sometimes called “the lazy (but scalable) approach to identity management.”

Basic flow: the user signs in to the IdP → the application forwards an authentication request to the IdP → the IdP sends back an authentication token → the token is included in every subsequent request, sparing the user from re-authenticating at each service. This is the basis of **Single Sign-On (SSO)**, detailed with the **SAML 2.0** standard in the next section.

SSO trade-off	
Usability	Only one password to remember for many apps
Security	Only one password to compromise (single point of failure)
Usability	Sessions can be shared across apps
Security	Encourages users to pick a stronger single password

Privacy caveat: an IdP trusted by many services can potentially link a user's actions *across* all of them, since it sees every authentication event.

3.6.1 Summary

- Authentication is the “front door” of any application or system.

- Passwords remain the most widespread user-authentication method despite serious shortcomings.
- Stored passwords must be protected with hashing *and* salting, alongside proper access control.
- Authentication is only *one* phase of the identity lifecycle — weaknesses in enrollment (identity proofing) undermine security just as much as weak login mechanisms.
- Given the cost and difficulty of doing identity management well, it is frequently outsourced to trusted third parties — with security and privacy trade-offs of its own.

4 Cryptography I: Symmetric and Public-Key Cryptography

4.1 A Brief History of Cryptography

Cryptography as a discipline is centuries old. **Leon Battista Alberti**’s *De componendis cifris* (“On writing in ciphers,” 1466) marks a major turning point in the field. Many other works followed, each proposing a new technique to hide the content of messages from everyone except the intended recipient — notably the ciphers attributed to **Vigenère**, **Bellaso**, and **Vernam**. *Nota: la slide originale riporta “Bernam” anziché “Vernam” — quasi certamente un refuso di trascrizione, dato che Gilbert Vernam è la figura storicamente associata alla cifratura a flusso e al cifrario a chiave usa-e-getta (one-time pad), di cui parleremo a proposito del cifrario di Vigenère.*

Centuries later, the **Enigma** machine was used to secure the communications of the German military throughout the Second World War — and breaking it (the work most famously associated with **Alan Turing** and his colleagues) changed the course of human history.

4.2 Basic Notions

4.2.1 Cryptography, Cryptanalysis, Cryptology

- **Cryptography** is the science and study of secret writing.
- **Cryptanalysis** is the science and study of methods of breaking ciphers.
- **Cryptology** is the umbrella term covering both cryptography and cryptanalysis.

Today, cryptography is understood more broadly as **the study of mathematical techniques related to aspects of information security**, namely: **confidentiality**, **data integrity**, **entity authentication**, and **data origin authentication**.

4.2.2 Security Services Provided by Cryptographic Mechanisms

- **Data confidentiality**: encryption algorithms hide the content of messages.
- **Data integrity**: integrity-check functions provide the means to detect whether a document has been changed.

- **Data origin authentication:** message authentication codes (MACs) or digital signature algorithms provide the means to verify the source and the integrity of a message.

4.2.3 The Cryptosystem

A **cryptosystem** is formally a 5-tuple (E, D, M, K, C) where:

- E is an **encryption** algorithm;
- D is a **decryption** algorithm;
- M is the set of **plaintexts**;
- K is the set of **keys**;
- C is the set of **ciphertexts**.

Abstractly, E and D are characterized as functions

$$E : M \times K \rightarrow C, \quad D : C \times K \rightarrow M,$$

subject to the fundamental correctness requirement

$$D(E(m, k), k) = m.$$

Esercizio 4.1

(Dal file di domande di ripasso) Define the notion of a cryptosystem.

Soluzione: A cryptosystem is the 5-tuple (E, D, M, K, C) defined above: E and D are the encryption/decryption algorithms, M the plaintext space, K the keyspace, C the ciphertext space, with $E : M \times K \rightarrow C$, $D : C \times K \rightarrow M$, and the key requirement that decryption undoes encryption under the *same* key used in encryption: $D(E(m, k), k) = m$. *(Whether the encryption key and the decryption key are the same or different is exactly what distinguishes symmetric from asymmetric cryptography — see below.)*

4.2.4 Kerckhoffs' Principle

Kerckhoffs' principle: do not rely on the secrecy of the encryption *algorithm* — the **key** should be the only secret that needs protection. The rationale is a trade-off in disguise: an algorithm kept secret can only be scrutinized by its own designers, while a **public**, well-known algorithm can be studied, attacked, and improved by the entire cryptographic community, so weaknesses are far more likely to surface (and get fixed) before real damage is done. The key, by contrast, is cheap to change and unique per communication/party, so it is the sensible single point of secrecy to protect. This is why virtually every algorithm discussed in this course (DES, AES, RSA, Diffie–Hellman) is entirely public.

4.2.5 Keys: Keyspace, Entropy, and Storage

A **key** is an input to a cryptographic algorithm used to obtain confidentiality, integrity, authenticity, or another security property over some data; the security of the cryptosystem often **depends on keeping the key secret** from some parties.

- The **keyspace** is the set of all possible keys.
- **Entropy** is a measure of the variance (randomness) in keys, typically measured in **bits**. For most cryptographic systems, keys should be as random as possible: intuitively, **randomness** is the apparent or actual lack of pattern or predictability in events, and it is exactly this randomness that an operating system or application must collect and feed into cryptographic key generation.

Keys are often stored in some secure place — **passwords**, on-disk keyrings, a **TPM** (Trusted Platform Module), a secure co-processor, smartcards — ... and **sometimes not**: e.g., the key material inside a **certificate** need not be kept secret at all, because (as we will see later in this lesson) a certificate carries a *public* key, not a private one.

4.2.6 Key Management and Key Distribution

Key management is not just about sharing a key — it also concerns *where* keys are generated, *how* they are generated, *where* they are stored, *how* they get to where they are used, *where* they are actually used, and *how* they are revoked and replaced.

Among these, **key distribution** deserves special attention: the two parties to an exchange must share the same key, and that key must be protected from access by others; frequent key changes are desirable to limit the amount of data compromised if an attacker learns the key. **The strength of any cryptographic system ultimately rests with the key distribution technique** — no matter how strong the algorithm is, a poorly distributed key breaks everything. Four example approaches for two entities *A* and *B*:

1. a key could be selected by *A* and physically delivered to *B*;
2. a third party could select the key and physically deliver it to both *A* and *B*;
3. if *A* and *B* have previously and recently used a key, one party could transmit the new key to the other, encrypted using the *old* key;
4. if *A* and *B* each have an encrypted connection to a third party *C*, then *C* could deliver a key to *A* and *B* over those encrypted links.

4.2.7 Computational Security and Brute Force

An encryption scheme is **computationally secure** if the ciphertext it generates meets one or both of the following criteria:

- the **cost** of breaking the cipher exceeds the **value** of the encrypted information;
- the **time** required to break the cipher exceeds the **useful lifetime** of the information.

It is very difficult, in general, to estimate the effort required to cryptanalyze a ciphertext successfully. Assuming there are no inherent mathematical weaknesses in the algorithm, a **brute-force approach** is indicated — i.e., trying every possible key until an intelligible translation of the ciphertext into plaintext is obtained — which at least allows reasonable estimates about cost and time. On average, **half of all possible keys** must be tried before success, since one does not know in advance where in the keyspace the correct key lies.

In short, writing P for plaintext, C for ciphertext, and k for the key:

- $E(k, P) = C$: computing C from P is **hard** without k , but **easy** with k ;
- $D(k, C) = P$: computing P from C is **hard** without k , but **easy** with k .

This computational asymmetry — easy with the key, hard without it — is exactly what a cryptosystem is designed to guarantee.

4.2.8 Cryptography Is No Silver Bullet

Cryptographic keys are themselves sensitive data stored in a computer system, so access-control mechanisms have to protect *these keys* in turn. **Cryptography (alone) is rarely ever *the* solution to a security problem — rather, it is a translation mechanism: it usually converts a communication-security problem into a key-management problem, which ultimately becomes a computer-security problem (protecting the machine that holds the keys).**

4.3 Classical Ciphers: Substitution and Transposition

Two types of transformation underlie (almost) all ciphers, classical and modern alike:

- **Substitution**: each element of the plaintext (a bit, a letter, a group of bits or letters) is mapped into another element — in other words, letters are replaced by other letters.
- **Transposition**: the elements of the plaintext are rearranged — in other words, the same letters appear, but in a different order.

A fundamental requirement of both is that **no information shall be lost**, i.e., every operation must be reversible; most real encryption systems combine **multiple stages** of substitution and transposition.

4.3.1 Substitution Ciphers: the Caesar Cipher and ROT k

A **substitution cipher** is a method of encrypting by which units of plaintext are replaced with ciphertext according to a fixed system; the “units” may be single letters (the most common case), pairs, triplets, or mixtures thereof. The receiver deciphers the text by performing the **inverse** substitution. In brief: the cipher substitutes one symbol for another, and **the key is the substitution itself**.

The **Caesar cipher** is a substitution cipher in which every character is replaced by the character some fixed number of positions to its right in the alphabet; **the key is the**

number of characters used to shift the cipher alphabet. Mathematically, associating the letters $a, b, c, \dots, z$ with the numbers $0, 1, 2, \dots, 25$, the encryption function is

$$e(x) = (x + k) \bmod 26,$$

and the decryption function is

$$d(x) = (x - k) \bmod 26,$$

where k is the key (the shift applied to each letter); after applying either function, the resulting number is translated back into a letter. This scheme is also called **ROT k** (“ROTate by k ”).

Worked example — cryptanalysis of ROT k : ROT k is probably the easiest of all ciphers to break. Since the shift must be a number between 1 and 25 (a shift of 0 or 26 leaves the plaintext unchanged), one can simply try every possibility and see which one yields readable text — a **brute-force attack** with only 25 candidates. A more principled approach is **frequency analysis**: count how often each letter appears in the ciphertext. English text has a very distinctive letter-frequency distribution — ‘e’ is the most common letter (appearing almost 13% of the time), ‘z’ the least frequent (about 1%) — so one looks for the shift that makes the ciphertext’s letter frequencies line up with the natural English distribution, then decrypts using that shift.

As an illustration, consider the ciphertext `orgursbeprijvgulbh`. Trying $k = 13$ (decrypting with $d(x) = (x - 13) \bmod 26$) gives the perfectly readable phrase “**be the force with you**” — confirming $k = 13$ is the correct key. *Nota: $k = 13$ è il caso speciale noto come ROT13, particolare perché è auto-inverso: applicarlo due volte restituisce il testo originale.*

4.3.2 The Vigenère Cipher

The **Vigenère cipher** can be seen as a **generalization of the Caesar cipher**, whereby several Caesar ciphers in sequence, each with a different shift value, are used. As with the Caesar cipher, letters $a, b, c, \dots, z$ are associated with $0, 1, 2, \dots, 25$. The procedure is:

1. select a keyword;
2. write the keyword repeatedly underneath the plaintext, until every plaintext letter has a keyword letter beneath it;
3. encrypt each plaintext letter using a Caesar cipher whose key is the number associated with the keyword letter written beneath it.

Worked example — plaintext ATTACKATDAWN, keyword LEMON:

A	T	T	A	C	K	A	T	D	A	W	N
L	E	M	O	N	L	E	M	O	N	L	E
L	X	F	O	P	V	E	F	R	N	H	R

(top row: plaintext; middle row: keyword repeated to match the plaintext length; bottom row: ciphertext). For instance, the first A is associated with 0, the first L with 11, and $0 + 11 \bmod 26 = 11$, which is L; the second T is associated with 19, the second E with 4, and $19 + 4 \bmod 26 = 23$, which is X — and so on for every column.

Notice that the same letter A is encrypted to four *different* letters (L, O, E, N) depending on its position in the plaintext, and the letter T occurs three times but maps to only two different ciphertext letters — this makes straightforward frequency analysis considerably harder than for the Caesar cipher. Since the key is the keyword, its length ℓ determines the size of the keyspace (*la slide originale riporta “ ℓ^{26} ”; il valore corretto, per un alfabeto di 26 lettere e parola chiave di lunghezza ℓ , è 26^ℓ — quasi certamente un refuso nell’ordine base/esponente*). As ℓ grows, brute-forcing becomes increasingly complex, until it becomes practically impossible.

The limiting case — an unbreakable cipher: when the length of the keyword equals the length of the plaintext, a genuinely different Caesar cipher encrypts every single plaintext letter, making it impossible to determine the correct plaintext without the key. In this case, **the Vigenère cipher cannot be broken** — *provided, additionally, that the key is used only once*. This is exactly the idea behind an unbreakable **one-time pad**, and connects directly back to the **Vernam** cipher mentioned in the history above.

4.3.3 Transposition Ciphers: Columnar Transposition

In brief, a **transposition cipher** scrambles the symbols of the plaintext to produce the ciphertext; **the key is the permutation** applied to the symbols. More precisely: the units of the plaintext are rearranged in a different (and usually quite complex) order, but the units themselves are left unchanged — the exact opposite trade-off to substitution, where units are retained in sequence but the units themselves are altered.

In a **columnar transposition**, the message is written out in rows of a fixed length, and then read out again *column by column*, with the columns read in some scrambled order. Both the width of the rows and the permutation of the columns are usually defined by a keyword.

Worked example — keyword ZEBRAS (length 6, so rows are 6 characters wide); the column-reading order is given by the **alphabetical rank** of each keyword letter (A=1, B=2, E=3, R=4, S=5, Z=6), so columns Z, E, B, R, A, S are labelled “6 3 2 4 1 5”:

Z	E	B	R	A	S
6	3	2	4	1	5
W	E	A	R	E	D
I	S	C	O	V	E
R	E	D	F	L	E
E	A	T	O	N	C
E	Q	K	J	E	U

Plaintext: WE ARE DISCOVERED. FLEE AT ONCE (25 letters once spaces/punctuation are removed); since 25 is not a multiple of 6, the final row is padded with **nulls** (here Q,K,J,E,U, chosen at random — they are *not* part of the message). Reading the columns in rank order 1, 2, 3, 4, 5, 6 (i.e., column A, then B, then E, then R, then S, then Z) gives the ciphertext:

EVLNE ACDTK ESEAQ ROFOJ DEECU WIREE

4.4 Symmetric Key Cryptography

4.4.1 From Classical to Modern Cryptography

With the rapid development of computers, it became difficult to rely on elementary classical techniques, since ciphertext produced by hand-oriented methods (substitution/transposition on letters) became easy to decrypt by machine. The new generation of encryption techniques instead operates on **bits** rather than characters: **stream encryption** and **block encryption** (together forming **symmetric key encryption**), and **public-key encryption** (**asymmetric key encryption**, covered later in this lesson).

An overview of modern cryptography branches as follows: **Cryptography** splits into **Asymmetric encryption** (public key: **RSA**, **Diffie–Hellman**, others) and **Symmetric encryption** (private key: **block ciphers** — DES, AES, Blowfish, others — and **stream ciphers** — RC4, TKIP, others); a third branch, **Hash Functions** (MD5, SHA, others), is only glimpsed here, since it was already covered in depth when discussing the protection of password files (Section 3.4).

4.4.2 Definition and Notation

In **symmetric key** cryptography, a single key k is used for *both* E and D :

$$D(k, E(k, p)) = p.$$

All intended receivers must have access to this same key. **The management of keys determines who has access to the encrypted data** — whoever holds k can both encrypt and decrypt, so protecting k is protecting the data.

4.4.3 Stream Ciphers vs. Block Ciphers

- **Stream ciphers** encrypt sequences of “short” data blocks under a *changing* key stream. Security relies on the design of the **key-stream generator**; encryption itself can be as simple as **XOR**. Typical block length: 1 bit/byte.
- **Block ciphers** encrypt sequences of “long” data blocks *without* changing the key. Security relies on the design of the **encryption function** itself. Typical block length: 64/128 bits.

Under the hood:

- Stream ciphers use the idea underlying the **Vigenère cipher**, adapted to streams of bits rather than texts of letters.
- Block ciphers use substitution and transposition adapted to manipulate blocks of bits rather than letters. Substitutions are called **S-boxes**, and aim to “hide” the relationship between the key and the ciphertext, achieving **confusion** (each bit of the ciphertext should depend on several parts of the key). Transpositions are called **P-boxes**, and aim to permute bits across S-box inputs, achieving **diffusion** (changing a single bit of the plaintext should change about half of the bits of the ciphertext — the same **avalanche effect** already discussed for hash functions).

Stream cipher mechanics: a stream cipher operates on plaintext and ciphertext one bit at a time; the same plaintext bit encrypts to a *different* ciphertext bit every time it is encrypted, since the keystream changes. Stream ciphers can be designed to be exceptionally fast — much faster than any block cipher in hardware — with less complex circuitry. The simplest implementation is

$$C_i = M_i \oplus K_i \quad (\text{encryption}), \quad M_i = C_i \oplus K_i \quad (\text{decryption}),$$

where K_i is a bit generated by a (pseudo-)random **keystream generator**, shared identically by sender and receiver, and $\oplus$ is bitwise XOR:

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Stream ciphers are typically used in real-time applications such as mobile communications and video streaming — examples include **A5/1** (GSM traffic encryption), **E0** (Bluetooth), and **RC4** (historically used in SSL).

Block cipher mechanics: a block of plaintext is treated as a whole and used to produce a ciphertext block of equal length (typical block size: 64–256 bits). A block cipher breaks the message M into successive blocks $M_1, M_2, \dots, M_n$ and enciphers each M_i with the *same* key k . **DES** and **AES** are the two best-known examples, discussed next.

4.4.4 The Data Encryption Standard (DES)

DES was designed by IBM in the 1970s and adopted by NIST in 1977 for commercial and unclassified government applications. It is a **Feistel block cipher** employing a **56-bit key** that operates on **64-bit blocks**, designed specifically to yield fast hardware implementations (and comparatively slow software implementations — though this is no longer significant today, given how much faster processors have become).

Common misconception — the NSA and DES: the NSA proposed a number of tweaks to DES that many people, at the time, believed were introduced to *weaken* the cipher. Cryptanalysis in the 1990s showed the opposite: the NSA’s suggested changes actually **strengthened** DES.

DES was formally defined in American National Standard X3.92 and three Federal Information Processing Standards (FIPS); all of these were withdrawn in **2005**.

The Feistel structure (1973) is a **block cipher**, i.e., a deterministic algorithm operating on fixed-length groups of bits (a *block*), with an unvarying transformation specified by a symmetric key. Mechanically:

1. the plaintext block is split into two equal halves (L, R);
2. a round function F (a substitution) is applied to one half, using a round **subkey** K_i ;
3. the output of F is **XORed** with the other half;
4. the two halves are then **swapped**, and the process repeats for the next round.

DES has 16 rounds. Instead of reusing the same key in every round, a *different* subkey is derived from the main key for each round; when decrypting, the subkeys must be applied in **reverse order**.

Case study — cracking DES, the EFF effort (1998): the easiest known way to build a practical DES cracker is simply to have it try every key until it finds the right one. The **Electronic Frontier Foundation (EFF)** built a “DES Cracker” consisting of an ordinary personal computer driving a large array (about 1,500) of custom “Deep-Crack” chips, parallelizing the key search in hardware (a thousand DES-cracker chips solve the same problem in one-thousandth of the time a single chip would need). The machine found a 56-bit key in **under 3 days**, searching a total of 17,902,806,669,197,312 keys — an average of **88 billion keys per second**. The whole project cost **under \$250,000** (\$80,000 for design/integration/testing, \$130,000 for hardware); the software was developed in 4–5 weeks, and the entire project was completed within about **eighteen months**.

4.4.5 The Advanced Encryption Standard (AES)

In **1997**, NIST initiated a **public**, four-and-a-half-year process to develop a new secure cryptosystem for U.S. government applications — a deliberate contrast to the *closed* process used to adopt DES 25 years earlier. The result, **Rijndael**, a block cipher designed by the Belgian cryptographers **Joan Daemen** and **Vincent Rijmen**, was selected by NIST in October 2000 and became the official **Advanced Encryption Standard (AES)**, the successor to DES, in **December 2001**. The algorithm can use a variable block length and key length; the specification allows any combination of key lengths of 128, 192, or 256 bits and blocks of length 128, 192, or 256 bits.

AES overview: the encryption process is based on a series of table lookups and **XOR** operations (rounds of SubBytes, ShiftRows, MixColumns, AddRoundKey), which are **very fast operations to perform on a computer**. Decryption consists of running the encryption process in reverse order; however, *unlike* a Feistel cipher, the encryption and decryption algorithms of AES must be **separately implemented** (InvSubBytes, InvShiftRows, InvMixColumns), even though the two are very closely related.

AES is the cryptographic algorithm mandated for U.S. government organizations to protect sensitive, unclassified information (commercial and other non-federal organizations are invited, but not required, to adopt it). No one can be sure how long AES — or any cryptographic algorithm — will remain secure; but as a benchmark, DES was a U.S. government standard for approximately 20 years before a brute-force attack with specialized hardware became practical (the EFF effort above). AES supports significantly larger keys than DES:

Algorithm / key size	Approximate number of keys
DES (56-bit)	7.2×10^{16}
AES-128	3.4×10^{38}
AES-192	6.2×10^{57}
AES-256	1.1×10^{77}

There are, in other words, on the order of 10^{21} **times more** AES-128 keys than DES-56 keys. Barring any attack against AES faster than brute-force key exhaustion, and even accounting for future advances in computing power, AES has the potential to remain secure well beyond 20 years.

Esercizio 4.2

(Tema d'esame, 17 giugno 2025 — 40 punti, parti a–d) In the context of cryptography, (a) state and discuss the Kerckhoffs' principle, (b) explain the purpose of key management, (c) explain the notion of symmetric key cryptography, and (d) describe the difference between a hash function and an encryption function.

Soluzione:

- (a) **Kerckhoffs' principle** (Section 4.2.4) states that the security of a cryptosystem must not depend on keeping the *algorithm* secret, only the *key*. Rationale: a public algorithm can be studied and stress-tested by the whole cryptographic community, so flaws surface (and get fixed) quickly; a secret, home-grown algorithm is reviewed by nobody but its author and tends to hide weaknesses that only become apparent once it is too late. This is why DES, AES, RSA, and Diffie–Hellman are all fully public specifications — only the key is secret.
- (b) **Key management** exists because a cryptosystem's real-world security does not stop at the mathematics of E and D : it depends on *how the key is generated* (with high entropy/randomness), *where it is stored* (securely — e.g. TPM/smartcard, access-controlled), *how it is distributed* to the legitimate parties without leaking to anyone else, *where it is used*, and *how/when it is revoked and replaced*. Cryptography is a *translation mechanism*: it turns a communication-security problem into a key-management problem, and ultimately into a computer-security problem — so weak key management undermines even a mathematically perfect cipher.
- (c) **Symmetric key cryptography** (Section 4.4.2) uses a *single* key k for both encryption and decryption: $D(k, E(k, p)) = p$. All intended receivers must possess the same k ; whoever holds it can both read and produce valid ciphertexts, so the whole security of the scheme reduces to keeping k confidential and getting it safely to every legitimate party (the key-distribution problem discussed above). Stream ciphers (e.g., RC4) and block ciphers (e.g., DES, AES) are the two families of symmetric algorithms.
- (d) The core difference is **reversibility**. An **encryption function** E is a *two-way*, keyed transformation: $E(k, p) = c$ is deliberately designed so that $D(k, c) = p$ recovers the original plaintext — that is the whole point, since encryption exists to protect confidentiality *while still allowing legitimate recovery* of the data. A **hash function** h (Section 3.4) is instead a *one-way*, key-independent transformation: given $y = h(x)$, it must be computationally infeasible to recover x — there is no key that inverts it, by design. Hash functions also compress arbitrary-length input into a *fixed*-length digest, which encryption does not generally do. This is exactly why passwords are *hashed* (verification only needs to check $h(x') \stackrel{?}{=} h(x)$, never to recover x) while, e.g., a symmetric session key or a stored file is *encrypted* (the legitimate holder of the key must be able to get the original data back).

4.5 Asymmetric (Public) Key Cryptography

4.5.1 History and Basic Idea

Asymmetric key cryptography, also called **Public Key Cryptography (PKC)**, is arguably the most significant development in cryptography in the last 300–400 years. It was first described publicly by Stanford professor **Martin Hellman** and his graduate student **Whitfield Diffie** in **1976**: a two-key crypto system in which two parties could engage in secure communication over a *non-secure* communications channel **without ever sharing a secret key**.

The basic idea: use two keys, one to encode and the other to decode, such that they are **mathematically related**, although knowledge of one key does not allow someone to easily determine the other. It does not matter which key is applied first, but *both* are required for the process to work. One of the keys is designated the **public key** and may be advertised as widely as the owner wants; the other is the **private key** and is never revealed to another party.

- **Confidentiality**: suppose Alice wants to send Bob a message. Alice encrypts the information using **Bob's public key**; Bob decrypts the ciphertext using **his own private key**.
- **Authentication / non-repudiation**: the same mechanism can prove who sent a message. Alice encrypts some plaintext with **her own private key**; when Bob decrypts it using **Alice's public key**, he knows Alice sent the message (authentication), and Alice cannot deny having sent it (non-repudiation).

4.5.2 One-Way and Trapdoor One-Way Functions

The main ingredient of PKC is the existence of so-called **one-way functions**: mathematical functions that are *easy* to compute, whereas their inverse is *relatively difficult* to compute. Two classic examples:

- **Multiplication vs. factorization**: given primes 3 and 7, it is easy to calculate the product, 21. Given the number 21 (known to be a product of two primes), it is *not* easy to determine the prime factors. The problem becomes much harder if the primes have, say, 400 digits each, since the product will then have around 800 digits.
- **Exponentiation vs. logarithms**: it is relatively easy to calculate $3^6 = 729$; but starting from 729, it is difficult to determine the two integers x, y such that $\log_x 729 = y$.

With suitable parameters, both problems are the basis for many cryptographic algorithms — although *not all instances* of these problems are difficult to solve (small numbers factor trivially). To be precise, one should be able to *quickly* invert the function if some additional information becomes known (e.g., a suitable key): this refined notion is called a **trapdoor one-way function**.

Worked example: 6,895,601 is the product of two prime numbers — what are they? A brute-force solution is to try dividing 6,895,601 by successive primes until finding the answer. But if one is *told* that 1,931 is one of the factors, the other is found instantly: $6,895,601 \div 1,931 = 3,571$. While a computer can try all small candidates within a second, inverting the function becomes more and more computationally expensive as the

primes grow larger. **Indeed, the problem is only *believed* to be difficult from a computational point of view — there is no mathematical proof that it is truly hard, only the fact that repeated attempts by cryptographers have failed to find an efficient general method.**

4.5.3 Digital Signatures

A **digital signature** is a data item that vouches for the origin and the integrity of a message. Using digital signatures: the originator signs the message with their **private key** and sends it, together with the signature, to a recipient; the recipient uses the originator's **public key** to verify both the origin of the message and that it has not been tampered with in transit. Formally, a digital signature gives a recipient reason to believe that:

- the message was created by a known sender (**authentication**);
- the sender cannot deny having sent it (**non-repudiation**);
- the message was not altered in transit (**integrity**).

Like a handwritten signature, only *one* entity could have produced it — but, done correctly, it is far more difficult to forge.

Esercizio 4.3

(*Dal file di domande di ripasso; una variante quasi identica compare anche al Tema d'esame del 19 febbraio 2026, Domanda 2, 30 punti*) In the context of cryptography, (a) state and describe the Kerckhoffs' principle, (b) explain the purpose of key management, (c) explain the notion of symmetric key cryptography, and (d) explain the notion of asymmetric (or public) key cryptography.

Soluzione:

- (a–c) See the fully worked exercise earlier in this lesson (Sections 4.2.4–4.4.2): (a) Kerckhoffs' principle says security must rest only on the secrecy of the *key*, never the algorithm; (b) key management covers how keys are generated, stored, distributed, used, and revoked, since a mathematically perfect cipher is worthless if its key leaks; (c) symmetric key cryptography uses one key k for both E and D : $D(k, E(k, p)) = p$.
- (d) **Asymmetric (public) key cryptography** (Section 4.5) uses *two* mathematically related keys per party instead of one shared key: a **public** key, freely advertised, and a **private** key, never revealed. Knowledge of the public key does not let an outsider feasibly derive the private one — this rests on a **trapdoor one-way function** (e.g., multiplying two large primes is easy; factoring the product back is hard, unless one already knows a factor). The same key pair supports two distinct guarantees: *confidentiality* (encrypt with the *receiver's* public key; only the receiver's private key decrypts it) and *authentication/non-repudiation* (encrypt — “sign” — with *one's own* private key; anyone can verify with the matching public key). Because it needs no pre-shared secret, PKC solves the key-distribution problem that symmetric cryptography has — but its operations are computationally far heavier than a symmetric cipher like AES, which is why in practice it is used only to bootstrap a session (exchange a symmetric key, or authenticate), not to encrypt bulk data.

Esercizio 4.4

(Tema d'esame, 3 luglio 2026 — 35 punti) You ask an LLM to describe how **Asymmetric Encryption** works. It answers: “*Asymmetric Key Encryption is a hashing method where three entities use a common private key to communicate securely. The sender uses the key to encrypt the ciphertext, and the receiver uses the same key to decrypt the message. Both the sender and the receiver use the same key to encrypt and decrypt messages, but only one of them keeps the private key secret. This maintains confidentiality. This mechanism is significantly faster than Symmetric Encryption.*” Describe the errors and provide a correct and more comprehensive version of the response.

Soluzione: every sentence confuses asymmetric with symmetric cryptography, plus one further factual error:

- “**a hashing method**” — wrong category: hashing is a one-way, unkeyed compression (Section 3.4); encryption is a two-way, keyed transformation designed to be *reversed* by the legitimate party.
- “**three entities use a common private key**” — there is no natural third party, and no “common” key at all: the defining feature of asymmetric cryptography is that *each* party has its *own* key **pair**, not one key shared by everybody. A single shared secret key is precisely the definition of *symmetric* cryptography, not asymmetric.
- “**the receiver uses the same key**” / “**both... use the same key to encrypt and decrypt**” — a second, more severe restatement of the same confusion. In real PKC, to send Bob a confidential message, Alice encrypts with **Bob’s public key**, and Bob decrypts with **his own private key** — two *different* keys.
- “**significantly faster than Symmetric Encryption**” — backwards. Asymmetric algorithms rely on heavy operations over very large numbers (e.g., RSA’s modular exponentiation with hundreds-of-digit numbers), far more expensive than the table lookups and XORs that make AES “very fast” (Section 4.4.5). This is exactly why PKC is used only to bootstrap a session, after which a fast symmetric cipher handles the bulk data.

A correct, comprehensive description: *asymmetric (public) key encryption gives each party a mathematically related key pair — a public key, freely shared, and a private key, kept secret. To send Bob a confidential message, Alice encrypts it with Bob’s public key; only Bob, holding the matching private key, can decrypt it. The two keys are related so that the public key does not reveal the private one (a trapdoor one-way function, e.g., RSA’s reliance on the difficulty of factoring). Because these operations are computationally expensive, asymmetric encryption is normally used only to establish trust or exchange a much shorter symmetric key, not to encrypt bulk data.*

Esercizio 4.5

(Tema d'esame, 17 giugno 2025 — 10 punti) You ask an LLM to describe how **Public Key Encryption** works. It answers: “*Public Key Encryption is a method where two people use a pair of keys to communicate securely. One key is the private key, and the other is the shared key. Both the sender and the receiver use the same key to encrypt and decrypt messages, but only one of them keeps the private key secret. For example, if Alice wants to send a secure message to Bob, she uses her private key to encrypt the message. Bob then uses his public key to decrypt it. This ensures that only Bob can read it, because the public key is tied to Alice’s private key.*” Describe the errors and provide a more comprehensive version of the response. (Suggerimento dell’esame originale: fai attenzione al genere dei pronomi.)

Soluzione: two layers of error:

- **Terminology and the “same key” claim:** the second key is the **public** key, not “the shared key”; and, as in the previous exercise, “both use the same key” describes symmetric, not asymmetric, cryptography.
- **The pronoun trap** — the deeper error, and exactly what the hint points at. As written, Alice encrypts with *her own* private key and Bob decrypts with *his own* public key — but Bob’s public key cannot decrypt something encrypted with *Alice’s* private key, since they are not a matching pair. Even correcting that slip (Bob decrypting with *Alice’s* public key, which does match), the conclusion “only Bob can read it” is still false: Alice’s public key is, by definition, public — *anyone* can obtain it and decrypt what Alice encrypted with her private key. This scheme achieves no confidentiality whatsoever; it is exactly the mechanism for **authentication and non-repudiation** (only Alice could have produced ciphertext that her own public key correctly opens), not for restricting *who* can read the message.

A correct, comprehensive description: *to send Bob a message that only Bob can read (confidentiality), Alice encrypts it using **Bob’s public key**; only Bob, holding the matching **private** key, can decrypt it. If instead Alice wants to prove the message really came from her (authentication/non-repudiation), she encrypts — “signs” — it with **her own private key**; anyone who successfully decrypts it with **Alice’s public key** thereby confirms it could only have been produced by Alice. These are two distinct guarantees, obtained by choosing whose key, and which half of the pair, is used at each step.*

4.6 RSA

4.6.1 Mathematical Background

A **prime number** is a natural number greater than 1 with no positive divisors other than 1 and itself. By the **fundamental theorem of arithmetic**, any integer greater than 1 is either prime itself or can be expressed as a product of primes, uniquely up to ordering. The slow method of verifying primality of n is **trial division**: test whether n is a multiple of any integer between 2 and $\sqrt{n}$ (if $n = a \cdot b$ is not prime, one of a, b must be at most $\sqrt{n}$); an improvement is to only test prime candidates.

Addition (easy)	Multiplication (easy)	Factoring (difficult)
$123 + 654 = 777$	$123 \times 654 = 80,442$	$221 = ? \times ?$ $(221/13 = 17 \Rightarrow 221 = 13 \times 17)$

Two further notions needed for RSA:

- For a positive integer n , two integers a, b are **congruent modulo n** if their difference $a - b$ is an integer multiple of n (i.e., $a - b = kn$ for some integer k), written $a \equiv b \pmod{n}$. Example: $38 \equiv 14 \pmod{12}$, since $38 - 14 = 2 \times 12$.
- Two integers a, b are **relatively prime** (or **coprime**) if the only positive integer dividing both of them is 1.
- **Modular exponentiation** can be simplified before or after exponentiating: $a^k \bmod n = (a \bmod n)^k \bmod n$. Example: $6^2 \bmod 4 = 36 \bmod 4 = 0$, and equally $(6 \bmod 4)^2 \bmod 4 = 2^2 \bmod 4 = 0$.

4.6.2 The RSA Algorithm

Let p, q be two large primes and:

- let $N = p \cdot q$ be the **modulus**;
- choose e relatively prime to $(p - 1)(q - 1)$;
- find d such that $e \cdot d \equiv 1 \pmod{(p - 1)(q - 1)}$ (i.e., d is the inverse of e modulo $(p - 1)(q - 1)$);
- set the **public key** to (N, e) and the **private key** to d .

To **encrypt** plaintext $0 < M < N$: $C = M^e \bmod N$. To **decrypt** ciphertext C : $M = C^d \bmod N$. Correctness follows since

$$C^d \bmod N = (M^e \bmod N)^d \bmod N = M^{e \cdot d} \bmod N = M \bmod N = M,$$

using $e \cdot d \equiv 1 \pmod{(p - 1)(q - 1)}$. Since N and e are public, **if an attacker can factor N** (recover p, q), they can compute $(p - 1)(q - 1)$ and hence d from $e \cdot d \equiv 1 \pmod{(p - 1)(q - 1)}$ — which is exactly why RSA's security rests on the (conjectured) difficulty of factoring.

RSA (Rivest, Shamir, Adleman) is used in hundreds of software products for key exchange, digital signatures, or encryption of small blocks of data; it uses a variable-size encryption block and variable-size key. The key pair is derived from $N = p \cdot q$, where p, q are typically 100+ digits each (so N has roughly twice as many digits). **An attacker cannot determine the prime factors of N (and hence the private key) from N alone — this is exactly what makes RSA secure.** The ability of computers to factor large numbers keeps improving (systems today can factor numbers with 200+ digits), yet if N is built from two roughly-equal-size prime factors, no known factorization algorithm solves the problem in reasonable time: a 2005 test factoring a 200-digit number of this kind took 1.5 years and over 50 years of aggregate compute time.

4.6.3 Worked Example

Pick primes $p = 11$, $q = 3$:

- $N = p \cdot q = 33$;
- pick $e = 3$ (relatively prime to both $p - 1 = 10$ and $q - 1 = 2$, so relatively prime to $(p - 1)(q - 1) = 20$);
- find d such that $3d \equiv 1 \pmod{20}$, i.e., $3d - 1$ divisible by 20: testing $d = 1, 2, \dots$ gives $d = 7$, since $3 \times 7 - 1 = 20$;
- **public key** $(N, e) = (33, 3)$; **private key** $d = 7$.

Take plaintext $M = 7$: the ciphertext is $C = M^e \bmod N = 7^3 \bmod 33 = 343 \bmod 33 = 13$. Decrypting: $M = C^d \bmod N = 13^7 \bmod 33 = 62,748,517 \bmod 33 = 7$ — matching the original plaintext. *Nota: si può evitare di calcolare 13^7 per intero sfruttando l'identità $a \cdot b \bmod n = (a \bmod n) \cdot (b \bmod n) \bmod n$, elevando a potenza in modo incrementale con numeri intermedi piccoli.*

4.6.4 History: Rivest, Shamir, Adleman

In 1976, **Len Adleman**, **Ron Rivest**, and **Adi Shamir**, then graduate students at MIT, read the Diffie–Hellman paper describing potential ways to send private messages without a shared secret key. Inspired, the three worked out an implementation, and in 1977 published a paper showing how a message could be encoded, sent, and decoded with little chance of a third party decoding it. The “RSA” method — now known simply as Public Key Cryptography — is used in almost all Internet-based commercial transactions; Rivest, Shamir, and Adleman received the **ACM A.M. Turing Award in 2002** for this work.

4.6.5 A Real Vulnerability: the Infineon RSA Key-Generation Flaw (2017)

A vulnerability was found in the generation of RSA keys by a software library adopted in cryptographic smartcards, security tokens, and other secure hardware chips manufactured by **Infineon Technologies AG**. It allows a **practical factorization attack**, in which the attacker computes the private part of an RSA key; the attack is feasible for commonly used key lengths (1024 and 2048 bits) and affects chips manufactured as early as 2012, by then already commonplace.

The vulnerability is an **algorithmic** one, in the specific structure of the generated RSA primes; only the **public** key is needed, with no physical access to the device required. **Crucially, the vulnerability does *not* depend on a weak or faulty random number generator — every RSA key generated by a vulnerable chip is affected**, regardless of how good its randomness was. The attack was practically verified on real 1024- and 2048-bit keys; the worst-case factorization cost on an Amazon AWS c4 instance was **\$76** for a 1024-bit key and about **\$40,000** for a 2048-bit key. Impact: a remote attacker can compute an RSA private key purely from its public key, enabling impersonation, decryption of sensitive messages, and forgery of signatures (e.g., on software releases). At the time, about 760,000 vulnerable keys had been confirmed, with possibly two to three orders of magnitude more affected.

4.6.6 Integrity with RSA: Signing and Verifying

Because RSA is relatively heavy computationally, signatures are computed over a **hash** of the message rather than the message itself (efficiency, not security, is the main reason):

- **Signing (sender):** message $\rightarrow$ hash function $\rightarrow$ digest $\rightarrow$ signature algorithm, using the sender's **private key** as the signature key $\rightarrow$ signature, sent together with the message.
- **Verifying (receiver):** recompute the hash of the received message independently; decrypt the received signature using the sender's **public key** as the verification key, obtaining the *expected* digest; compare it against the *actual*, freshly computed digest — if they match, the signature is valid.

It is important to observe that integrity is guaranteed only under the assumption that a single designated entity is in control of the signing key — the sender's private key, in the case of RSA.

4.7 Diffie–Hellman Key Exchange

4.7.1 Mathematical Background

Let $\mathbb{F} = GF(p)$ be a **finite field** (integers modulo a prime p ; GF stands for *Galois Field*). Exponentiation $x \rightarrow g^x \bmod p$ is the one-way function underlying Diffie–Hellman: computationally **easy** to compute. Its inverse is the **Discrete Logarithm Problem (DLP)**: given p, g, X , find x such that $X = g^x \bmod p$ — a computationally **difficult** problem for carefully chosen p, g, X , where p is prime and g is a **generator** (i.e., for every $0 < X < p$ there exists x such that $X = g^x \bmod p$). A key property making the whole protocol work is that **exponentiation and modulus commute**.

4.7.2 The Protocol

Diffie–Hellman allows two parties, Alice and Bob, to generate a shared secret key by exchanging information over an *insecure* channel, in such a way that an eavesdropper Eve — who sees everything exchanged — cannot determine the resulting key.

Alice	Bob
1. Choose a large random number a (private key).	1. Choose a large random number b (private key).
2. Compute $A = g^a \bmod p$ (public key).	2. Compute $B = g^b \bmod p$ (public key).
3. Send A to Bob, receive B from Bob.	3. Send B to Alice, receive A from Alice.
4. Compute $K_A = B^a \bmod p$.	4. Compute $K_B = A^b \bmod p$.

Here a, b are kept secret while g, p, A, B are openly shared. Both parties end up with the *same* shared secret:

$$K_A = B^a \bmod p = (g^b \bmod p)^a \bmod p = g^{ab} \bmod p = (g^a \bmod p)^b \bmod p = A^b \bmod p = K_B,$$

exploiting exactly the commutativity of exponentiation and modulus mentioned above.

4.7.3 Security Considerations and the Man-in-the-Middle Attack

The security of DH depends on the difficulty of the DLP: an attacker able to solve the DLP could recover a and b from the public values A, B . **However, the security of the DH protocol is not known to be equivalent to the DLP** — it could, in principle, be weaker, even though no such attack is currently known. DH is fundamentally a **key-agreement** protocol; its **secrecy** property holds assuming the underlying problem is hard: an attacker merely observing the exchanged messages does not learn the key.

Crucially, DH does not provide authentication: neither party has any guarantee of *whom* they are actually establishing a key with. This opens the door to a **Man-In-the-Middle (MITM) attack**: an attacker C sitting on the channel between A and B can run *two independent* DH exchanges instead of relaying one:

- A sends $g^x \bmod p$, believing it is going to B ; C intercepts it and replies with its own value $g^v \bmod p$, impersonating B . A and C now share a secret $g^{xv} \bmod p$, which A mistakenly believes is shared with B .
- Symmetrically, C sends B a value $g^u \bmod p$, impersonating A ; B replies with $g^y \bmod p$. C and B now share a *different* secret $g^{yu} \bmod p$, which B mistakenly believes is shared with A .

C now holds *two* separate keys — one with A , one with B — and can transparently decrypt, read, modify, and re-encrypt every message flowing between them, while both A and B believe they are talking directly and securely to each other.

4.7.4 History: Diffie and Hellman

In the fall of 1974, Whitfield Diffie and Martin Hellman met for what was intended to be a short meeting; the rich discussion that followed, lasting long into the night, forged an intellectual partnership yielding groundbreaking insights on which the modern field of cryptography still rests. In 1976, they published the revolutionary paper *New Directions in Cryptography* in *Communications of the ACM*, laying the framework for public-key cryptography and digital signatures — today the most widely used security protocols on the Internet, protecting trillions of dollars of web-based financial transactions every day. Diffie and Hellman received the **ACM A.M. Turing Award in 2015** for inventing public-key cryptography.

Esercizio 4.6

(Dal file di domande di ripasso) In the context of the Diffie-Hellman key exchange, describe (a) the protocol, (b) the Man-In-the-Middle (MITM) attack, and (c) how it is possible to mitigate the MITM attack.

Soluzione:

- (a) See Section 4.7: Alice and Bob agree publicly on a prime p and a generator g ; each picks a private random exponent (a for Alice, b for Bob), publishes $A = g^a \bmod p$ and $B = g^b \bmod p$ respectively, and each computes the same shared secret $K = A^b \bmod p = B^a \bmod p = g^{ab} \bmod p$ using their own private exponent and the other party's public value.

- (b) An attacker C positioned on the channel does *not* let A 's and B 's messages pass through untouched: instead, C runs one DH exchange with A (impersonating B) and a separate one with B (impersonating A), ending up with two different shared secrets — one with each victim. A and B believe they share a single secret directly with each other, while in reality C can decrypt, read, and re-encrypt everything passing between them.
- (c) The attack is possible only because **plain DH provides no authentication** of the exchanged public values. The mitigation is to *authenticate* those values: each party can digitally sign its own DH public value using a private key whose matching public key the other party already knows and trusts (Section 4.5), so an attacker cannot substitute a forged value without the forgery being detected. In practice, this trust is delivered via **digital certificates** issued by a trusted third party — the subject of Public Key Infrastructure, the next lesson.

4.8 Takeaways

- **Avoid designing your own crypto algorithms:** amateur designs are usually broken quite easily.
- **DES should no longer be used** (an extension, **Triple DES**, using 3 keys and invoking DES three times, is still used in the financial sector).
- Recommended key length for a block cipher: **at least 80/90 bits** — in practice, use AES.
- **No provable security exists for DES or AES:** both are only resistant to *known* attacks, i.e., attacks aiming to reduce the search space of a brute-force attack below what exhaustive key search would require.
- **Don't forget key management:** how to share the key? (key-distribution techniques, above); how to keep it secret? (access-control techniques). With n participants who all need pairwise symmetric keys, n^2 keys are needed in total — a scaling problem asymmetric cryptography helps avoid.

Common misconception — PKC $\neq$ RSA: some specific properties of RSA do *not* hold for public-key cryptography in general. What *is* general is the pattern of **provable security by reduction**: a scheme's security is proven *relative to* an open mathematical problem (factoring, for RSA; the DLP, for Diffie–Hellman) being hard — never proven outright. **Concrete** security therefore always depends on the current state of the art in solving those open problems, which is why recommended key sizes keep growing over time.

A final word of caution: the anthropomorphic metaphors used throughout this lesson — and throughout cryptography teaching generally — can mislead: stories about “Alice and Bob” create the illusion that one is talking about *persons*, when really every step is about *keys* and the mathematical operations performed on them.

4.9 Relative Security by Key Size

The following table (source: <https://www.keylength.com/>) compares, across several standardisation bodies and academic estimates, the **minimal** key/parameter sizes (in bits) considered secure for a given period, for each major family of algorithm:

Method	Date	Symmetric	Factoring Modulus	Discrete Log Key	Discrete Log
Lenstra / Verheul	2021	86	1937 (1536)	153	1937
Lenstra Updated	2021	82	1416 (1614)	164	1416
ECRYPT	2018–2028	128	3072	256	3072
NIST	2019–2030	112	2048	224	2048
ANSSI	2021–2030	128	2048	200	2048
NSA	—	256	3072	—	—
RFC3766	—	—	—	—	—
BSI	2020–2022	128	2000	250	2000

Notice how, across every body, the **Discrete Log Group** column tracks the **Factoring Modulus** column almost exactly — a concrete illustration that RSA and (finite-field) Diffie–Hellman need comparably-sized parameters to reach the same practical security level as a much shorter symmetric key.

4.10 Summary

- Cryptography moved from letter-based **classical** ciphers (substitution, transposition) to bit-based **modern** ciphers once computers made the classical schemes trivial to break.
- **Symmetric** cryptography (DES $\rightarrow$ AES) uses one shared key for both directions: fast, but it inherits the full **key-distribution** problem — every pair of communicating parties needs a securely pre-shared key.
- **Asymmetric** cryptography (RSA, Diffie–Hellman) solves exactly that problem with a mathematically related key *pair* per party, built on **trapdoor one-way functions** (factoring for RSA, the discrete logarithm for DH) — at the cost of being computationally far heavier, which is why in practice it only bootstraps a symmetric session key or provides authentication/signatures, never bulk encryption.
- None of this security is mathematically *proven*: it rests on decades of failed attempts to efficiently solve specific hard problems, so “computationally secure” is always relative to the current state of the art — hence key sizes must keep growing (Section above).
- **Kerckhoffs’ principle** and disciplined **key management** remain the two universal, algorithm-independent pillars of any cryptosystem.
- Diffie–Hellman provides secrecy but *no authentication* on its own, leaving it open to a **MITM attack** — exactly the gap that **Public Key Infrastructure**, the subject of the next lesson, is built to close.

5 Cryptography II: Public Key Infrastructure and TLS

5.1 Public Key Cryptography, Revisited

5.1.1 Pros and Cons of PKC

Public key cryptography (Section 4.5) has clear advantages:

- the **private key is known only by the owner** (less risk than a shared secret);
- **integrity and confidentiality** can be obtained by encrypting with the **receiver's** public key;
- **non-repudiation** can be obtained by encrypting with the **sender's** private key.

But it also has real costs:

- **Algorithms are 2–3 orders of magnitude slower than those for symmetric encryption** — exactly the point flagged, without the precise figure, back in the “LLM error” exercise on asymmetric encryption in the previous lesson. This is why PKC is typically used only in the *initial* phase of a communication, after which a symmetric secret key is generated to handle the actual bulk encryption.
- How are public keys made available to other people in the first place?
- **There is still a problem of authentication!** Who ensures that the owner of a key pair really is the person whose real-life name is “Alice”? Nothing in the mathematics of PKC ties a key to an identity — that binding has to be established *some other way*. This single question motivates everything else in this lesson.

5.1.2 Mitigating the Cons: Digital Signatures

A **digital signature** is a data item that vouches for the origin and the integrity of a message (already introduced in Section 4.5). In practice, for efficiency, the signature is computed over a **hash** of the message rather than the message itself:

- **Signer:** message $\rightarrow$ hash function (using a digest algorithm) $\rightarrow$ digest $\rightarrow$ encryption with the signer's **private key** $\rightarrow$ **signature**, sent alongside the message over the channel.
- **Receiver:** independently recomputes the hash of the received message (with the same digest algorithm) to get the *actual* digest; decrypts the received signature using the signer's **public key** to get the *expected* digest; compares the two — if they match, the message is authentic and unaltered.

5.1.3 Hash Functions: Recap and Their Limits

Recall the requirements on a cryptographic hash function h from Section 3.4: **ease of computation**, **compression** (arbitrary-length input, fixed-length output), **one-way**, **weak collision resistance**, and **strong collision resistance**. A **collision** is exactly the situation in which two distinct inputs $x \neq x'$ map to the same hash, $h(x) = h(x')$.

Worked example — program integrity: the goal is to protect a program x (e.g., device firmware) against tampering. The idea: compute $h(x)$ in a protected environment and store it somewhere it cannot be modified; to check later whether the program has been altered, recompute the hash and compare it against the stored value. **Over-the-air (OTA) firmware updates in connected vehicles** are a real example where this is safety-critical, not just a confidentiality matter — recall the **2015 Jeep hack** (Section 1.4): rewriting the infotainment unit’s firmware was precisely the first step attackers used to send forged commands over the car’s internal CAN bus to physical components like the engine and wheels. **Note carefully which property is doing the work here:** protecting a program requires more than h being merely *one-way*. The concern is an attacker who *deliberately* finds some x' with $h(x') = h(x)$ and substitutes it for the genuine program — i.e., a **collision** — not an attacker trying to reconstruct x from $h(x)$ alone. This is exactly why **collision resistance**, not just the one-way property, matters for integrity-checking use cases.

Problems with digital signatures: recall that the usage of some hash functions is deprecated because they are considered weak (Section 3.4). But even granting that hash functions are not themselves a source of security issues, there is still a problem linked to the “**real identity**” of the signer: why should anyone trust who the sender *claims* to be? In other words — who guarantees that whoever is using a given key is actually entitled to use it? Answering this question is the whole point of a **Public Key Infrastructure**.

5.2 Public Key Infrastructure (PKI)

5.2.1 The Identity-Binding Problem

The main requirement on a public key is to be **authentically bound** to the identity of the party controlling the corresponding private key. Failing this leads directly to **impersonation attacks**: e.g., party A might receive a digital signature on message M , supposedly generated by party B using B ’s private key, but actually generated by party C ; A would then verify the signature using C ’s public verification key while believing it is B ’s key. A second requirement: parties relying on the correctness of a public-key/identity binding need to be assured that the binding is **still valid** — not just that it was correct once. Satisfying both requirements is typically done by setting up a **Public Key Infrastructure (PKI)**.

5.2.2 From Bulletin Boards to Digital Certificates

An initial, naive proposal was to treat certificates as **bulletin boards**: public keys and identities simply listed somewhere — but this requires trusting the bulletin-board provider outright. A non-scalable, inflexible approach adopted in mobile and IoT applications is to **hardcode** the public key directly into the software of a party; the main security issue then becomes protecting that stored public key from tampering during software updates, or from substitution in the software provider’s own code repository.

The widely adopted solution for Internet applications and services is the use of **digital certificates**: digital objects containing data — including an identity and a public key — that are **signed by a Trusted Third Party (TTP)** attesting to the correctness of the binding between the two. **Notice what this actually does:** it *moves* the problem of checking the identity/public-key binding to the problem of *distributing and keeping up to date* the TTP’s own public key, needed to check the authenticity of the certificate.

This new problem can be deferred several times (but only **finitely many** times, to avoid infinite regression) by building a **certificate chain** whose trustworthiness ultimately leads back to a **root certificate** that must be **self-signed**.

5.2.3 Digital Certificates

A **digital certificate** is the binding between an entity's public key and one or more attributes concerning its identity. The entity can be a person, a hardware component, a service, etc. A digital certificate is issued (and signed) by someone — usually a TTP, also called a **Certificate Authority (CA)**. Indeed, TTPs *should* be trusted: trusting a certificate ultimately means trusting the digital certificates its issuing CA produces.

Informally, a certificate carries: the **Issuer** (the CA), the **Subject** (the entity the certificate is about), the **Subject's Public Key**, and the **Issuer's Digital Signature** over all of the above — plus bookkeeping fields such as a serial number and a validity period. The **X.509** standard formalizes this structure precisely:

Field	Meaning
Version	Version of X.509 to which the certificate conforms.
Serial Number	A number that uniquely identifies the certificate.
Signature Algorithm ID	The specific public-key algorithm the CA used to sign the certificate (e.g., RSA with SHA-1).
Issuer (CA) X.500 Name	The identity of the CA server that issued the certificate.
Validity Period	The time window (start date + expiration) during which the certificate is valid.
Subject X.500 Name	The owner's identity, in X.500 directory format (e.g., <code>cn=ausser, ou=SP, o=Alphawest</code>).
Subject Public Key Info	The public key of the certificate's owner, plus the specific public-key algorithm associated with it.
Issuer Unique ID	Information used to identify the issuer of the certificate.
Subject Unique ID	Information used to identify the owner of the certificate.
Extensions	Additional information, e.g., an alternate name or a CRL Distribution Point (CDP).
CA Digital Signature	The CA's actual digital signature over the whole certificate.

5.2.4 PKI Roles: CA, RA, VA, and Revocation

A **PKI** is an arrangement that binds public keys with the respective identities of entities (people, organizations, ...); an entity must be uniquely identifiable within each CA's domain on the basis of information about that entity. This binding is established through a process of **registration and issuance** of certificates at and by a CA — automated or manual, depending on the required level of assurance.

- The **root CA** is the pinnacle of trust and the most significant element in the CA hierarchy: it authorizes **subordinate CAs**, which are the ones actually responsible for issuing day-to-day certificates.
- The **Registration Authority (RA)** is the PKI role responsible for accepting requests for digital certificates and **authenticating** the entity making the request — certificates are not issued until the RA has validated the information. A third-party **Validation Authority (VA)** can provide entity information on behalf of the CA.

- The **Cryptographic Practices Statement (CPS)** is a declaration of the security practices an organization implements for every certificate issued under it, telling relying parties how well the security of that PKI is actually managed.
- A **certificate distribution system** — a repository for certificates (e.g., LDAP) together with a **Certificate Revocation List (CRL)**, a published list of certificates that have been revoked (no longer valid) — lets relying parties find and check certificates.

5.2.5 Obtaining a Certificate: the Certificate Signing Request (CSR)

Concretely, obtaining a certificate proceeds in seven steps:

1. User-A generates a public/private key pair (or is assigned one by an authority in their organization).
2. User-A requests the certificate of the CA server.
3. The CA responds with its own certificate, including its public key and its digital signature (signed with the CA's private key).
4. User-A gathers all the information the CA requires to obtain a certificate (e.g., email address, fingerprints, ...) — whatever the CA needs to be reasonably certain User-A is who she claims to be.
5. **User-A sends a certificate request to the CA**, consisting of her public key plus additional identifying information; the request is **signed with User-A's private key** — this is the **Certificate Signing Request (CSR)**.
6. The CA receives the request, **verifies User-A's identity**, and generates a certificate binding her identity to her public key.
7. The CA issues the certificate to User-A.

The CSR of step 5 therefore contains: information identifying the requester (signed with the requester's own private key, which the requester has already generated); the public key chosen by the requester; additional proofs of identity as required by the CA; and a digital signature over the whole request, produced with the requester's private key. **This self-signature is what prevents an attacker from requesting a bogus certificate for somebody else's public key:** only whoever actually holds the matching private key can produce a valid signature over the request. Note carefully: the requester's private key is *needed to produce* the CSR (to sign it), but it never itself becomes *part of* the CSR.

5.2.6 Requirements for a Trustworthy PKI

A number of further requirements make a PKI actually trustworthy in practice:

- A **standardized naming mechanism** for parties; parties must be able to prove ownership of a given identity to the TTP, and the TTP must actually check this.

- **TTPs must be trustworthy**, issuing certificates only to the “right” parties — a requirement subject to law and regulation. In the past, TTPs have been involved in incidents that resulted in certificates being issued to the *wrong* parties; as a response, Google and other companies launched an initiative for **Certificate Transparency**, building a platform where several independent logs keep track of every certificate issued, enabling monitoring and audit.
- Relying parties must be able to check the **time-validity window** of a certificate against a trusted time source, to prevent an attacker from using an *expired* certificate whose private key they have since compromised.
- Relying parties need **reliable and timely** information on whether a certificate is still valid or has been revoked. This is done either by distributing **CRLs** (checked *locally*, but with a synchronisation delay: there can be a time window in which a just-revoked certificate is not yet reflected) or via the **Online Certificate Status Protocol (OCSP)** (always up to date, but requires high-bandwidth availability, since checks are not performed locally).
- The **software** validating certificates has to work *correctly* — far from a given, given the complexity of the X.509 standard (the corresponding RFC, RFC 5280, is over 150 pages of complex data structures and encoding rules). Bugs happen: the infamous “**goto fail**” bug in Apple’s implementation was a coding error (a duplicated line) that made it possible to bypass *all* correctness checks on certificates.
- **Cryptographic primitives must be kept up to date** with known vulnerabilities. **Vulnerabilities in SHA-1 were already known in 2005**, and cryptographers began deprecating its use then; public collisions were demonstrated in 2017 and confirmed in 2019. Despite that decade-plus warning, web browsers only stopped accepting SHA-1 in 2017 — and certificates using SHA-1 are, remarkably, *still* in use today in some payment systems.

Esercizio 5.1

(Dal file di domande di ripasso — parti (a)–(c) di una domanda più ampia sul contesto di TLS e PKI; le parti (d)–(e), sull’handshake TLS 1.2 e su come TLS garantisce autenticazione/confidenzialità/integrità, saranno completate quando tratteremo l’handshake nella prossima parte) In the context of TLS and the PKI, describe: (a) what a digital certificate is and what are its main components; (b) what a root CA is; (c) how a system can validate a certificate.

Soluzione:

- A **digital certificate** (Section 5.2.3) binds an entity’s identity to its public key, signed by a Trusted Third Party (the CA) that attests the binding is correct. Its main components (X.509): **Issuer** (the signing CA), **Subject** (the entity the certificate is about), the **Subject’s public key**, the **validity period**, and the **Issuer’s digital signature** over the whole certificate — plus a serial number and optional extensions.
- The **root CA** is the top of the trust chain: the entity whose certificate is *self-signed* and, ultimately, hardcoded as trusted (e.g., in a browser’s or operating system’s trust store). It authorizes **subordinate CAs**, which in turn are the ones that actually

sign end-user/end-device certificates — this chain of trust exists precisely to avoid an infinite regression of “who signs the signer’s certificate?”.

- (c) A relying party validates a certificate by checking: the CA’s **digital signature** on it (verified with the issuing CA’s public key); the **certificate chain** up to a trusted root, if intermediate CAs are involved; the **validity period** (not expired, not used before its activation date) against a trusted time source; and the **revocation status** (not present in the relevant CRL, or confirmed valid via OCSP).

5.3 SSL and TLS: Introduction

SSL (Secure Sockets Layer) was developed by Netscape in the mid-1990s, originally shipped in Netscape Navigator 1.1, as a way to secure communications between a client and a server on the web. SSLv2 is now deprecated; SSLv3 is still, per the course material, widely supported.

TLS (Transport Layer Security) is the IETF-standardised evolution of SSL — the de facto standard for Internet security today, using the *same* protocol design as SSL but with different (and evolving) algorithms. As the RFC itself puts it: “*the primary goal of the TLS protocol is to provide privacy and data integrity between two communicating applications.*” In practice, TLS protects information transmitted between browsers and web servers. The version history:

Version	Relationship	RFC (year)
TLS 1.0	SSLv3 with minor tweaks	RFC 2246 (1999)
TLS 1.1	TLS 1.0 + tweaks	RFC 4346 (2006)
TLS 1.2	TLS 1.1 + more tweaks	RFC 5246 (2008)
TLS 1.3	major changes, removed insecure features	RFC 8446 (2018)

TLS is, today, deployed in essentially every web browser.

5.3.1 HTTPS: SSL/TLS in the Browser

The familiar **lock icon** in a browser’s address bar is the SSL/TLS indicator. Its intended goal is to (i) provide the user with the identity of the page’s origin, and (ii) indicate that the page’s contents were not viewed or modified by a network attacker.

HTTP = HyperText Transfer Protocol; **HTTPS** = HTTP *over* TLS/SSL (“HTTP Secure”). The main motivations for HTTPS are exactly (i) authentication of the visited website, and (ii) protection of the privacy and integrity of the exchanged data. Concretely, HTTPS provides:

- **authentication** of the website and the web server one is actually communicating with — protection against **man-in-the-middle attacks**;
- **bidirectional encryption** of communications between client and server — protection against **eavesdropping** and **tampering** (forging) with the contents of the communication.

Precisely, this is a *reasonable guarantee* that: one is communicating with exactly the website one intended to communicate with (as opposed to an impostor), *and* the contents of the communication between user and site cannot be read or forged by any third party. (“*Reasonable guarantee,*” not an absolute one — *Part 3 of this lesson catalogues real historical failures of that guarantee.*)

5.4 TLS in Client-Server Architectures

5.4.1 Client-Server Architecture and TCP

A **client-server architecture** is one in which many **clients** request and receive service from a centralized **server**. Clients provide an interface letting a user request services and display the results the server returns; servers wait for requests and respond to them, typically exposing a standardized, transparent interface so clients need not know the specifics of how the service is actually provided.

The **Transmission Control Protocol (TCP)** establishes a two-way connection between a server and a single client; it provides **reliable** byte-stream transmission of data, with error checking, correction, and message acknowledgement. **“Reliable” here means reliable in the sense of safety, not security**: TCP is built to handle *accidental* transmission errors, not those deliberately induced by an attacker following a strategy. TCP, by itself, guarantees nothing against a malicious adversary — that is exactly the gap TLS is layered on top of TCP to close.

5.4.2 The Transport Layer, in General

A client or server application interacts directly with a transport-layer protocol to establish communication and to send or receive information; the transport protocol in turn uses lower-layer protocols (down to the network layer, e.g., TCP/IP, and further down still) to actually move individual messages between machines. A computer needs a complete stack of protocols to run either a client or a server; the transport layer’s job is to provide a *transparent* transfer of data between end systems (“end points”), using the services of the layers below it.

5.5 How TLS Works: the Handshake and Record Protocols

TLS is built from two main protocols, layered directly on top of TCP/IP and directly below the application layer (HTTP, FTP, Telnet, ...):

- the **Handshake** protocol (together with two small companion protocols, Change Cipher Spec and Alert) — uses **public-key** cryptography, and is used by client and server to (1) negotiate the cipher suite, (2) authenticate each other, and (3) establish the keys later used by the Record protocol;
- the **Record** protocol — uses **symmetric** cryptography, with the keys established by the Handshake protocol, and provides confidentiality *and* integrity/authenticity of the application-layer data actually being exchanged.

5.5.1 The Handshake Protocol, in a Nutshell

Two parties, client and server:

- negotiate the **version** of the protocol and the set of cryptographic algorithms to be used — for interoperability between different implementations of the protocol;
- **authenticate** client and server (*optional* — typically only the server is authenticated), using digital certificates to learn each other’s public keys and verify each other’s identity;
- use those public keys to establish a **shared secret**.

5.5.2 The Record Protocol, in a Nutshell

Provides **confidentiality** and (**message**) **integrity**. On the **send** side: the data is fragmented into blocks; authentication and encryption primitives are applied to each block; the block is handed to TCP for transmission over the network. On the **receive** side: blocks are decrypted, verified for message integrity, reassembled, and delivered to the higher-level (application) protocol.

5.5.3 Two Companion Protocols

- **Change Cipher Spec**: used to switch the encryption being used by client and server — normally used as part of the handshake, to switch over to symmetric-key encryption once it has been agreed.
- **Alert**: its primary use is to report the cause of a failure — e.g., an invalid message received, a message that cannot be decrypted, or a connection being closed.

5.6 The TLS 1.2 Handshake, Step by Step

(Reference: RFC 5246, <https://tools.ietf.org/html/rfc5246>)

Step 1 — Client Hello

The client can start the handshake at any time by sending a **Client Hello** message to the server, containing:

- the **version** of the protocol the client wants to use — it should be the highest one available;
- a **random value**, obtained by chaining a timestamp (32-bit, UNIX time) with a randomly generated nonce (28 bytes);
- **session_id**: an empty field indicating the intent to create a *new* session (in case of session *resumption*, the client behaves differently);
- the list of **supported cipher suites**, each of the form TLS_<KeyExchange>_WITH_<Cipher>_<Mac>;
- the list of supported compression methods;
- a list of requested extensions (additional services required from the server).

Being the initiator, the client effectively decides whether the server must be authenticated, and whether messages might go unencrypted at all: to request this severe a lack of security, the client would have to explicitly propose a cipher suite such as TLS_DH_anon_WITH_... (anonymous Diffie–Hellman: key exchange with *no* authentication) or the extreme TLS_NULL_WITH_NULL_NULL (no encryption, no MAC at all).

Cipher suites. A **cipher suite** is a structure describing the algorithms a machine supports, so that two machines can decide which algorithms to use to secure their connection. It is a set of algorithms that usually includes a **key exchange** algorithm, a **bulk encryption** algorithm, and a **MAC** algorithm — and can include signature/authentication algorithms to help authenticate server or client. Hundreds of cipher suites exist, combining

these components differently; some offer meaningfully better security than others (a full registry is maintained by IANA).

Naming convention example — TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256:

- **TLS** — the protocol this cipher suite is for;
- **ECDHE_RSA** — the **key exchange** algorithm: **E**lliptic **C**urve **D**iffie–**H**ellman, **E**phemeral (ephemeral guarantees no session key is ever used twice) for key negotiation, with **RSA** for certificate/signature validation — together, these determine *if* and *how* client and server authenticate during the handshake;
- **AES_128_GCM** — the **block cipher** used to encrypt the message stream, together with its mode of operation;
- **SHA256** — the **message authentication algorithm** used to authenticate a message.

A message authentication code (MAC) is a short piece of information used to authenticate a message, i.e., to confirm both that the message came from the stated sender (**authenticity**) and that it has not been tampered with in transit (**integrity**). A common example is **HMAC** (Hash-based MAC): the sender hashes the content of a message combined with a key (negotiated during the handshake) into a short MAC sent alongside the message; the receiver repeats the exact same process and checks the result matches, confirming the message was not altered in transit.

Step 2 — Server Hello

The server responds with a **Server Hello** message, containing: the **chosen protocol** version (supported by both parties), the **chosen cipher suite** (supported by both parties), and a **session ID**: a freshly generated value identifying the new session. *Una sessione è una serie di interazioni tra due end point di comunicazione che avvengono nell'arco di una singola connessione: inizia quando la connessione viene stabilita da entrambe le parti e termina quando la connessione viene chiusa.*

Step 3 — Server Continues

Still from the server:

- **Certificate*** — *if* the client requested an authenticated connection, the server must send an X.509 certificate (Section 5.2.3);
- **Server Key Exchange** — sets the **pre-master secret** that will later be used to generate the master secret (via Diffie–Hellman or RSA, in TLS 1.2);
- **Certificate Request*** — a non-anonymous server can send this if it requires the client to be authenticated too;
- **Server Hello Done.**

(An asterisk marks messages that are *optional*, depending on whether mutual authentication was requested.)

Step 4 — Client Responds

From the client, once it has received the above:

- **Certificate*** — *if* the client received a **CertificateRequest** message, it must send its own X.509 certificate;
- **Client Key Exchange** — sets the **pre-master secret** that will later be used to generate the master secret (again, Diffie–Hellman or RSA);
- **Certificate Verify*** — a message providing explicit verification of the client’s certificate;
- **Change Cipher Spec, Finished** (see Step 5 below).

On Pre-Master and Master Secrets

The **pre-master key/secret** is the value obtained directly from the key exchange (e.g., with Diffie–Hellman, the pre-master secret is $g^{ab} \bmod p$, exactly as in Section 4.7). Depending on the cryptographic parameters chosen, the size of the pre-master key can vary — so, to simplify generating session keys, both sides derive a fixed-length value from it, the **master secret**, always exactly **48 bytes**. The master secret is derived from the pre-master key using a **Pseudo-Random Function (PRF)**: an efficient, deterministic function returning a pseudorandom output indistinguishable from a truly random sequence (the main difference from a plain pseudorandom *generator* is that a PRF can accept arbitrary input data). **From the master secret, both client and server derive *multiple* session keys by applying the PRF again:** half of the resulting keys are used for message integrity, the other half for message confidentiality — following the general principle of never using the same key for both encryption and signing/authentication.

Step 5 — Change Cipher Spec and Finished

Change Cipher Spec is a message (a single byte of value 1) sent by *both* parties; once received, that participant switches over to the just-agreed cipher suite. **Finished** is generated by hashing the *entire* handshake so far, and is likewise sent by both parties, signalling that the handshake has completed successfully.

Steps 6–7 — Application Data

From the reception of the **Change Cipher Spec** message onward, *every* subsequent message, sent by either party, is encrypted using the newly shared key — this is exactly the **Record protocol** taking over from the Handshake protocol.

5.6.1 Certificate Verification, Revisited

Steps 3 and 4 above are where certificate verification actually happens: in Step 3, the **TLS client** verifies the **server’s** digital certificate; in Step 4, the **TLS server** verifies the **client’s** certificate (when requested). As already detailed in Section 5.2, there are four aspects to this verification: (1) the digital signature is checked; (2) the certificate chain is checked, in case of intermediate CAs; (3) the expiry/activation dates and validity period are checked; (4) the revocation status of the certificate is checked.

5.7 How TLS Delivers the CIA Triad

5.7.1 Authentication

For **server authentication**: the client uses the server's public key to encrypt the data used to compute the secret key; the server can only successfully generate that secret key if it can decrypt that data with the *correct* private key. For **client authentication** (when requested): the server uses the public key found in the client's certificate to decrypt the data the client sent during Step 4 (Client Key Exchange); the subsequent exchange of **Finished** messages, themselves encrypted with the now-shared secret key, confirms that authentication is complete. If *any* authentication step fails, the handshake fails and the session terminates. The exchange of digital certificates during the handshake is, in short, itself part of the authentication process.

5.7.2 Confidentiality

SSL and TLS combine symmetric and asymmetric encryption to ensure message privacy: during the handshake, client and server agree on an encryption algorithm *and* a shared secret key, used for **one session only**. All messages transmitted between client and server from that point on are encrypted using that algorithm and key, so the message remains private even if intercepted. **Because SSL/TLS use asymmetric encryption specifically to transport the shared secret key, there is no key-distribution problem** — exactly the key-distribution problem flagged as central back in Section 4.2.4, solved here by handing it off to PKC.

5.7.3 Integrity

SSL and TLS provide data integrity by using a **Message Authentication Code (MAC)** — *provided* that the CipherSpec negotiated for the channel actually uses a suitable hash or MAC algorithm. **Use of MD5 and SHA-1 is strongly discouraged**: both are now very old and no longer secure for most practical purposes (recall the SHA-1 timeline in Section 5.2).

5.7.4 MAC, Revisited

To recap: a MAC is a tag used to confirm that a message came from the stated sender (authenticity) *and* has not been tampered with (integrity), letting a verifier detect any change to the message content. A **MAC algorithm** is a family of cryptographic functions, parameterized by a **symmetric** key, used to provide data-origin authentication and data integrity by producing a MAC on arbitrary messages. **MAC functions are similar to hash functions, but with different security requirements; they are similar to digital signatures, but differ in that a MAC is both *generated and verified using the same (symmetric) key***, whereas a digital signature is generated with a private key and verified with the corresponding *different*, public key.

6 Halfway Wrap-Up

This lecture (28 October 2021) is a checkpoint, not new material in the usual sense: most of its 19 slides are unanswered recap questions covering Lessons 1–5. Rather than re-answer each one in full (they are already covered, with worked examples, in the sections cited

below), this section covers only what is genuinely **new** here — a clarifying diagram, one missing definition, and two real-world case studies — and closes with a compact index mapping every recap question to where it is already answered.

6.1 Clarifying the Security Terminology Map

Lesson 1 defined *information security*, *computer security*, *network security*, and *cybersecurity* somewhat independently. This recap slide (citing Von Solms & Van Niekerk, *From Information Security to Cyber Security*, via <https://www.researchgate.net/publication/281107085>) makes their relationship explicit with a Venn diagram of two large, overlapping regions:

- **Information Security** covers *all* information-based assets, **whether or not** ICT techniques are involved in storing, transmitting, or processing them — consistent with Lesson 1’s own definition (“regardless of its form, electronic or physical”).
- **Cyber Security** covers threats that arise specifically **from the use of ICT techniques** — and, crucially, this includes **non-information-based assets** that become vulnerable only *because* ICT is involved. The **2015 Jeep hack** is exactly this case: the asset under attack was the physical car, not information, yet the vulnerability existed only because the car’s internal systems were networked and reachable remotely.
- Where the two regions **overlap** — information-based assets specifically stored, transmitted, or processed *using* ICT — sits **ICT Security**, fed by two narrower, more technical disciplines: **Computer Security** and **Network Security**.

In short: “Information Security” and “Cyber Security” are neither synonyms nor one a strict subset of the other — they overlap, and that overlap (protecting information specifically *via* ICT) is exactly what “ICT Security” names.

6.1.1 Mapping Attacker Actions onto the CIA Triad

A companion diagram pairs the familiar **CIA triad** (Confidentiality/Integrity/Availability — explicitly flagged as “the focus of this course”) with the corresponding **attacker actions** that violate each property: **Disclosure** (violates confidentiality), **Alteration** (violates integrity), and **Denial** (violates availability). This is a useful three-word mnemonic for connecting an attack’s *mechanism* to the specific *property* it breaks — exactly the mapping exercise underlying the real-world violations already catalogued in Lesson 2 (data breaches → disclosure/confidentiality; the FCC bot comments → alteration/integrity; Mirai → denial/availability).

6.1.2 Asset, Threat, Vulnerability, Risk: the Venn Version

A third diagram draws **Threat**, **Vulnerability**, and **Asset** as three overlapping circles, with **Risk** sitting exactly at their common intersection — a compact visual restatement of Lesson 1’s definitions, plus the one term not yet given explicitly: an **Asset** is any data, device, or other component of the environment that supports information-related activities. The accompanying likelihood×impact severity table is the same 5 × 5 risk matrix already given in Lesson 1. *Corredato da una citazione di Salman Rushdie: “There is no such*

thing as perfect security, only varying levels of insecurity” — un buon promemoria di cosa significhi davvero “sicurezza”.

Esercizio 6.1

(Domanda di ripasso, dalle slide del corso) What is the difference between security and reliability?

Soluzione: the two concepts target different *causes* of failure. **Reliability/dependability** (Lesson 1) is about avoiding failures that arise from *accidental* causes — hardware faults, bugs triggered by ordinary use, transmission noise — and is measured against how often/severely a system may deviate from correct service. **Security** is specifically about withstanding a **malicious, adaptive adversary** who is actively trying to cause a failure and will adjust their strategy in response to defenses. The distinction was already made concretely for TCP (Section 5.5’s discussion of the transport layer): TCP is “reliable” in the sense that it detects and corrects *accidental* transmission errors, but that guarantees nothing against an attacker deliberately crafting malicious packets — which is exactly the gap TLS exists to close. A system can therefore be highly reliable (rarely fails on its own) while being completely insecure (trivially broken by an attacker), and vice versa.

6.2 Filling Two Small Gaps

6.2.1 Credential Stuffing

Recap slides ask for this attack without it having been defined yet: **credential stuffing** is an attack in which username/password pairs leaked from a breach of *one* service are automatically tried, at scale, against *other, unrelated* services — exploiting the extremely common practice of reusing the same password across sites. Unlike a dictionary attack (guessing likely passwords) or brute force (trying every possibility), credential stuffing uses *real, already-valid* credentials, just against the wrong target; this makes it effective even against services with strong password policies, since the weak link is password *reuse*, not password *strength*. Mitigations: encourage (or enforce) unique passwords per service — which is exactly the usability trade-off a password manager exists to solve (Lesson 3); require MFA, so a leaked password alone is insufficient; rate-limit and monitor login attempts for the anomalous volume characteristic of automated stuffing; and check new/changed passwords against known-breached-password lists at signup.

6.2.2 Two Refinements on Hashing and Salting

Two framings from this recap sharpen what Lesson 3 already established (Section 3.4):

- Hashing is explicitly described as a **keyless** cryptographic primitive — worth holding onto as the sharpest one-line contrast with encryption (Section 4.4.2’s exercise part (d)): encryption needs a key precisely *because* it must be reversible; hashing needs none precisely *because* it must not be.
- On salting and rainbow tables: **salting mitigates the rainbow-table attack, it does not prevent it** — it makes precomputation prohibitively expensive (a fresh table would be needed per distinct salt), but does not make a targeted, single-password brute-force attack against one specific, known salt+hash pair impossible in principle.

6.3 Case Study: How Passwords and TLS Combine in a Real Login

Lessons 3 and 5 were developed somewhat independently; this slide ties them together into the login flow actually used by essentially every web service:

1. **TLS** secures the channel between client and server, and — critically — **authenticates the server to the client, but not the client to the server** (Section 5.7: server authentication is standard, client-certificate authentication is the optional, rarely-used part of the handshake).
2. Since TLS alone does not tell the *service* who the user is, the service still needs its own user-authentication step: it asks the user for credentials.
3. The client sends the username and password to the server *over the now-secure TLS channel* — the password is not re-hashed/salted by the client for transmission (that would be pointless without also re-deriving it correctly server-side); rather, TLS’s encryption is what protects the credentials *in transit*, while hashing and salting (Section 3.4) protect them *at rest* in the server’s database. **Without the TLS channel, sending a password like this would be exactly equivalent to not having a password at all** — anyone on the network path could simply read it in the clear.
4. The server retrieves the stored salt and digest for that username, recomputes the hash of (salt + submitted password), and compares it against the stored digest — exactly the verification procedure from Section 3.4.

The general lesson: confidentiality *in transit* (TLS) and confidentiality/integrity *at rest* (hashing + salting) are two independent layers protecting two different stages of the same secret’s lifecycle — removing either one breaks the whole chain, even if the other is implemented perfectly.

6.4 Case Study: Let’s Encrypt and the Slow Rise of HTTPS

Despite HTTPS having existed since the mid-1990s, its adoption stayed remarkably slow for roughly two decades. According to **Let’s Encrypt**’s own account of its impact (2020 Internet Security Research Group Annual Report, <https://letsencrypt.org/>), the bottleneck was never awareness — it was that obtaining and managing SSL/TLS certificates (Section 5.2.3) was cumbersome enough that many site operators simply didn’t bother. **The intervention that actually moved the needle was the opposite of “teach people to care more about certificates”: it was making certificate issuance and renewal fully automatic, so that operators would need to think about it as little as possible.** HTTPS adoption rates rose sharply once this friction was removed. *Un buon esempio di come, in pratica, l’usabilità (Sezione 1, “Security and Human Factors”) sia spesso il vero collo di bottiglia dell’adozione di una misura di sicurezza, più della sua qualità tecnica.* For hands-on intuition about what the resulting TLS handshake actually looks like on the wire, the course points to two interactive walkthroughs: <https://tls12.ulfheim.net/> (TLS 1.2) and <https://tls13.ulfheim.net/> (TLS 1.3).

6.5 Case Study: the 2021 EU Digital COVID Certificate Key Compromise

In late October 2021, it emerged (reported by ANSA and others) that the **private keys used to digitally sign EU Digital COVID Certificate (“Green Pass”) QR codes** had apparently been compromised for at least two member states (France and Poland) — and a fraudulent, obviously-invalid QR code, digitally signed as if genuine, was shown to be verifiable as valid by standard checking apps. A second, independent possibility discussed alongside the key-theft explanation: the reference implementation template for the certificate-issuance web frontend reportedly shipped with **default authentication credentials**, which at least one deploying member state may not have changed in production.

Why this matters, in terms already established in this course: a digital signature (Section 5.1.2) only guarantees authenticity and integrity *under the assumption that a single designated entity is in control of the signing key* (exactly the caveat already flagged when introducing RSA signatures). Once that private key leaks, the mathematics of the signature scheme is completely unbroken — verification still works flawlessly — yet the guarantee it was meant to provide is gone, because the attacker can now produce signatures indistinguishable from genuine ones for *any* content they like. This is a clean real-world instance of “cryptography is a translation mechanism” (Section 4.2.4’s discussion of no-silver-bullet): the cryptography itself was sound; the failure, in both proposed explanations, is a **key-management** failure (a stolen signing key, or a set of unrotated default credentials guarding access to it).

The slide leaves three genuinely open discussion questions rather than a single correct answer; a reasoned position on each, grounded in the concepts above:

- *Joke, or serious?* The mechanism is serious even if a specific instance is played as a joke: once a signing key is exposed, *every* certificate ever issued with it is suspect, and an attacker with no comedic intent could produce a fraudulent vaccination/test certificate just as easily as a fraudulent celebrity one.
- *What should be done?* At minimum: revoke and rotate the compromised signing key(s), audit deployed instances of the issuance software specifically for unchanged default credentials, and check whether the PKI design allows timely revocation of everything signed with a compromised key (Section 5.2’s discussion of CRL/OCSP) — a much harder problem when many already-issued, still-valid-looking certificates are in active use.
- *Consequences of the mitigation?* Revoking a compromised key invalidates every certificate it ever signed, including all the *legitimate* ones — at national or EU scale, this could mean re-issuing certificates for a large fraction of a population on short notice, which is itself an operational and trust-management problem, not just a technical one.

6.6 Cross-Reference Index for the Recap Questions

The table below maps every cluster of recap questions from this lecture to where it is already fully answered.

Recap question cluster (this lecture)	Already answered in
Define information/computer/network/cyber security; cyber security vs. readiness/posture; security vs. reliability	Lesson 1, “What is *-Security?”; Section 6.1 above (terminology map; security vs. reliability)
Define/give examples for confidentiality, integrity, availability, authenticity, non-repudiation	Lesson 1, “The CIA Triad” (§1.4)
Define vulnerability, threat, exploit, risk, likelihood, impact, risk matrix; the attribution problem	Lesson 1, “Vulnerability, Threat, and Risk” and “The Risk Matrix”; Lesson 2, “The Attribution Problem”; Section 6.1 above (Asset)
Security policy vs. mechanism vs. service, with examples	Lesson 1, “Security Policy, Mechanism, and Service”
Password guessing (brute force/dictionary), spoofing, phishing, and their countermeasures	Lesson 3, “Common Attack Patterns” and “Mitigations”
Hash function properties; weak vs. strong example; hashing + salting; password-file structure; can salts be stored in clear; credential stuffing	Lesson 3, “Protecting the Password File” (§3.4); Section 3.4 above (credential stuffing, gaps)
Authentication vs. identification vs. onboarding; the 3 factor types; MFA definition	Lesson 3, “Extensions of Password-Based Authentication”
TOTP, authenticators, NIST assurance levels; SSO experience, pros and cons	Lesson 3, “Time-Based One-Time Password”, “NIST Authenticator Assurance Levels”, “Outsourcing Identity Management”
Cryptosystem, computational difficulty, Kerckhoffs, key management, substitution/transposition examples	Lesson 4, “Basic Notions” and “Classical Ciphers”
Symmetric vs. asymmetric crypto; stream vs. block ciphers; DES/AES; Diffie–Hellman and its MITM attack	Lesson 4, “Symmetric Key Cryptography” and “Asymmetric (Public) Key Cryptography”
PKC for confidentiality/integrity/authenticity; digital certificates; PKI purpose and components; obtaining a certificate	Lesson 5, “Public Key Cryptography, Revisited” and “Public Key Infrastructure”
TLS purpose; SSL/TLS relationship; TLS’s protocols; the TLS 1.2 handshake; TLS vulnerabilities; TLS 1.2 vs. 1.3	Lesson 5, “How TLS Works”, the full handshake walkthrough, “TLS Vulnerabilities”, “TLS 1.3”
How passwords and TLS combine in practice	Section 6.3 above

7 Authentication II: Single Sign-On, SAML, SPID, and CIE 3.0

7.1 Single Sign-On (SSO)

7.1.1 Motivation: Password Fatigue

The straightforward approach to authentication — a separate account and password per application — does not scale. Multiple systems typically require multiple sign-on dialogues, leading to **password fatigue**: multiple sets of credentials to remember, and credentials to present over and over. This is a headache for both users and administrators; the more security domains a user interacts with, the more separate sign-ons are required. **Single Sign-On (SSO)** addresses this by **outsourcing authentication to a trusted third-party identity provider**, so a single act of authentication grants access across many affiliated services.

Security domain: an application, or collection of applications, trusting a common security token for authentication, authorization, or session management. A **security token** is issued to a user after they have actively authenticated with their identifiers and credentials (e.g., a password, or another authentication factor) to the security domain.

7.1.2 SSO: Basic Idea

SSO involves four parties: the **User**, the **User Agent** (typically the browser), the **Identity Provider (IdP)**, and the **Service Provider (SP)**. The basic flow:

1. The user (via the User Agent) **accesses an application** at the Service Provider.
2. The Service Provider **refers the User Agent to the Identity Provider** for authentication.
3. The Identity Provider **asks for credentials** (3a); the User Agent **provides credentials** (3b).
4. The Identity Provider **transfers the User Agent back to the application**, with evidence that the user has successfully authenticated — this evidence is the **authentication token** (step 5, really the *content* of the transfer in step 4).

In general, more than one Service Provider can trust the same Identity Provider — this is precisely what turns a single IdP into the hub of an entire federation of services a user can access without re-authenticating at each one.

7.1.3 SSO Properties

- **Credentials never leave the *authentication domain*** (i.e., Identity Provider + User Agent) — the Service Provider never sees the user's actual password.
- **Service Providers (affiliated domains) have to trust the authentication domain**: credentials must be asserted correctly, and protected from unauthorised use. This trust runs specifically *from* the SP *toward* the IdP.

- **Authentication transfer has to be protected**, against two distinct attack classes:
 - **Replay**: an authentication token intercepted once could otherwise be reused indefinitely — mitigated by including a **nonce** (Section 5.6) as a freshness parameter, limiting both reuse and lifetime of the token/assertion.
 - **Interception/masquerade**: an attacker forging or tampering with an assertion — mitigated by attaching **cryptographic signatures** to authentication assertions (Section 5.1.2).

The resulting trust chain is worth making explicit: the SP grants “*user is entitled to access the provided services*” **because** it received a valid “*authentication token*”, which in turn packages the claim “*user has been authenticated by means of a password*” — this whole package is the **authentication assertion**.

7.2 SAML: Overview

7.2.1 What SAML Is

SAML (Security Assertion Markup Language) is a security service for authentication, configurable to fit the goals of several different use-case scenarios: a common language and message flow for systems that want to offer users an SSO experience, answering the historical lack of standards and interoperable solutions for exchanging authentication and authorization information across security domains. SAML 1.0 was defined in 2002, SAML 2.0 in 2005; it is an open standard from **OASIS** (Organization for the Advancement of Structured Information Standards), and relies heavily on mechanisms already implemented in browsers (e.g., HTTP redirections).

SAML is a data format — for exchanging both authentication and authorization data — built on **XML** (Extensible Markup Language, <https://www.w3.org/TR/xml11/>). XML is a software- and hardware-independent, self-descriptive framework for storing and transporting data; unlike HTML, which focuses on *how* data looks and uses a fixed, predefined tag set, XML focuses on *what* the data is and lets applications define their own tags.

7.2.2 SAML’s Building Blocks

The OASIS specification organizes SAML into four layered building blocks, plus two supporting concepts:

- **Assertions** (the base layer): authentication, attribute, and entitlement information.
- **Protocols**: requests and responses for obtaining assertions and doing identity management.
- **Bindings**: mappings of SAML protocols onto standard messaging/communication protocols (e.g., HTTP).
- **Profiles** (the outermost layer): combinations of assertions, protocols, and bindings supporting a specific, defined use case (e.g., Web Browser SSO).
- **Authentication Context**: detailed data on the type and strength of a given act of authentication.

- **Metadata:** configuration data describing identity and service providers.

This section and the next work outward through exactly these layers: assertions first, then protocols and bindings, then the Web Browser SSO profile that ties them together.

7.2.3 Entities: Identity Provider, Service Provider, Federation

SAML distinguishes two main entities:

- the **Identity Provider (IdP)**: authenticates the user and provides authorisation information;
- the **Service Provider (SP)**: a server hosting protected resources, which relies on information provided by the IdP and enforces its own local access policies.

The authentication act itself is always between a User and an IdP — never directly between the User and an SP. A **federation** is a group of entities sharing a common policy, managed as a single entity; crucially, **the federation only establishes the initial trust among its members** — trust relations are set up once, by exchanging metadata, not re-negotiated on every login.

7.2.4 Metadata and Trust Establishment

IdP and SPs share **metadata** in advance, by whatever means possible, to establish this initial trust. At minimum, the shared metadata must include:

- the **Entity ID**: a globally-unique identifier, included in every message the entity issues;
- **cryptographic keys**: for authentication purposes, a SAML message may be digitally signed by its issuer, and its content can be protected with a public encryption key belonging to the ultimate receiver — so trusted public keys must be shared *in advance*, exactly the PKI bootstrapping problem from Section 5.2.

Ulteriori dettagli: https://en.wikipedia.org/wiki/SAML_metadata.

7.2.5 Worked Example: Alice, the Airline, and the Car Rental

1. Alice visits an airline website to book her trip and provides her credentials to it; the airline site plays the role of **Identity Provider** (and, simultaneously, of an SP for her booking).
2. After booking, she follows a link to a partner car rental site.
3. She visits the car rental website — and **rents a car without signing in again**, because the car rental site (the **Service Provider**) trusts the airline site, and accepts the authentication assertion it transmits (e.g., “*Alice is a VIP member*”).

Concretely: (1) Alice authenticates to the airline site; (2) the airline site asserts “*Alice is a VIP member*” to the car rental site; (3) Alice accesses car-rental resources directly, with no second login. **All of these steps are backed by a SAML assertion, in XML format** — the subject of the next subsection.

7.2.6 Choosing an IdP: the Discovery Service

When a user can, in principle, authenticate via any one of several IdPs, a **Discovery Service** helps them choose which:

1. the user wants to access an SP;
2. the user is redirected to a Discovery Service (which can be external, or embedded in the SP itself), which lets the user choose their IdP;
3. the user goes back to the SP carrying the ID of their own IdP;
4. the user is redirected to that IdP;
5. authentication is performed;
6. the user goes back to the SP, now carrying proof of authentication.

7.3 SAML: Assertions in Detail

An **assertion** is a set of statements (claims) made by a SAML authority (the “asserting party”); it is the basic unit of information exchanged in SAML. There are three types:

- **Authentication assertion:** issued by the party that authenticates users. It describes who issued the assertion, the authenticated **Subject**, the assertion’s **validity period**, and other authentication-related information.
- **Attribute assertion:** defines specific details *about* the Subject. Example: “*Alice has ‘VIP member’ status.*”
- **Authorization assertion:** defines something the Subject is *entitled to do*. Example: “*Alice is permitted to rent a car when on a business trip.*”

7.3.1 Worked Example: an Authentication Assertion

```
1: <saml:Assertion xmlns:saml="urn:oasis:names:tc:SAML:2.0:assertion"
2:   Version="2.0"
3:   IssueInstant="2005-01-31T12:00:00Z">
4:   <saml:Issuer Format="urn:oasis:names:tc:SAML:2.0:nameid-format:entity">
5:     http://idp.example.org
6:   </saml:Issuer>
7:   <saml:Subject>
8:     <saml:NameID
9:       Format="urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress">
10:      j.doe@example.com
11:    </saml:NameID>
12:  </saml:Subject>
13:  <saml:Conditions
14:    NotBefore="2005-01-31T12:00:00Z"
15:    NotOnOrAfter="2005-01-31T12:10:00Z">
16:  </saml:Conditions>
17:  <saml:AuthnStatement
```

```

18:     AuthnInstant="2005-01-31T12:00:00Z" SessionIndex="67775277772">
19:     <saml:AuthnContext>
20:       <saml:AuthnContextClassRef>
21:         urn:oasis:names:tc:SAML:2.0:ac:classes:PasswordProtectedTransport
22:       </saml:AuthnContextClassRef>
23:     </saml:AuthnContext>
24:   </saml:AuthnStatement>
25: </saml:Assertion>

```

Reading it top to bottom: lines 4–6 (<saml:Issuer>) identify the **SAML authority making the claim** — here, `idp.example.org`. Lines 7–12 (<saml:Subject>) identify the **entity about which the claim is being made** — here, `j.doe@example.com`. Lines 13–16 (<saml:Conditions>) give the **assertion’s validity period** (from 12:00:00Z, until 12:10:00Z — only ten minutes). Lines 17–24 (<saml:AuthnStatement>) are the **AuthN element describing the act of authentication at the identity provider** — here, that it happened via `PasswordProtectedTransport`.

Notice carefully: authentication itself is *not* part of SAML. This statement merely *refers* to an authentication act that took place at some *prior* time (the actual login, at the IdP) — SAML packages and transports the *claim* that authentication happened, it does not perform the authentication.

7.3.2 Worked Example: an Attribute Assertion

```

1: <saml:AttributeStatement>
2:   <saml:Attribute
3:     xmlns:x500="urn:oasis:names:tc:SAML:2.0:profiles:attribute:X500"
4:     NameFormat="urn:oasis:names:tc:SAML:2.0:attrname-format:uri"
5:     Name="urn:oid:2.5.4.42"
6:     FriendlyName="givenName">
7:     <saml:AttributeValue xsi:type="xs:string"
8:       x500:Encoding="LDAP">John</saml:AttributeValue>
9:   </saml:Attribute>
10:  <saml:Attribute
11:    NameFormat="urn:oasis:names:tc:SAML:2.0:attrname-format:basic"
12:    Name="LastName">
13:    <saml:AttributeValue
14:      xsi:type="xs:string">Doe</saml:AttributeValue>
15:  </saml:Attribute>
16:  <saml:Attribute
17:    NameFormat="http://smithco.com/attr-formats"
18:    Name="CreditLimit"
19:    xmlns:smithco="http://www.smithco.com/smithco-schema.xsd">
20:    <saml:AttributeValue xsi:type="smithco:type">
21:      <smithco:amount currency="USD">500.00</smithco:amount>
22:    </saml:AttributeValue>
23:  </saml:Attribute>
24: </saml:AttributeStatement>

```

Each <saml:Attribute> block asserts that the enclosing assertion’s subject is associated with attribute *attrib* with value *val*: here, lines 10–15 assert that the

value of the attribute “LastName” associated with the assertion’s subject is “Doe.” The third attribute (lines 16–23) shows SAML’s extensibility: `CreditLimit` is not a standard attribute at all, but one defined in a completely custom namespace (`smithco`) with its own value type — SAML places no restriction on what attributes an IdP can assert, only on how they must be structured.

7.4 SAML: Protocols and Bindings

7.4.1 Protocols

A **communication protocol** is a system that allows two or more entities to exchange information, by defining the rules, syntax, semantics, and synchronization of that communication. A **cryptographic protocol** is typically a much smaller, more compact communication — often just a few exchanged messages in given formats — designed to secure communication (toward various security goals) using cryptographic primitives (ciphers, hash functions, ...). **The typical security goal of SAML’s protocols is user authentication: the SP authenticates the user *through* an IdP assertion, never directly.**

SAML defines several protocol flows, including:

- **Assertion query and request:** for obtaining SAML assertions.
- **Authentication request.**
- **Artifact resolution:** a mechanism by which protocol messages may be passed *by reference* (an “artifact,” a small reference token) rather than by full value, with the actual message fetched separately.
- **Single logout** (Section 7.5 below).

Nota importante dalla slide: il comportamento di un protocollo è tipicamente indipendente da come esso venga effettivamente implementato.

7.4.2 Bindings

Recall that **HTTP** (HyperText Transfer Protocol) is the application-layer protocol used to load web pages via links, transferring information between networked devices on top of the rest of the network stack; a typical flow has a client making a request to a server, which sends back a response.

SAML requestors and responders communicate by exchanging messages; the mechanism used to **transport** these messages is a **SAML binding**. Types include **SAML URI**, **SAML SOAP**, **HTTP redirect**, **HTTP POST**, and **HTTP artifact**. **HTTP redirect** enables SAML protocol messages to be transmitted within URL parameters, letting SAML requestors and responders communicate through an HTTP user agent acting as an intermediary — necessary either when the two entities have no direct communication path, or when the responder needs to interact with the user agent directly (e.g., an authentication agent asking the user to log in). HTTP redirect is sometimes called **browser redirect** in SSO contexts, and is the **default** profile choice.

*Richiamo sulla struttura di una URL (Unique Resource Locator): `https://www.fbkk.eu/en/about-f` si scompone in **protocollo** (`https`), **hostname** (`www.fbkk.eu`) e **file/percorso** (`/en/about-fbkk/`); lo schema `https` indica per l’appunto “HTTP over TLS/SSL” (Sezione 5.5).*

7.5 SSO Within a Domain vs. Federated SSO Across Domains

A distinction worth making precise: plain **SSO** enables access to applications and resources within a *single* security domain (recall the definition: an application or collection of applications trusting a common security token). **Federated SSO**, by contrast, enables SSO to applications *across multiple* security domains — and this is exactly what **SAML 2.0 profiles** are for: combining protocols, assertions, and bindings to *create* a federation and enable federated single sign-on.

SAML defines several profile types, including **Web Browser SSO** (the main subject of the next subsection) and **Artifact Resolution**, plus:

- The **Web Browser SSO** profile offers two independent choices: *flow initiation* (the message flow can start at either the Identity Provider or the Service Provider — the two scenarios detailed next) and *bindings* (HTTP redirect, HTTP POST, or HTTP artifact).
- The **Single Logout** profile terminates *all* of a specified user’s currently active login sessions within the federation at once. A user who achieves SSO across a federation ends up with sessions established at *multiple* participants simultaneously; these sessions are tracked by a **session authority** (often the IdP itself). When the user wants to end every session at once, the session authority uses the Single Logout profile to globally terminate all of them — rather than the user having to log out of each participant individually.

7.6 The Web Browser SSO Profile, Step by Step

7.6.1 IdP-Initiated SSO

Scenario: a user has an active login session on one site (say, `airline.example.com`, playing IdP) and is accessing resources there. At some point they are directed — explicitly or transparently — to a partner site (`cars.example.co.uk`, playing SP), *assuming a federated identity for the user has already been established* between the two sites via a prior business agreement. The IdP site asserts to the SP site that the user is known (by reference to their federated identity), has authenticated to it, and has certain identity attributes (e.g., “has a Gold membership”). Since the SP trusts the IdP, it trusts that the user is valid and properly authenticated, and creates a local session for them — no second login.

Detailed flow (HTTP POST binding), between Browser, Service Provider (`sp.example.com`), and Identity Provider (`idp.example.org`):

1. User selects a menu option or link *on the IdP* requesting access to the SP website — this calls the IdP’s **Single Sign-On Service** (“select remote resource”).
2. The IdP challenges the user for credentials (“challenge for credentials”).
3. The user provides valid credentials; a local logon security context is created at the IdP (“user login”).
4. The IdP’s SSO Service **builds a SAML assertion** representing the user’s logon security context.

5. The browser issues an **HTTP POST** request, sending a form containing the signed **<Response>** to the SP's **Assertion Consumer Service**.
6. The SP performs a final **access check** to allow or deny access to the resource.
7. The SP supplies the requested resource to the browser.

7.6.2 SP-Initiated SSO

Scenario (the more common case in practice): a user visits an SP site *first*, possibly accessing resources that need no special authentication or authorisation. When they subsequently attempt to access a *protected* resource, the SP sends the user to the IdP with an authentication request, so the user can log in there. Once logged in, the IdP produces an assertion the SP can use to validate the user's access rights to the protected resource.

Detailed flow (redirect + POST bindings):

1. User attempts to **access a resource** directly on the SP.
2. The SP's **access check** finds no valid session, and sends a **redirect response** carrying an **<AuthnRequest>**.
3. The browser performs a **GET** to the IdP's **Single Sign-On Service** using that **<AuthnRequest>**; the SSO Service determines whether the user already has an existing logon security context at the IdP — if not, the IdP challenges the user for credentials.
4. The user provides valid credentials (“user login”); a local logon security context is created.
5. The IdP's SSO Service **builds a SAML assertion** representing the user's logon security context.
6. The browser issues an **HTTP POST** request, sending the signed **<Response>** form to the SP's **Assertion Consumer Service**.
7. The SP performs its final **access check** and **supplies the resource** to the browser.

Comparing the two: the steps are almost identical once the flow reaches the IdP — credential challenge, login, assertion built, POSTed back, access check. The *only* structural difference is *where the flow starts*: at the IdP's own SSO menu (IdP-initiated) versus at the SP redirecting an unauthenticated visitor outward (SP-initiated, triggered by an **<AuthnRequest>** the IdP-initiated flow never needs).

7.7 SAML: Security and Privacy Considerations

7.7.1 Authentication Context and Assurance Levels

The **Authentication Context** indicates *how* a user authenticated at the Identity Provider. The IdP includes it in an assertion either at the SP's request, or based on the IdP's own configuration; an SP can require this information to establish a level of confidence in the assertion *before* granting access to resources. The motivating idea is a **Levels Of Assurance (LOA)** model: existing SAML federation deployments categorize the wide

variety of authentication methods into a small number of levels, typically reflecting authentication *strength*; SPs then decide which LOA is required for a given protected resource, based on an assessment of its value or risk.

7.7.2 Metadata, Trust Bootstrapping, and Signing/Encryption in Practice

Recall (Section 7.2) that a minimum metadata set — **Entity ID**, **cryptographic keys**, and **protocol endpoints** (bindings and URLs) — must be shared in advance between IdP and SP. Concretely, this bootstraps the signing/encryption process used on the wire:

1. the issuer signs (and possibly encrypts) the message, then sends it to a **trusted protocol endpoint** whose location is already known;
2. the receiver **decrypts** it (using its own private key) and **verifies the signature** (using a trusted public key found in the sender’s metadata) *before* mapping the entity ID in the message to a trusted partner.

This whole scheme requires each party to already know the other in advance — exactly the PKI/trust bootstrapping problem from Section 5.2. In practice, metadata sharing starts simple (even just an email) and, as the number of partners grows, naturally tends toward automation; any real implementation of SAML Web Browser SSO requires a schema-valid metadata file for *every* partner.

7.7.3 Trust, PKI, and TLS Recommendations

Simply providing assertions from an asserting party to a relying party is not, on its own, adequate for a secure system: how does the relying party trust what is being asserted? What stops a MITM attacker from grabbing an assertion and **replaying** it later? SAML defines mechanisms to detect and protect against exactly this. The primary one is for relying party and asserting party to have a **pre-existing trust relationship**, typically resting on a **PKI** (Section 5.2) — not mandated by the SAML standard, but recommended.

General recommendations, per binding:

- where message **integrity and confidentiality** are required, **SSL/TLS** is recommended;
- when a relying party *requests* an assertion directly from an asserting party, bilateral authentication is required, and **mutual-authentication TLS** is recommended;
- when a response containing an assertion is delivered *via the user’s browser* (e.g., the HTTP POST binding), it is **mandated** that the response be digitally signed using **XML Signature**, to guarantee message integrity even though it passes through an untrusted intermediary (the browser).

7.7.4 A Small Catalogue of SAML-Specific Attacks

- **Message expiration:** a SAML message should carry a timestamp of when it was issued, when it expires, or both; if a message never expires, or an implementation fails to check the expiration, a message that falls into an attacker’s hands remains dangerous indefinitely. Mitigation: always check timestamps.

- **Message replay:** assertions should carry a unique ID, accepted only *once* by the receiving application — exactly the nonce-based anti-replay logic already seen for SSO tokens (Section 7.1.2) and TLS (Section 5.6).
- **SAML intended for a different recipient:** an application should only accept a SAML message that was actually intended for *it*. Skipping this check lets an attacker take a message generated by authenticating to one application, and replay it to a *different* application, walking in as the victim there too.
- **XML External Entity (XXE):** a SAML message is, ultimately, user-provided XML processed by the Service Provider — so it must be checked against the standard XML attack surface (XXE forces the parser to resolve malicious external entities), like any other XML input from an untrusted source.

7.7.5 Privacy Mechanisms

Privacy here means both a user’s ability to **control** how their identity data is shared and used, *and* mechanisms that prevent a user’s actions across multiple SPs from being **inappropriately correlated**. SAML supports several:

- **Persistent pseudonyms**, established between one IdP and one SP: unlike a *global* identifier (the same identifier asserted to every SP), a pseudonym specific to one IdP–SP pair does not itself enable different SPs to correlate the same user’s activity across them.
- **One-time/transient identifiers:** guarantee that every SSO login a user performs at a given SP looks like a *fresh, unrecognizable* individual to that SP, even if the same user visited before.
- The **Authentication Context** lets a user be authenticated at a *sufficient but not excessive* assurance level for the resource they are accessing — avoiding leaking more about the authentication act than the SP actually needs.
- SAML lets the *fact* of a user’s consent to certain operations be expressed between providers — but *how, when, or where* that consent was actually obtained is explicitly out of scope for SAML itself.

7.8 National Identity Infrastructures: SPID

Italy has two digital-identity solutions built on the concepts above: **SPID** (*Sistema Pubblico di Identità Digitale*, Public System for Digital Identity) and **CIE 3.0** (*Carta d’Identità Elettronica 3.0*, Electronic Identity Card, next subsection).

SPID is based directly on **SAML 2.0** and, specifically, the **Web Browser SSO Profile** (Section 7.6). It is regulated by **AgID** (*Agenzia per l’Italia Digitale*), the technical agency of the Presidency of the Council of Ministers, tasked with coordinating public administrations on Italy’s Three-Year Plan for information technology. SPID defines four **assurance levels**: **Level 0** (little or no confidence in the asserted identity’s validity), **Level 1** (some confidence), **Level 2** (high confidence), **Level 3** (very high confidence).

Real-world friction worth knowing about:

- SPID’s **Identity Providers are private companies** — trusting them is, at bottom, a philosophical/political question, not just a technical one.

- **Legal provisions can get in the way** of the technology: at higher assurance levels, the legally-mandated identification requirements can make true SSO (never re-verifying identity) seem practically impossible.
- **User identification during enrollment is delicate.** An Italian journalist once “hacked” the enrollment procedure and successfully obtained a colleague’s digital identity (reported by *Il Fatto Quotidiano*, 2016, regarding then-Prime Minister Matteo Renzi) — a reminder that the SAML/PKI machinery can be flawless while the much lower-tech *identity-proofing step* (verifying a real person is who they claim to be, before ever issuing credentials) remains the weak link.
- Adoption **lagged for years**, though it rose substantially during the COVID-19 pandemic, when many public services moved online.

Metadata management in SPID. The **federation registry** is the repository of every entity adhering to SPID, and is the evidence of the “circle of trust” the federation establishes. Trust is achieved through **AgID’s own intermediation**, as a third-party guarantor: AgID accredits digital identity providers, attribute authorities, and service providers. Joining SPID means establishing a trust relationship with *every other accredited member*, based on the shared, AgID-guaranteed assurance levels — membership itself is evidenced simply by an entity’s presence in the AgID-managed **SPID Register**. For each entity, the register stores an **AuthorityInfo** entry: its SAML identifier, the name of the subject the entity represents, its type (Identity Provider / Attribute Authority / Service Provider), the URL of its metadata provider service, and the list of qualified attributes an Attribute Authority is allowed to certify. The registry is populated and kept updated by AgID as agreements are made and members are supervised.

7.9 National Identity Infrastructures: CIE 3.0

CIE 3.0 carries: name, surname, place and date of birth, residency, the holder’s picture, and (for holders aged 12 or over) two fingerprints, one from each hand — accessible only by law-enforcement agencies. Validity periods differ by age: **10 years** for adults (18+), **5 years** for minors (3–18), **3 years** for children under 3.

CIE 3.0 uses **NFC** (Near Field Communication): a set of communication protocols for two electronic devices at a distance of at most 4 cm, offering a low-speed, simple-to-set-up connection; NFC devices can act as electronic identity documents and keycards, and are also used for contactless/mobile payment. The card’s cryptographic toolbox (only certain combinations are actually supported in practice) includes: **AES** (192 or 256-bit), **Triple DES** (112-bit), **SHA-2** or **SHA-1**, **Diffie–Hellman** (2048-bit) or **Elliptic-Curve Diffie–Hellman** (192-bit), **RSA** (2048-bit, PKCS#1 v1.5), and an **X.509 certificate** carrying the holder’s personal data, signed by an official CA.

Login flow (compliant with the SAML Web Browser SSO Profile): after a one-time preliminary step — downloading the **CieID** smartphone app and registering the physical card to it via NFC — logging into an SP proceeds as:

1. the SP shows an “Entra con CIE” (“Log in with CIE”) button;
2. the user is warned they will need both the physical CIE and their registered smartphone;
3. the user is asked to enter a number printed on the CIE;

4. the user is shown a QR code and asked to scan it with the CieID app, then follow the app's instructions (which trigger the actual NFC read of the card);
5. the user is authenticated.

7.10 European Identity Infrastructure: eIDAS

7.10.1 Purpose and Architecture

eIDAS is, in short, about the **portability of national digital identities across EU Member States** (and more besides). **Both of Italy's own solutions are eIDAS-interoperable: SPID is interoperable with eIDAS, and so is CIE.**

Worked example — opening a bank account: an Italian citizen wants to authenticate against a *German* online banking service.

1. The user browses to the German digital service.
2. The service redirects the user to *its own* local eIDAS-Node (the German **eIDAS-Connector**) to authenticate.
3. The user is sent, via that Connector, to *his* national eID service — the Italian **eIDAS-Node** (playing the role of **eIDAS-Service**).
4. The user signs in with his national eID system (e.g., SPID or CIE).

More precisely: the German eIDAS-Connector, directed by the web application, initiates authentication by sending a request to the Italian eIDAS-Service; the Italian node forwards the user to a system equipped to authenticate Italian citizens via the national eID scheme; after authentication, the German eIDAS-Connector receives the citizen's information and forwards it to the web application. **eIDAS relies on SAML for the communication between the eIDAS-Connector and the eIDAS-Service** — together, these two roles are what is meant by “eIDAS-Nodes.” Each Member State is required to run its own eIDAS-Node; the European Commission provides an **eIDAS-Node Integration Package** as a common implementation basis.

7.10.2 Case Study: the 2019 eIDAS-Node Signature-Verification Bypass

In October 2019, the security firm **SEC Consult** found a vulnerability in the SAML-parsing code of the reference eIDAS-Node implementation that allowed an attacker to **bypass signature verification entirely** — meaning an attacker could send *any* SAML message to an affected eIDAS-Node and have it accepted, e.g., a manipulated SAML response letting them authenticate to a connected service **as anybody**.

The vulnerable logic for deciding whether to trust the certificate a SAML response was signed with worked in three steps: (1) accept the certificate outright if it is already in the local trust store; (2) otherwise, retrieve the *issuer* certificate of the entity certificate, either from the local trust store or from *supplemental certificates bundled inside the SAML message itself*; (3) if a trust path can be established between that issuer certificate and something in the trust store, accept the entity certificate.

The actual bug was in step 2: the application located the “issuer certificate” purely by **string-matching** the Issuer field (DN) of the entity certificate against the Subject field (DN) of candidate issuer certificates — *without ever verifying that the entity certificate*

had actually been cryptographically signed by that issuer certificate (other standard checks, like whether the issuer certificate’s basic constraints even permit it to act as a CA, were skipped too). Since the “supplemental certificates” establishing this chain came from the attacker-controlled SAML message itself, an attacker could simply **forge their own certificate** whose Issuer-DN *textually matches* the Subject-DN of some certificate already sitting in the trust store (e.g., a country’s real root CA) — say, a forged certificate stating CN=FAKE, C=AT — sign their malicious SAML response with it, and have the buggy verification logic accept the chain as trusted, *without the attacker ever possessing the real CA’s private key at all*.

Why this belongs in this course, precisely: this is a textbook illustration of the gap between “a certificate chain exists” and “a certificate chain was actually cryptographically *verified*” (Section 5.2) — confirming a chain of *names* is not the same thing as confirming a chain of *signatures*, and skipping the latter turns a PKI’s entire security guarantee into theater.

7.10.3 Looking Ahead: Digital Identity Wallets

A **personal digital identity wallet** is a user-controlled app empowering citizens to prove their identity or attributes, selectively sharing personal information with services by retrieving **verifiable credentials** from the wallet (*la slide dice “fishing... credentials”*: *quasi certamente un refuso per “fetching”*). Such a wallet could integrate credentials from private or public providers well beyond the national eID — e.g., driver’s licenses, health certificates, insurance, education, or professional qualifications.

This connects to a broader concept: **self-sovereign identity** — users themselves control the verifiable credentials they hold, and their consent is required to use them — built on **decentralized identifiers** (identifiers for subjects, where the *controller* decides what/whom they identify) and **verifiable credentials** (the digital equivalent of physical credentials: passports, driving licenses, qualifications, awards, ...). The EU is building an eIDAS-compatible **European Self-Sovereign Identity Framework (ESSIF)**, using decentralized identifiers on top of the **European Blockchain Services Infrastructure (EBSI)** — with **data interoperability** flagged as the main open challenge.

7.11 Summary

- **SAML lets Service Providers outsource identity management** entirely, reducing administrative burden while improving interoperability, usability, security, and privacy — so they can focus on their core business instead of building authentication from scratch.
- Of SAML’s profiles, **Web Browser SSO** is by far the most widely adopted, and SSO generally keeps gaining adoption.
- **Credentials never leave the authentication domain** (Section 7.1.2); everything downstream — assertions, protocols, bindings — exists to carry *proof* of authentication to other domains without ever carrying the credentials themselves.
- SAML is a genuinely solid **starting point** for real digital-identity infrastructure at national (**SPID**, **CIE**) and European (**eIDAS**) scale — but, as both the enrollment-fraud story and the 2019 eIDAS-Node bug show, **the weakest link is rarely the cryptography itself**: it is identity-proofing at enrollment, and correctly

implementing the trust/verification logic around otherwise-sound cryptographic building blocks.

Esercizio 7.1

(*Dal file di domande di ripasso*) In the context of the SAML standard, (a) explain its main goal, (b) describe the two scenarios for the Web Single Sign On profile, namely Identity provider initiated and Service provider initiated, (c) explain how is it possible to guarantee trust in the assertions consumed by the relying party (hint: remember that assertions are cryptographically signed), and (d) explain how it is possible to avoid man-in-the-middle attacks.

Soluzione:

- (a) SAML's main goal (Section 7.2) is to give systems a **common language and message flow** for exchanging authentication and authorization information across security domains, so that an SSO experience can be offered without every pair of systems having to invent their own bespoke protocol. It packages claims (assertions) about a user — that they authenticated, or hold some attribute, or are entitled to some action — in a standard, XML-based format that an Identity Provider issues and a Service Provider consumes.
- (b) See Section 7.6 in full. In brief: in **IdP-initiated** SSO, the user starts already logged in at the IdP and *selects* a link to a partner SP, which trusts the assertion the IdP pushes to it without ever issuing its own authentication request. In **SP-initiated** SSO — the more common case — the user starts at the SP; only when they hit a *protected* resource does the SP redirect them, carrying an `<AuthnRequest>`, to the IdP to log in, after which the IdP's assertion is POSTed back to the SP.
- (c) Trust in a consumed assertion rests on the **pre-existing trust relationship** established via shared metadata (Section 7.2) and, typically, a **PKI** (Section 5.2): the IdP **digitally signs** the assertion with its private key; the SP, having obtained the IdP's public key *in advance* (via metadata, not from the message itself), **verifies** that signature before trusting the assertion's content. Concretely, this is **mandated** via **XML Signature** whenever the assertion is delivered through the user's browser (e.g., HTTP POST binding), precisely because the browser is an untrusted intermediary.
- (d) The general MITM defense is the same digital-signature mechanism, combined with **TLS**: SSL/TLS is recommended wherever integrity and confidentiality of the channel are required, and *mutual*-authentication TLS is recommended whenever a relying party requests an assertion directly from an asserting party (so each side authenticates the other, not just one direction). On top of the channel security, signed assertions with **unique IDs** and **validity windows** (Section 7.3) directly block the specific MITM variant of *capturing* a valid assertion and **replaying** it later — a stolen assertion is single-use and expires quickly, so replaying it does the attacker no good even if TLS itself were somehow bypassed.

Esercizio 7.2

(*Domanda di ripasso, dalle slide del corso*) What is SPID? What is eIDAS? Is there a relationship between the two?

Soluzione: SPID (Section 7.8) is Italy’s national digital-identity system, built directly on SAML 2.0’s Web Browser SSO Profile and regulated by AgID, letting citizens authenticate to public (and some private) online services with one set of credentials, at one of four AgID-defined assurance levels. **eIDAS** (Section 7.10) is the EU-level framework for the *cross-border portability* of Member States’ own national digital identities — it does not replace SPID, it *sits on top of* it: eIDAS-Nodes (Connector at the relying country, Service at the citizen’s home country) exchange the actual authentication using SAML, exactly the same underlying technology SPID itself is built on. In short: **SPID is Italy’s national eID scheme; eIDAS is the plumbing that lets a SPID-authenticated Italian citizen be recognized by another Member State’s online services, and vice versa** — both SPID and CIE are explicitly eIDAS-interoperable.

8 Access Control I: Basic Notions, the Access Matrix, and DAC/MAC/RBAC

8.1 Access Control: Basic Notions

8.1.1 Definition and Architecture

Access control is the process of (i) mediating requests to the resources and data of a system, and (ii) determining whether a given request should be granted or denied. The basic flow, for a subject s wanting to perform action a on resource r :

1. an **access request** (s, a, r) is sent to the access control module;
2. the module returns **grant** or **deny**;
3. if granted, the system allows s to perform a on r ; otherwise s is informed it cannot.

This is of **paramount importance** to system security: protecting resources from unauthorized access. What counts as “unauthorized” is very context-dependent (an operating system, a military system, online banking, a smartphone, . . . all need different rules) — which is exactly why access control is defined in terms of a **policy**: a set of rules to implement specific security properties (recall **CIA**, Section 1.4). Equivalently, a policy can be seen as a *contract* between a system’s designer/implementor and its customers.

The architecture realizing this has four parts, matching the Policy/Model/Enforcement separation already introduced (Section on access control in the *domande* file, and generalized here):

- a **Subject** (or **Agent/Principal**) — any entity, including a program, allowed to ask to access a resource/object. A **user** is specifically a subject identifying a human being.
- The subject’s request passes through the **Guard**, also called the **Policy Decision Point (PDP)** or **Access Control Module** — this is where **authorization** actually happens (as opposed to **authentication**, which establishes *who* the subject is, *before* the request even reaches the Guard).
- The Guard consults the **Policy** to decide grant/deny, and every decision is recorded in an **Audit Log**.

- Only if granted does the request reach the **Object/Resource**.

Isolation boundaries play a key role:

- the **outer** boundary (wrapping Guard + Policy + Audit Log) prevents *bypassing* the Guard: every authorization request must go through it, be evaluated against the policy, and only then reach the object. It can only be bypassed by *privileged* users (e.g., administrators) who need to install/update policies or perform other routine administrative operations.
- the **inner** boundary (wrapping just the Audit Log) guarantees the **integrity** of the log, so it can later be inspected to understand what happened and why. **Unlike the outer boundary, the inner one cannot be bypassed even by privileged users** — doing so would enable insider attacks on the log's integrity, making auditing meaningless.

Esercizio 8.1

(Dal file di domande di ripasso) In the context of access control, describe the general architecture of an access control enforcement mechanism including (a) subjects, (b) requests, (c) guard, (d) policy, (e) isolation boundaries, (f) audit log, and (g) the role of authentication and authorization.

Soluzione:

- A **Subject** is any entity (user, program, or device) allowed to ask for a resource or service; a **User** specifically identifies a human being.
- A **Request** (s, a, r) — subject s asking to perform action a on resource r — can be granted or denied; every request must be routed to the Guard, which the outer isolation boundary makes mandatory.
- The **Guard** (PDP) is responsible for granting or denying access by evaluating the request against the policy; if authorized, the subject may perform the requested action.
- A **Policy** is a set of rules implementing specific security requirements, specifying which actions a subject can perform on which resources. Policy, model, and enforcement are kept independent (**separation of concerns**, Section 8.1.3): reasoning about a policy does not require knowing how it is enforced.
- Isolation boundaries** prevent unauthorized access to sensitive parts of the mechanism. The **outer** boundary ensures nobody can bypass the Guard — only privileged users (e.g., administrators performing policy updates) may cross it. The **inner** boundary protects the Audit Log and can *never* be bypassed, even by administrators, since that could enable integrity attacks on the log itself.
- The **Audit Log** stores every past request together with its outcome, protected by the inner boundary against any modification of already-granted or already-denied decisions.

- (g) **Authentication** and **authorization** are two distinct processes protecting a system: authentication verifies *who* a subject is (through whatever means match the required assurance level); authorization — performed by the Guard — then determines *what* that subject, once identified, is allowed to do, based on the attributes/privileges associated with it.

8.1.2 A Motivating Example: the Operating System

An **Operating System (OS)** is software managing hardware and software resources, providing common services to programs and letting users communicate with their computer. A **multi-user** OS lets several users share the same computer (concurrently or at different times); a **multi-tasking** OS lets multiple software processes run at the same time. Either way, **the OS must protect users from each other**: memory protection, file protection, general control of access to objects, and user authentication. **The fundamental trade-off of OS security** is exactly **sharing** (desirable) versus **protection** (difficult).

Historically, OS protection evolved through levels: **no protection** (early single-user systems, e.g., early Windows, designed for just one user); **isolation** (processes unaware of each other, each with its own memory/address space/files — the OS provides *confinement*, though some resources like libraries, files, or a database still need sharing); and **share** (either *all-or-nothing* or *granular*).

Managing shared resources well relies on the **principle of least privilege** (already used implicitly in Lesson 3): every subject — process, user, or program — must be able to access only the information and resources strictly necessary for its legitimate purpose. **An important aspect of privacy protection specifically** is the specification and enforcement of **purpose**: users should be confident their data is processed only as intended (e.g., an email address collected for billing should not silently be reused for marketing) — checking this is difficult, which is exactly what makes **auditing** crucial for *a posteriori* verification.

A useful abstraction supporting granular sharing is **resource encapsulation**: all data and functions needed to use a resource are packaged into a single, self-contained component, so the OS can allow access or manipulation of the resource only in the way its designer intended — and, beyond controlling *which operations* are possible on a resource, the OS can further control *which users* may perform *which* of those operations.

8.1.3 Policy, Model, Enforcement

As already introduced: **Policy** = the rules controlling what actions subjects may perform on resources (*how* policies are specified); **Model** = a formal (mathematical) representation of the policy and how it works (*the meaning* of policies); **Enforcement** = the low-level software or hardware functions actually implementing the controls the policy and model formally state (*the implementation* of policies).

Why bother separating these three? When policy and model are both mathematical objects, the meaning of a policy is **independent of any particular implementation** — opening the door to **automated reasoning** techniques for analyzing policies for correctness, conflicts, or unintended consequences. This is **separation of concerns**: when reasoning about a policy, one can simply *assume* enforcement works correctly, and forget about it entirely.

*Motivazione storica per l'attenzione alla correttezza dell'enforcement: il comando Unix **finger** accettava argomenti di lunghezza arbitraria pur avendo allocato solo 256*

byte, e i byte in eccesso finivano per essere eseguiti dalla CPU. Nel 1988 il worm Morris sfruttò esattamente questa vulnerabilità, portando gli attacchi di memory-overwriting (*stack smashing*: un input contenente NOP, shellcode e un nuovo indirizzo di ritorno sovrascrive lo stack per dirottare l'esecuzione) all'attenzione dei media. Il problema — controlli mancanti sulla dimensione degli argomenti — continua a comparire ancora oggi.

The access-control models covered in this course were designed for different application contexts: **AC Matrix, ACLs, and Capabilities** for **operating systems** (this subsection); **Multi-Level Security / Bell–La Padula** for **military systems** (Section 8.4); **Role-Based** and **Attribute-Based AC** for **enterprise systems** (Section 8.8, and the next lesson).

8.2 The Access Control Matrix, ACLs, and Capabilities

8.2.1 The Access Control Matrix (ACM)

The meaning of policies of the form “*a subject can perform some actions on an object*” can be given precisely via the **Access Control Matrix (ACM)**: given all subjects and objects, the ACM simply enumerates which actions are allowed for each subject/object pair — rows are subjects, columns are objects (or resources), and each cell lists the permitted actions.

Esercizio 8.2

(Dalle slide del corso) Alice can read and write the file **f1**, can read the file **f2**, and can execute the file **f3**. Bob can read **f1**, can read and write **f2**, and cannot access **f3**. (1) Write a set of access control lists for this situation — which list is associated with which file? (2) Write a set of capability lists for this situation.

Soluzione: the underlying ACM is:

	f1	f2	f3
Alice	r,w	r	x
Bob	r	r,w	—

An **Access Control List (ACL)** is enforced *per object*: it lists, for one resource, which subjects may do what to it.

Object	ACL
f1	Alice: r,w Bob: r
f2	Alice: r Bob: r,w
f3	Alice: x

A **Capability List** is the transpose: it is enforced *per subject*, listing which objects that subject may access and how — a capability itself is a $\langle \text{object}, \text{rights} \rangle$ pair the subject holds.

Subject	Capability list
Alice	f1: r,w f3: x f2: r
Bob	f1: r f2: r,w

Nota dalle slide: talvolta il permesso di esecuzione (x) implica anche read (r) e write (w).

8.2.2 Domains of Protection

A computer can be viewed as a collection of processes and objects (hardware or software). The **need-to-know principle**: a process should only have access to the objects it needs to accomplish its task, only for the operations it needs, and only during the time frame it needs them — directly generalizing least privilege. A **domain of protection** is formally a set of pairs

$$\langle \text{object}, \{\text{access-right set}\} \rangle,$$

and can be implemented as users, processes, or procedures. **Example (Unix/Linux)**: each *user* corresponds to a domain, defining that user's access rights; changing domains means changing user identifiers, and a program running on a user's behalf inherits that user's identifier, hence that same domain of protection. Precisely because of this, when an executable is given permission to access resources the user running it *cannot itself* access, security issues can arise — exactly the confused-deputy attack, next.

8.2.3 The Confused Deputy Attack

Confused deputy: a privilege-escalation attack where an adversary who lacks direct access to some sensitive resource indirectly *writes* that resource by confusing a subject (the **deputy**) who *can* access it.

Standard example — a compiler service. A compiler service takes a source-file name and an output-file name; it compiles the source, writes the compiled binary to the output file, and — on the side — writes a **billing file** recording how much the caller must pay for using it. The user **Kenny** wants to compile his own file *In*, expecting the binary in *Out*: this needs the authorization queries $(\text{Kenny}, x, \text{Compiler})$, $(\text{Compiler}, r, \text{In})$, $(\text{Compiler}, w, \text{Out})$.

The attack: the caller instead tricks the compiler by passing the name of the **billing file** in place of the output file — causing the compiler to *overwrite the billing file with a binary*, destroying its integrity. In Linux terms: every process executes on behalf of some user and can access whatever files that user can access *and more*, if the system administrator has granted the executable extra access (here, the compiler needs write access to the billing file to do its job at all).

Why ACLs let this happen. Suppose the following ACLs are in place: $\text{Compiler} \rightarrow [\text{Kenny}|x]$, $\text{In} \rightarrow [\text{Kenny}|o, r, w]$, $\text{Bill} \rightarrow [\text{Compiler}|w]$. Kenny can invoke the compiler (x on *Compiler*); the compiler, as deputy, can read *In* (inherited via Kenny's identity) *and* can write *Bill* (granted directly to the *Compiler* process by the OS). **The confusion is exactly this: the Compiler is delegated rights from two different sources — by Kenny, to read his source file; by the OS, to write the billing file — and during execution it cannot tell which right came from where, so it uses both indiscriminately.** If Kenny now passes *Bill* as the “output file” argument, all three authorization queries — $(\text{Kenny}, x, \text{Compiler})$, $(\text{Compiler}, r, \text{In})$, $(\text{Compiler}, w, \text{Bill})$ — are still granted under these ACLs, and the billing file gets silently overwritten.

How capabilities avoid it. Suppose instead **capabilities** are used: $\text{Kenny} \rightarrow [\text{Compiler}|x] \rightarrow [\text{In}|r]$, $\text{Compiler} \rightarrow [\text{Bill}|w]$. Now the first two queries, $(\text{Kenny}, x, \text{Compiler})$ and $(\text{Compiler}, r, \text{In})$, are still granted — but the third, $(\text{Compiler}, w, \text{Bill})$, is **denied**. There is no more confusion, because **Kenny would have to explicitly pass his own capability for the target file to the Compiler** for it to be used: he can do this for *In* (he holds a capability for it), but he has *no* capability for *Bill* at all, so he cannot pass

what he does not have. **This is the deep reason capabilities avoid confused-deputy attacks where ACLs do not: rights travel explicitly *with the subject that holds them*, rather than being silently inherited by whatever process happens to be acting on that subject's behalf.**

8.3 Discretionary Access Control (DAC)

Under **DAC**, the **owner** of a resource decides how it can be shared: there is **no central entity** deciding who can do what on which resources, and an owner can choose to grant read/write/...access to other users on resources they own. DAC is typically used in operating systems, and usually enforced via **ACLs** (Section 8.1.1's worked exercise).

8.3.1 Groups, for Manageable DAC

Large organizations usually group most employees by profile into a small number of categories — e.g., a bank might have 40–50 such **groups** (teller, chief teller, branch accountant, branch manager, ...), with only a handful of remaining individually-managed positions (security manager, chief foreign-exchange dealer, ...). A group may then appear directly in an ACM or ACL; checking whether a subject has a permission amounts to finding a group the subject belongs to that appears in the relevant ACL with that permission. *Questo è un antipasto dell'idea alla base di RBAC (Sezione 8.8): raggruppare permessi per profilo, non per singolo utente.*

8.3.2 DAC: Pros and Cons

- **Pros:** **flexible** (its main reason for popularity in OSes); **well understood** (e.g., the Unix implementation); **intuitive**.
- **Cons:** **subjective** — it relies entirely on the owner's judgement to modify a resource's protection state; it only works if users make no mistakes, which is *impossible* — losing control of the situation is very easy; and it is **vulnerable to trojans** (information leakage).

8.3.3 Worked Example: DAC Defeated by a Trojan

Trojan: any malicious computer program that misleads users about its true intent, typically spread via social engineering (e.g., duping a user into executing a disguised email attachment).

Setup: resource R is readable only by Alice.

1. Bob **induces Alice** (via social engineering) to install and execute a trojan.
2. The trojan runs **with Alice's own DAC permissions** (it is, after all, just a program she chose to run) — so it *can* read R .
3. The trojan **reads R and copies its content** into a second resource R' , which is readable by Bob.
4. **Bob reads R'** , and thereby reads the content of R too — despite never having had permission to access R directly.

The root cause is structural, not a bug: DAC permissions attach to the *user*, and any program that user runs inherits them fully — there is no mechanism distinguishing “Alice, acting deliberately” from “a program Alice unknowingly launched, acting on her identity.”

8.4 Mandatory Access Control (MAC) and Multi-Level Security

Under **MAC**, by contrast, users operate within an organization, and it is the **organization** — a central entity — that decides how data is shared, not individual resource owners. Example: a hospital owns patient records and limits their sharing, often because of external **regulatory/legal requirements**. MAC is usually enforced using **security labels**, leading to **multi-level security (MLS)** models.

8.4.1 The Motivating Problem

The classic MAC problem — confidentiality protection in a **military** setting — predates computers entirely: given information at various sensitivity levels, and users with various degrees of trustworthiness, how do we control access to protect confidentiality? Information has different sensitivity (a war plan versus a cafeteria menu); users have different trustworthiness (a general versus a janitor). **Goal:** prevent sensitive information (e.g., war plans) from reaching untrusted users (e.g., janitors).

8.4.2 Sensitivity Labels

To categorize information by sensitivity, each resource (document, folder, file) is labeled $L = (S, N)$:

- S takes values in a **linearly ordered** set, e.g., $\text{Top Secret} \geq \text{Secret} \geq \text{Confidential} \geq \text{Unclassified}$. *Una relazione binaria $\geq$ su un insieme è un ordine lineare se soddisfa: **antisimmetria** ($a \geq b$ e $b \geq a$ implicano $a = b$), **transitività** ($a \geq b$ e $b \geq c$ implicano $a \geq c$), e **proprietà connessa** ($a \geq b$ oppure $b \geq a$, per ogni coppia).*
- N is a set of **need-to-know categories**, from an *unordered* set expressing membership in an interest group (e.g., Crypto, Nuclear, Personnel, Janitorial, ...).

Example: label $(\text{Secret}, \{\text{Nuclear}, \text{Crypto}\})$ marks a resource with sensitive information about both categories; $(\text{Top Secret}, \{\text{Crypto}\})$ marks something even more sensitive, but about Crypto alone.

At creation time, a resource’s originator assigns its sensitivity label according to criteria the MLS model itself leaves unspecified; if a document mixes sensitive and non-sensitive content, the originator must use the *highest* applicable level. When conditions change (e.g., with time), a label may later be **downgraded** (declassified) as content becomes less sensitive.

8.4.3 Clearances and the Dominates Relation

Symmetrically, each **user** is assigned a **clearance** (authorization level) with the same structure, $C = (S, N)$: S is the hierarchical level to which the user has been vetted; N is the set of need-to-know categories in which they are authorized to operate. **Sensitivity labels (on resources) indicate the sensitivity of contained information; clearances (on**

users) indicate the classes of information a user is authorized to access — and need-to-know categories reflect the **principle of least privilege** directly: even within one security level (say, Top Secret), not everyone needs to know *everything* classified at that level.

(S_1, N_1) **dominates** (S_2, N_2) iff $S_1 \geq S_2$ and $N_1 \supseteq N_2$ — **both** conditions are required. Examples: (Secret, {Crypto}) dominates (Confidential, {Crypto}), since Secret $\geq$ Confidential and {Crypto} $\supseteq$ {Crypto}. But (Secret, {Crypto,Nuclear}) does *not* dominate (Top Secret, {Crypto}): despite {Crypto,Nuclear} $\supseteq$ {Crypto}, we have Secret $\not\geq$ Top Secret — **a higher need-to-know set never compensates for an insufficient hierarchical level, and vice versa.**

8.4.4 No Read Up, No Write Down

Access control policies are then defined directly in terms of dominance:

- **No Read Up:** subject s can read resource r only if (S_s, N_s) **dominates** (S_r, N_r) — the subject's clearance must dominate the resource's sensitivity label.
- **No Write Down:** subject s can write to resource r only if (S_r, N_r) **dominates** (S_s, N_s) — the resource's sensitivity label must dominate the subject's clearance.

No Write Down exists specifically to prevent a subject with access to a Top Secret file from copying its content down into an Unclassified file — i.e., laundering sensitive information into a less-protected container.

8.4.5 The Tranquility Principle

An implicit assumption underlies all of this: the **Tranquility Principle** — security labels (and clearances) must *not* be changed arbitrarily, or the whole scheme can be subverted. **Why this matters, concretely:** let s_1 have the highest clearance, s_2 the lowest, r_1 the highest sensitivity, r_2 the lowest. After s_1 legitimately reads r_1 (satisfying No Read Up), suppose s_1 were then allowed to *lower their own clearance* to the minimum and write to r_2 (which would now satisfy No Write Down, since r_2 's low label would dominate s_1 's newly-lowered clearance!). **Both basic properties are individually satisfied at every step, yet confidentiality is completely broken:** s_2 , with the lowest clearance, can now read r_2 — which contains information originally from the highly sensitive r_1 .

8.4.6 Bell–La Padula and Its Shortcoming

No Read Up, No Write Down, and the Tranquility Principle together are the heart of the **Bell–La Padula** security model (1973) — despite its age, still a cornerstone of modern computer security, widely used in military applications.

A well-known shortcoming, stated as a question: can a corporal with *no* clearance overwrite the war plan (associated with the highest sensitivity level)? No Write Down does *not* prevent this — and indeed it should not, since **Bell–La Padula is concerned with confidentiality, not integrity:** writing to a highly sensitive resource by a user without adequate clearance is an *integrity* problem (it is likely to make the resource's content useless), entirely outside Bell–La Padula's scope. This gap is addressed by the **Biba model** (1977), a mathematically similar model but with a different meaning attached to its labels: an **integrity label** on a resource characterizes the *trustworthiness* of the

information it contains, while an integrity label on a subject measures the confidence placed in that subject's ability to *produce or handle* information.

8.5 Worked Example: a Military MLS Scenario

Setup. Security levels: Top Secret $\geq$ Secret $\geq$ Confidential $\geq$ Unclassified. Categories: {Nuclear, Army}.

Subject	Clearance
President	(Top Secret, {Nuclear, Army})
Colonel	(Secret, {Nuclear, Army})
Major	(Confidential, {Army})
Soldier	(Unclassified, {Nuclear})

Resource	Sensitivity label
Army position	(Secret, {Army})
Number of army units	(Confidential, {Army})
Number of nuclear units	(Confidential, {Nuclear})
Cost of nuclear program	(Unclassified, {Nuclear})
Cost of army	(Unclassified, {Army})
Nuclear code	(Top Secret, {Nuclear})

Esercizio 8.3

(Dalle slide del corso) Given the scenario above: (1) Can the president compute the overall defense costs (army + nuclear)? (2) Can the major compute the total number of nuclear and army units? (3) Can the colonel compute the total number of nuclear and army units? (4) Can the colonel change the army position? (5) Can the major change the nuclear code? (6) Can the soldier change the nuclear code? (7) What problem is raised by the previous question?

Soluzione: every answer follows from checking **No Read Up** (reading needs the subject's clearance to *dominate* the resource) or **No Write Down** (writing needs the resource's label to *dominate* the subject's clearance).

1. **Yes.** Computing costs means *reading* “Cost of nuclear program” (Unclassified, {Nuclear}) and “Cost of army” (Unclassified, {Army}). The President's clearance (Top Secret, {Nuclear, Army}) dominates both — Top Secret $\geq$ Unclassified and {Nuclear, Army} $\supseteq$ each category set. Read allowed by No Read Up (reading *down* is always fine).
2. **No.** This needs reading “Number of nuclear units” (Confidential, {Nuclear}), but the Major's clearance is (Confidential, {Army}) — same hierarchical level, but {Army} $\not\supseteq$ {Nuclear}. **Fails need-to-know**, even though the security *level* alone would be sufficient.
3. **Yes.** The Colonel's clearance (Secret, {Nuclear, Army}) dominates (Confidential, {Army}) *and* (Confidential, {Nuclear}) — Secret $\geq$ Confidential, and the category set covers both. Allowed by No Read Up (again, reading *down*).

4. **No.** Changing (writing to) the army position needs (Secret, {Army}) to *dominate* the writer's clearance (No Write Down). The Colonel's clearance is (Secret, {Nuclear, Army}) — same level (Secret $\geq$ Secret, fine), but {Army} $\not\supseteq$ {Nuclear, Army}: the *resource's* category set does not cover the *subject's*. **Fails need-to-know, despite matching hierarchical levels.**
5. **No.** Writing the nuclear code needs (Top Secret, {Nuclear}) to dominate the writer's clearance. The Major's clearance, (Confidential, {Army}), fails on *both* counts: Top Secret $\not\geq$ Confidential (true) **and** {Nuclear} $\not\supseteq$ {Army} (false). **Fails need-to-know** (the category mismatch), even though the hierarchical levels alone would permit it.
6. **Yes.** The nuclear code's label, (Top Secret, {Nuclear}), must dominate the Soldier's clearance, (Unclassified, {Nuclear}): Top Secret $\geq$ Unclassified, and {Nuclear} $\supseteq$ {Nuclear} — **both** hold, so **No Write Down is satisfied**, and the write is allowed.
7. **This is exactly Bell–La Padula's known shortcoming** (see above): a Soldier with the *lowest* clearance level is permitted to overwrite the nuclear code, one of the most sensitive resources in the entire scenario. No Write Down protects *confidentiality* only — it says nothing about whether the Soldier's write is *trustworthy* — so this is an **integrity** problem, entirely outside Bell–La Padula's scope (and exactly what the Biba model exists to address).

8.6 MAC: Pros, Cons, and Covert Channels

- **Pros: not vulnerable to trojans**, precisely because of the No Write Down property (Section 8.3's trojan attack relied on a low-privilege process silently writing to a resource a higher-privilege user could read — No Write Down blocks exactly that direction); **rigid**, in the sense that this makes the situation easy to keep under control.
- **Cons:** that same **rigidity** may hinder business continuity; and information leakage remains possible through a **covert channel**.

Covert channel: a path for the flow of information between subjects within a system, using system resources that were *not* designed to be used for inter-subject communication at all.

Remarkably, Bell–La Padula's own two key properties — No Read Up and No Write Down — can themselves be exploited to build a covert channel, letting a high-clearance subject leak information to a low-clearance one *without ever violating either property*:

1. A **low-level subject** (e.g., a soldier) creates a dummy object, `dummy.obj`, at its *own* (low) security level.
2. Its **high-level accomplice** (e.g., a general, colluding with the soldier) either **upgrades** `dummy.obj`'s sensitivity level, or **leaves it unchanged** — this choice is the “bit” the general wants to transmit.
3. The low-level subject later **attempts to read** `dummy.obj`.

- If the object was **upgraded**: the read **fails** (No Read Up blocks it) — the soldier infers, say, a “0” bit was sent.
- If the object was **left unchanged**: the read **succeeds** — the soldier infers a “1” bit was sent.

Either outcome — success or failure of the read — discloses the general’s action, and that is exactly one bit of information having flowed from high to low. Repeating this exchange (create a fresh dummy object, upgrade-or-not, attempt to read, observe success/failure) as many times as needed lets the high-level accomplice transmit an *arbitrary* amount of information to the low-level subject — one bit at a time, without either party ever performing an action Bell–La Padula’s own rules forbid.

8.7 Closing DAC vs. MAC

Esercizio 8.4

(Dal file di domande di ripasso — parti (c)–(e); le parti (a)–(b) sul confused deputy sono state svolte nella prima parte di questa lezione, Sezione 8.2.3) (c) explain the notion of Discretionary Access Control (DAC), (d) explain the notion of Mandatory Access Control (MAC), and (e) discuss the main advantages and disadvantages between DAC and MAC.

Soluzione:

- (c) **DAC** (Section 8.3): the *owner* of a resource decides how it is shared — there is no central deciding authority, and rights can be granted by whoever currently holds them (typically enforced via ACLs, with groups used to keep administration manageable).
- (d) **MAC** (Section 8.4): a *central, organizational* authority — not individual owners — decides how resources can be shared, typically enforced via security labels (sensitivity labels on resources, clearances on subjects) and the No Read Up / No Write Down properties.
- (e) **DAC**: flexible and easy to implement (pro), but subjective (rests entirely on individual owners’ judgement), assumes users never make mistakes (unrealistic), and is vulnerable to trojans — a low-privilege attacker can leak information out of a resource simply by tricking its legitimate, higher-privilege owner into running malicious code on their behalf (con). **MAC**: centrally enforced and immune to that specific trojan attack, since No Write Down blocks the disclosure direction the trojan needs (pro), but rigid — potentially hindering legitimate business continuity — and still leaks information via covert channels, which do not violate either of its core properties (con).

8.8 Role-Based Access Control (RBAC)

8.8.1 Motivation

DAC is usually too flexible, MAC is usually too restrictive. This motivates a search for models that are both flexible *and* expressive: **Role-Based Access Control (RBAC)** is widely considered superior to both.

8.8.2 A University Example: the Naive Approach

Setting: users Alice, Bob, Charlie, David, Eve, Fred, Greg; permissions **GrantTenure**, **AssignGrades**, **AssignHWScores**, **ReceiveBenefits**, **UseGym**, **Register4Courses** (a **permission** being an abstraction of an (action, resource) pair). Suppose: Alice has **GrantTenure**, **AssignGrades**, **ReceiveBenefits**, **UseGym**; Bob and Charlie have **GrantTenure**, **AssignGrades**, **UseGym**; David has **AssignHWScores**, **Register4Courses**, **UseGym**; Eve has **ReceiveBenefits**, **UseGym**; Fred has **Register4Courses**, **UseGym**; Greg has just **UseGym**.

The **naive** implementation is a flat user–permission table (one row per (user, permission) pair). **The problem: administration is difficult.** Suppose Alice resigns from the Promotion Committee and becomes an ordinary Faculty Member: an administrator must find and update *every single row* involving Alice, one permission at a time, very carefully — easy to get wrong, and it does not scale.

8.8.3 A Better Answer: Roles, UA, and PA

Looking at the permissions above, they naturally group by *profile*: Alice is a **Promotion Committee (PC) Member**; Bob and Charlie are **Faculty** members; David is a **Teaching Assistant (TA)**; Eve is a **University Employee**; Fred is a **Student**; Greg is simply a **University Member**. **Words in boldface here are exactly the roles** to which *both* users and permissions get associated — instead of assigning permissions directly to users, RBAC introduces two separate relations:

- **User Assignment (UA)**: which users hold which roles;
- **Permission Assignment (PA)**: which permissions are granted to which roles.

User u has permission $p \iff \exists \text{ role } r \text{ s.t. } (u, r) \in UA \text{ and } (r, p) \in PA.$

This one indirection fixes the administration problem. Reconsidering Alice's resignation: (1) delete (Alice, PCMember) from UA; (2) add (Alice, Faculty) to UA; (3) **PA is left completely unchanged.** One relation, two small edits — versus editing every permission row by hand.

8.8.4 An Even Better Answer: a Role Hierarchy

Looking at PA closely reveals redundancy: the permission set for **PCMember** is a *superset* of **Faculty**'s; **Faculty**'s is a superset of **TA**'s and of **UEmployee**'s; and so on. Rather than list every inherited permission explicitly for every senior role, RBAC allows a **role hierarchy (RH)**: a partial order $\succeq$ on roles, so that a senior role automatically inherits everything a junior role can do.

User u has permission $p \iff \exists \text{ roles } r, r' \text{ s.t. } (u, r') \in UA, r' \succeq r, \text{ and } (r, p) \in PA.$

By convention, **more senior (powerful) roles are drawn toward the top** of a role-hierarchy diagram, junior roles toward the bottom.

8.8.5 Core RBAC: the Formal Model

Let U, R, P, S be the sets of users, roles, permissions, and sessions, respectively.

- $UA \subseteq U \times R$: a many-to-many **user-role assignment** relation.

- $PA \subseteq P \times R$: a many-to-many **permission-role assignment** relation.
- $user : S \rightarrow U$: maps each session to a user — **fixed** for the session's whole lifetime.
- $roles : S \rightarrow 2^R$: maps each session s to a set of *currently active* roles, with $\{r \mid (user(s), r) \in UA\} \supseteq roles(s)$ — **this** function *can* change during the session, since a user may activate or deactivate roles they hold at will.
- $permissions : S \rightarrow 2^P$: $permissions(s) = \bigcup_{r \in roles(s)} \{p \mid (p, r) \in PA\}$.

Letting a user activate only a subset of their roles per session (rather than always all of them at once) is precisely what helps implement the **principle of least privilege** in practice: a user with a powerful role need not run *every* task with that role's full power switched on.

8.8.6 Worked Example: Core RBAC (No Hierarchy)

Consider three roles with *no* hierarchy between them: **Resident** (permissions: Ops₁ on Object₁, Ops₂ on Object₂; users: User1, User2, User3), **Physician** (Ops₃/Object₃, Ops₄/Object₄; users: User4, User5, User6), **Cardiologist** (Ops₅/Object₅, Ops₆/Object₆; users: User7, User8, User9).

$UA = \{(Useri, Resident) \mid i = 1, 2, 3\} \cup \{(Useri, Physician) \mid i = 4, 5, 6\} \cup \{(Useri, Cardiologist) \mid i = 7, 8, 9\}$.

$PA = \{(Ops_1/Obj_1, Resident), (Ops_2/Obj_2, Resident), (Ops_3/Obj_3, Physician), (Ops_4/Obj_4, Physician), (Ops_5/Obj_5, Cardiologist), (Ops_6/Obj_6, Cardiologist)\}$.

For sessions $S = \{s_1, s_2\}$ with $user(s_1) = User5$, $user(s_2) = User9$: since Core RBAC has *no* hierarchy, each user's only possible role is their one direct UA assignment, so the upper bound is exact: $roles(s_1) = \{Physician\}$, and $permissions(s_2) = \{(Ops_5, Obj_5), (Ops_6, Obj_6)\}$ (User9's only role is Cardiologist).

The equivalent **access matrix**:

	Obj ₁	Obj ₂	Obj ₃	Obj ₄	Obj ₅	Obj ₆
User1,2,3 (Resident)	Ops ₁	Ops ₂				
User4,5,6 (Physician)			Ops ₃	Ops ₄		
User7,8,9 (Cardiologist)					Ops ₅	Ops ₆

8.8.7 Hierarchical RBAC: the Formal Model

Same $U, R, P, S, UA, PA, user$ as Core RBAC, plus $RH \subseteq R \times R$: a **partial order** on R (written $r \succeq r'$). Both remaining functions are updated to follow the hierarchy:

$$roles(s) \subseteq \{r \mid \exists r' \text{ s.t. } r' \succeq r \text{ and } (user(s), r') \in UA\},$$

$$permissions(s) = \bigcup_{r \in roles(s)} \{p \mid \exists r' \text{ s.t. } r \succeq r' \text{ and } (p, r') \in PA\}.$$

In words: a user may activate not only their directly-assigned role, but *any role that role dominates* (its juniors); and once a role is active, it carries *all* permissions assigned to itself *and* to every role it dominates.

8.8.8 Worked Example: Hierarchical RBAC

Same U, R, P, UA, PA as the Core RBAC example above, now *with* a hierarchy: Cardiologist $\succeq$ Physician $\succeq$ Resident (so, by transitivity, Cardiologist $\succeq$ Resident too) — equivalently, Resident's permissions are inherited by Physician, and Physician's (hence also Resident's) are inherited by Cardiologist.

For the same sessions: the upper bound for $roles(s_1)$ (User5, directly Physician) is {Physician, Resident} — Physician itself, plus everything it dominates. For $permissions(s_2)$ (User9, directly Cardiologist), the upper bound for $roles(s_2)$ is {Cardiologist, Physician, Resident}, so

$$permissions(s_2) = \{(Ops_1, Obj_1), \dots, (Ops_6, Obj_6)\} \quad \text{— all six permissions.}$$

The equivalent access matrix now reflects inheritance directly:

	Obj ₁	Obj ₂	Obj ₃	Obj ₄	Obj ₅	Obj ₆
User1,2,3 (Resident)	Ops ₁	Ops ₂				
User4,5,6 (Physician)	Ops ₁	Ops ₂	Ops ₃	Ops ₄		
User7,8,9 (Cardiologist)	Ops ₁	Ops ₂	Ops ₃	Ops ₄	Ops ₅	Ops ₆

Notice the difference from Core RBAC is entirely in what a *senior* role's users can now do — a Cardiologist's own direct grant never changed, but the hierarchy means their session can carry every junior role's permissions too, without PA ever having to list them redundantly.

8.8.9 Mathematical Digression: Binary Relations, Revisited

Let S be a set and $R \subseteq S \times S$ a binary relation over S . R is:

- **reflexive** iff $\forall e \in S : (e, e) \in R$;
- **antisymmetric** iff $\forall e, e' \in S$: if $(e, e') \in R$ and $(e', e) \in R$ then $e = e'$;
- **transitive** iff $\forall e, e', e'' \in S$: if $(e, e') \in R$ and $(e', e'') \in R$ then $(e, e'') \in R$.

A role hierarchy RH is exactly a reflexive, antisymmetric, transitive relation — i.e., a **partial order**. **Compare this carefully with the security-level order S used for MLS** (Section 8.4): there, S additionally satisfies the **connex property** ($a \geq b$ or $b \geq a$, for *every* pair), making it a *linear* order — any two security levels are always comparable. A role hierarchy drops this requirement: two roles (e.g., Nurse and Lab Technician) can simply be **incomparable**, neither senior to the other, which a strictly linear chain of levels could never express.

A **Hasse diagram** is a way of drawing a finite partially ordered set via its **transitive reduction**: the directed graph with the same vertices and as *few* edges as possible, such that every directed path present in the original relation is still reachable in the reduction (i.e., it omits edges implied by transitivity, like Cardiologist $\rightarrow$ Resident when Cardiologist $\rightarrow$ Physician $\rightarrow$ Resident already exists) — exactly the clean, minimal-arrow diagrams used above for the Physician/Cardiologist/Resident hierarchy.

8.9 Constrained RBAC: Separation of Duty

Separation of Duty (SoD), also known as the **four-eyes principle**, is a widely recognized constraint capturing conflict-of-interest policies that restrict the authority of any *single* entity — key to fraud prevention. Classic example: a single person should not be allowed to *both* approve a check *and* cash it — whatever roles/permissions structure is built, an SoD constraint additionally forbids any one user from holding *both* capabilities at once. *Riferimento: D.F. Ferraiolo, R. Sandhu, S. Gavrila, D.R. Kuhn, R. Chandramouli, “Proposed NIST Standard for Role-Based Access Control,” ACM Transactions on Information and System Security, vol. 4, no. 3, 2001, pp. 224–274.*

8.10 RBAC: Pros and Cons

- **Pros:** easy to grasp; easy to manage *in principle* — roles decouple digital identities from permissions, so a new subject is simply assigned roles rather than having access decided resource-by-resource, and authorization is easy to revoke by removing roles; easy to tell, *through* roles, which permissions a subject has and *why*.
- **Cons:** deciding the right **granularity** of roles is genuinely difficult (should modifying client information and deleting a client be one role or two? is a role’s authorization still appropriate, and not overly broad, for *every* subject holding it?); and a role’s **meaning can be fuzzy** — an employee’s actual position in the company may not line up neatly with the RBAC roles defined for the system, a common source of misunderstanding between administrators and managers who assign them.

8.11 Summary

- Access control is the Guard/PDP mediating every request against a Policy, with isolation boundaries making the Guard un-bypassable and the Audit Log tamper-proof (Section 8.1.1).
- The **ACM/ACL/Capability** trio (per-object vs. per-subject views of the same matrix) is enough to explain a real, subtle vulnerability class: the **confused deputy** (Section 8.2.3) — and to show precisely *why* capabilities, not ACLs, close it (rights travel with the holder, not silently with whichever process acts on their behalf).
- **DAC** (owner-controlled, flexible, trojan-vulnerable) and **MAC** (centrally-controlled, rigid, covert-channel-vulnerable) sit at opposite ends of a flexibility/control trade-off; **Bell–La Padula’s** No Read Up/No Write Down solves confidentiality but explicitly *not* integrity (Section 8.4).
- **RBAC** threads the needle with one extra layer of indirection — **UA** and **PA** instead of a flat user-permission table — turning routine reassignment into a couple of UA edits with PA untouched, and a **role hierarchy** on top removes redundant listing of inherited permissions entirely.
- Across every model in this lesson, the same principle keeps resurfacing in a new guise: **least privilege** — confinement in OS design, need-to-know in MLS, granular role activation per session in RBAC.

Esercizio 8.5

(Domanda di ripasso, dalle slide del corso) How can access control mitigate command injection attacks?

Soluzione: access control cannot *prevent* the injection itself (that is an input-validation problem — see the SQL-injection discussion in the domande file, and the **finger**/Morris-worm case above for the same root cause with buffer bounds) — but the **principle of least privilege** (Section 8.1.1) directly limits the *damage* a successful injection can do. If the process a command gets injected into runs with only the minimal privileges its legitimate task actually requires (exactly the “give the sysadmin an ordinary account too, and reserve **root** only for tasks that truly need it” example already given for OS security), then even a fully successful injection is confined to whatever that narrow privilege set allows — it cannot read, write, or execute anything beyond it. Command injection turns into a serious, system-wide compromise specifically when the vulnerable process is over-privileged relative to what it actually needs to do.

Domande di ripasso rimanenti dalle slide, già risposte per intero altrove in questa sezione: definizione di AC e sua architettura (→ Sezione 8.1.1), ACM/ACL/Capabilities (→ l'esercizio in Sezione 2 di questa lezione), DAC e MAC con No Read Up/No Write Down (→ Sezioni 8.3–8.4), vantaggi/svantaggi DAC vs. MAC (→ l'esercizio all'inizio di questa parte), RBAC e la semplificazione amministrativa (→ questa sezione), principio di least privilege, confused deputy, trojan, e covert channel (tutti definiti con esempio completo nelle sezioni indicate sopra).